Labelled Process Logic

YUANRUI ZHANG, College of Software, Nanjing University of Aeronautics and Astronautics, China

This paper develops a complete labelled proof-theoretic framework for process logic — an extension of dynamic logic in which formulas specify properties of execution traces rather than only final states. The main difficulty is that first-order process logic must reason about concrete computations while preserving temporal information along regular-program traces. Existing compositional calculi cover important fragments but do not provide a complete treatment of full first-order process logic over regular programs. We address this difficulty by enriching process-logic formulas with labels that explicitly record trace and update information during derivations. Based on this construction, we define cyclic labelled proof systems for both propositional and first-order process logic, respectively denoted by G3PPL and G3FOPL. We study their soundness and completeness. Their soundness follows from a global progress condition: an invalid cyclic derivation would induce infinitely many strict decreases in a natural-valued counter-example measure. The completeness of G3PPL is established by deriving the axioms and rules of the Hilbert system for propositional process logic, while the completeness of G3FOPL is obtained relative to the underlying arithmetic theory through the standard arithmetical encoding used for first-order dynamic logic. The result is a uniform framework for process logic in which, for the first time, trace-based program properties and first-order computations can be handled within the same proof structure.

CCS Concepts: • **Theory of computation → Modal and temporal logics**; **Programming logic**; **Logic and verification**; *Proof theory*.

Additional Key Words and Phrases: Process Logic, Program Logic, Temporal Properties, Labelled Proof Systems, Cyclic Proofs, Program Verification

ACM Reference Format:

Yuanrui Zhang. 2018. Labelled Process Logic. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 57 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Process logic [16] extends dynamic logic [17] from reasoning about the final states of program executions to reasoning about the whole execution process. In ordinary dynamic logic, a formula $[\alpha]\phi$ states that every terminating execution of program α ends in a state satisfying ϕ. This view has supported a wide range of program logics, including logics for programming languages [3, 14], process algebra [5, 6], synchronous systems [36, 37], hybrid systems [25, 26], and many related verification frameworks (e.g. [11, 27, 29, 30, 37]). Process logic strengthens this setting by allowing ϕ to describe trace properties using linear temporal logic (LTL [28]): a formula $[\alpha]\phi$ states that every execution trace of α satisfies the temporal property ϕ. ϕ can speak about what happens during the execution rather than only after termination [16]. This makes process logic a natural formalism for specifications in which intermediate behaviour is essential.

The propositional theory of process logic (PPL) has been thoroughly studied, including decidability and completeness [16]. The first-order level, however, is more delicate. First-order process logic (FOPL) interprets atomic programs as concrete

Author's Contact Information: Yuanrui Zhang, yuanruizhang@nuaa.edu.cn, College of Software, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Logic	Propositional level	First-order level	LTL operators	Program model	Complete proof system
PPL [16]	√	×	Full	Regular programs	√
DLT [4]	×	√	□	Deterministic while programs	√
DLT_d [24]	×	√	□	Regular programs	×
DLT_d^2 [19]	×	√	□, ◇	Regular programs	Partial [1]
DTL [2]	×	√	Full	Deterministic while programs	√
LPL	√	√	Full	Regular programs	√

Table 1. Comparison of propositional process logic, trace-based dynamic logics, and labelled process logic (LPL). Here, × in the final column means that completeness is not established yet. For a first-order logic, its completeness is related to some arithmetic theory.

computations, such as assignments over an arithmetic structure, and interprets atomic propositions as predicates over program states. This expressiveness is necessary for reasoning about actual computational systems, but it also exposes a structural difficulty: the proof system of PPL does not lift smoothly to the first-order case (cf. [16, 17]). In particular, for a sequential program $\alpha\,;\beta$, a proof of $[\alpha\,;\beta]\phi$ would require a compositional way to split the trace property ϕ across the executions of α and β. Such a split depends heavily on the shape of the LTL formula involving the operators **n** (next), □ (always), ◇ (eventually), and U (until).

Several lines of work address fragments of this difficulty. Beckert's sequent calculus for first-order dynamic logic (FODL) with trace modalities (DLT) [4] studies formulas of the form $[|\alpha|]\phi$, which correspond to the special process-logic case $[\alpha]\Box\phi$. Its proof system is complete for deterministic while programs. The crucial compositional rule for sequence programs reduces $[\alpha\,;\beta]\Box\phi$ to obligations about the trace of α and the subsequent behaviour of β, but this rule is specific to the always modality □. Later, [24] employs the trace modality and inherits the proof system of DLT in a differential temporal dynamic logic (to avoid ambiguity, we name it DLT_d) for regular programs, but it remains open whether its proof system is complete. [35] shows, by means of a counterexample, that the proof system of DLT/DLT_d is insufficient to derive every valid temporal property of general sequential regular programs and thus needs to be improved. A variant of DLT_d which we name DLT_d^2 [19] extends DLT with a richer collection of temporal modalities, so that it can reason about properties such as $[\alpha]\Diamond\phi$. Moreover, [1] establishes its completeness for the deterministic fragment of regular programs. But ${\text{DLT}_d}^2$ still cannot handle the temporal operator until U. Based on DLT, dynamic trace logic (DTL) [2] first proposed a complete calculus for all LTL operators, but only for deterministic while programs rather than regular programs.

The comparison of these works with ours is illustrated in Table 1. Consequently, none of these systems above provides a complete proof-theoretic treatment of full first-order process logic over regular programs.

This paper takes a different route. Instead of directly searching for a suitable set of compositional rules for temporal formulas, we enrich process-logic formulas with *labels* that explicitly record the relevant execution information. A labelled formula has the form $\sigma : \phi$, where σ represents a trace-like program configuration and ϕ is a process-logic formula. Labels allow the intermediate execution results of programs to accumulate at each derivation step, so that the proof of a trace property can be deferred until the entire program execution has been unfolded. This circumvents the difficulty encountered in previous approaches: the proof no longer needs to split every sequential trace by a special-purpose temporal rule at the point where the sequence is encountered. The labelled form is conservative over ordinary process logic: an unlabelled formula can be recovered by using a free trace variable as its label.

With this labelling technique, we manage to develop a complete logical framework for both propositional and first-order process logic: the propositional system, G3PPL, extends labelled sequents for both modal logics [22] and propositional dynamic logic [12], but adapts them to the trace semantics of process logic; the first-order system, G3FOPL, is then developed based on G3PPL by building specific rules for explicit program updates in place of their counterparts for abstract labels. Since regular programs include iteration, proof search may generate cyclic derivations. We therefore equip the labelled systems with cyclic proof conditions in the style of cyclic proof theory [8, 9, 31]. The cyclic condition provides the mechanism that distinguishes sound cyclic reasoning from arbitrary finite graphs with unsound back-links.

The proposed labelled calculus provides a uniform and modular framework for reasoning about process-logic formulas at both the propositional and first-order levels. Compared to the Hilbert-style proof system for PPL (see Table 2), it is more concise: it requires only 5 rules for the temporal operators (see Table 4) in place of the 15 rules (i)–(xv) (Table 2). This compactness also benefits automated proof search: rather than choosing among rules (i)–(xv) and constructing suitable intermediate formulas, a proof-search procedure need only apply the rules (**suf** R), (**suf** L) to unfold each temporal formula uniformly.

To the best of our knowledge, our work is the first to provide a complete proof system for first-order process logic over regular programs.

The contributions of this paper are as follows.

- We introduce labelled versions of process-logic formulas, defining trace labels for PPL and update-based labels for FOPL.
- We define cyclic labelled proof systems G3PPL and G3FOPL, sharing a common labelled framework while building rules specific to propositional and first-order process logic.
- We prove the soundness of the cyclic labelled systems by reducing a progressing infinite derivation path to an infinite descent in a well-founded ordering.
- We prove the completeness of G3PPL by deriving the proof system of PPL, and establish the corresponding first-order result for G3FOPL through the usual arithmetical encoding of FODL.

The rest of this paper is organized as follows. Section 2 first recalls the syntax, semantics, and proof-theoretic background of process logic. Section 3 then provides an overview: it motivates our labelled approach by showing in detail how classical and existing trace-based program logics are embedded into process logic and what their limitations are, describes the high-level design of our work, and compares our work with other recent temporal program logics. Section 4 introduces labelled PPL formulas and the proof system G3PPL, followed by the analysis and proofs of its soundness and completeness in Section 5 and 6 respectively. The first-order labelled system G3FOPL is developed afterwards in Section 7, where we also analyze its soundness and completeness. Section 8 concludes this paper and discusses about future work. The appendix contains the detailed derivations and auxiliary proofs used by the main theorems.

2 Brief Introduction to Process Logic

This section recalls the parts of process logic that are used in the rest of the paper. We first review propositional process logic [16], including its syntax, trace semantics, and proof system, and then describe the first-order instance over arithmetic that is used when defining labelled FOPL formulas.

2.1 Propositional Process Logic

Definition 2.1 (Syntax of PPL Formula). *Let a range over atomic programs and p range over atomic propositions. The syntax of programs, trace formulas, and state formulas is given by:*

$$\alpha =_{df} a \mid \phi? \mid \alpha;\alpha \mid \alpha \cup \alpha \mid \alpha^*,$$
$$\pi =_{df} \phi \mid \mathbf{f}\,\pi \mid \pi\ \mathbf{suf}\ \pi,$$
$$\phi =_{df} true \mid p \mid \neg\phi \mid \phi \wedge \phi \mid [\alpha]\pi,$$

An atomic program a is also called an *action*. The program $\phi?$ is a test, $\alpha;\beta$ is sequential composition, $\alpha \cup \beta$ is nondeterministic choice, and α^* is finite iteration, also called a *star program*. The modality $[\alpha]\pi$ states that every trace generated by executing α satisfies the trace formula π.

true is the Boolean true. A trace formula π is evaluated over a finite trace rather than only over a single state. The formula $\mathbf{f}\,\pi$ states that π holds at the first state of the current trace, while $\pi_1\ \mathbf{suf}\ \pi_2$ is the process-logic until-like suffix connective (cf. [16]): along a proper suffix of the current trace, π_2 eventually holds and π_1 holds on the intermediate suffixes. The standard temporal operators of LTL can be expressed by $\mathbf{f}$ and $\mathbf{suf}$ (cf. [16]), for instance:

$$\mathbf{n}\,\phi =_{df} (\neg true)\ \mathbf{suf}\ \phi,\ \phi\,\mathrm{U}\,\psi =_{df} \psi \vee (\phi \wedge \phi\ \mathbf{suf}\ \psi),$$

where $\mathbf{n}$ denotes the *next* operator and U denotes the *until* operator of LTL. Intuitively, $\mathbf{n}\,\phi$ holds on a trace when ϕ holds from the second state onward, while $\phi\,\mathrm{U}\,\psi$ holds when ψ eventually holds and ϕ holds on all intermediate steps. Other Boolean connectives, such as $\vee$ and $\rightarrow$, are treated as abbreviations in the standard way. We also use the dual modality $\langle\alpha\rangle\pi$, defined as $\neg[\alpha]\neg\pi$, to state that there exists an execution trace of α satisfying π. We introduce a dual operator $\mathbf{fus}$ for the operator $\mathbf{suf}$. Formula $\phi\ \mathbf{fus}\ \psi$ is defined as $\neg(\neg\phi\ \mathbf{suf}\ \neg\psi)$.

The semantics of PPL is based on *worlds*, which abstract program states. A *trace* tr over a set $\mathcal{S}$ of worlds is a finite non-empty sequence of worlds. We introduce a concatenation operator $\cdot$ between any two *strings*, i.e., a sequence of some type of elements. It does not apply only to traces, as seen in later sections. Given two strings str_1 and str_2 with $str_1 = w_1 \ldots w_n$ $(n \geq 1)$ and $str_2 = t_1 \ldots t_m \ldots$ $(m \geq 1)$, $str_1 \cdot str_2$ is partially defined as $w_1 \ldots w_n t_2 \ldots t_m \ldots$ provided that $w_n = t_1$. Sometimes we omit the "$\cdot$" and simply write $str_1 \cdot str_2$ as $str_1 str_2$. For a string str, let str_b and str_e denote its first and last elements. We use $|str|$ to denote the length of a string str.

To align with the later notion of "empty label" (Definition 4.1), we also introduce an "empty string" ϵ, which satisfies $str \cdot \epsilon = \epsilon \cdot str = str$ for any string str.

A *Kripke frame* is a pair $(\mathcal{S}, \mathcal{I})$, where $\mathcal{S}$ is a set of worlds and $\mathcal{I}$ interprets each atomic program as a set of length-two traces and each atomic proposition as a set of worlds.

Definition 2.2 (Semantics of PPL). *Given a Kripke frame $(\mathcal{S}, \mathcal{I})$, the semantics of programs and formulas is defined by extending the function $\mathcal{I}$, using simultaneous induction on the syntactic structures of programs and formulas:*

1. $\mathcal{I}(\phi?) =_{df} \{ss \mid s \in \mathcal{I}(\phi)\}$.
2. $\mathcal{I}(\alpha;\beta) =_{df} \mathcal{I}(\alpha) \cdot \mathcal{I}(\beta)$, *where* $\mathcal{I}(\alpha) \cdot \mathcal{I}(\beta) =_{df} \{tr_1 \cdot tr_2 \mid tr_1 \in \mathcal{I}(\alpha), tr_2 \in \mathcal{I}(\beta)\}$.
3. $\mathcal{I}(\alpha \cup \beta) =_{df} \mathcal{I}(\alpha) \cup \mathcal{I}(\beta)$.
4. $\mathcal{I}(\alpha^*) =_{df} \bigcup_{n\geq 0} \mathcal{I}(\alpha^n)$, *where* $\alpha^0 =_{df} true?$, $\alpha^n =_{df} \alpha;\alpha^{n-1}$ $(n \geq 1)$.
5. $\mathcal{I}(true) =_{df} \mathcal{S}$.
6. $\mathcal{I}(\neg\phi) =_{df} \mathcal{S} \setminus \mathcal{I}(\phi)$.
7. $\mathcal{I}(\phi \wedge \psi) =_{df} \mathcal{I}(\phi) \cap \mathcal{I}(\psi)$.

8. $\mathcal{I}(\mathbf{f}\,\phi) =_{df} \{tr \mid tr_b \in \mathcal{I}(\phi)\}$.
9. $\mathcal{I}(\phi\,\mathbf{suf}\,\psi) =_{df} \{tr \mid \exists tr'.tr' \prec_s tr \wedge tr' \in \mathcal{I}(\psi) \wedge (\forall tr''.tr' \prec_s tr'' \prec_s tr \rightarrow tr'' \in \mathcal{I}(\phi))\}$, *where* $tr_1 \prec_s tr_2$ *means that* tr_1 *is a proper suffix of* tr_2.
10. $\mathcal{I}([\alpha]\phi) =_{df} \{tr \mid$ *for all* $tr' \in \mathcal{I}(\alpha)$, $tr \cdot tr' \in \mathcal{I}(\phi)\}$.

The clauses above follow the intended trace interpretation of regular programs and process formulas. Tests produce stuttering traces when their condition holds; sequence composes compatible traces; choice takes unions; and iteration takes the union over all finite unfoldings. Boolean connectives are interpreted set-theoretically. The trace operators inspect the beginning and suffixes of a trace, and $[\alpha]\phi$ holds of a trace precisely when appending any trace generated by α yields a trace satisfying ϕ.

Given a Kripke frame $(\mathcal{S}, \mathcal{I})$, a trace $tr \in \mathcal{S}^*$ *satisfies* a formula ϕ, written $tr \models_{(\mathcal{S},\mathcal{I})} \phi$ or simply $tr \models \phi$, if $tr \in \mathcal{I}(\phi)$. State satisfaction is included as the special case of traces of length one.

The proof system of PPL is recalled in Table 2. It consists of the usual propositional dynamic logic (PDL) axioms and rules (cf. [17]), together with the additional rules for process-logic trace operators. The non-primitive operators $\bar{\mathbf{n}}$ and **fin** appearing in the rules are defined as follows:

$$\bar{\mathbf{n}}\,\phi =_{df} \neg\,\mathbf{n}\,\neg\phi, \quad \mathbf{fin} =_{df} \bar{\mathbf{n}}\,\mathit{false} \vee (\mathit{true}\,\mathbf{suf}\,(\mathit{true} \wedge \bar{\mathbf{n}}\,\mathit{false})),$$

where $\bar{\mathbf{n}}$ is the dual of $\mathbf{n}$; $\bar{\mathbf{n}}\,\phi$ holds on a trace when it has a next state from which ϕ holds onwards or when it does not have a next state; and **fin** intuitively means that the trace has finite length.

For further explanation of these additional rules, see [16].

The following result is directly from [16].

Proposition 2.1. *The proof system of PPL in Table 2 is sound and complete.*

2.2 First-order Process Logic

First-order process logic (FOPL) enriches PPL by interpreting atomic programs as concrete assignment steps over an arithmetic structure and atomic propositions as first-order predicates over program states. This expressiveness is sufficient to capture a wide range of computational systems: first-order dynamic logic (FODL) underlies verification frameworks for Java [3], hybrid systems [26], and synchronous programs [36, 37], among others.

Studies of FODL often consider a first-order structure in which first-order quantification is allowed [17]. For simplicity, we consider a basic version of the arithmetic theory over the integers, named $\mathfrak{A}$, as the first-order structure for FOPL. $\mathfrak{A}$ is expressive enough to capture most interesting computational structures in computer systems. The results of this paper can also be extended easily to other first-order structures.

The setting of $\mathfrak{A}$ is as follows. Let *Var* be the set of variables over the integer set $\mathbb{Z}$. We use letters u, v to range over the variables of *Var*. An *arithmetic expression* (simply "expression") e is defined recursively as follows:

$$e =_{df} u \mid n \mid e \bowtie e,$$

where $u \in \mathit{Var}$; $n \in \mathbb{Z}$ is a constant; and $\bowtie \in \{+, -, *, \div\}$ represents one of the usual arithmetic operators. A world is a mapping from *Var* to $\mathbb{Z}$. Given a world $s : \mathit{Var} \rightarrow \mathbb{Z}$, $s[u \mapsto i]$ returns a mapping that maps u to the value $i \in \mathbb{Z}$ and maps any other variable v to the value $s(v)$. Given an expression e, the *evaluation* of e by s, denoted by $s(e)$, is defined in the usual way: $s(n) =_{df} n$; $s(e_1 \bowtie e_2) =_{df} s(e_1) \bowtie s(e_2)$ for any e_1, e_2.

No.	Rule or axiom
PDL part	
(1)	$[\alpha](\phi \to \psi) \to ([\alpha]\phi \to [\alpha]\psi)$
(2)	$[\alpha](\phi \wedge \psi) \leftrightarrow [\alpha]\phi \wedge [\alpha]\psi$
(3)	$[\alpha \cup \beta]\phi \leftrightarrow [\alpha]\phi \wedge [\beta]\phi$
(4)	$[\alpha;\beta]\phi \leftrightarrow [\alpha][\beta]\phi$
(5)	$[\psi?]\phi \leftrightarrow (\psi \to \phi)$
(6)	$[\alpha^*]\phi \leftrightarrow \phi \wedge [\alpha][\alpha^*]\phi$
(7)	$\phi \wedge [\alpha^*](\phi \to [\alpha]\phi) \to [\alpha^*]\phi$
(MP)	$\dfrac{\psi \quad \psi \to \phi}{\phi}\ (MP)$
(Gen)	$\dfrac{\phi}{[\alpha]\phi}\ (Gen)$
PPL-specific part	
(i)	$\mathbf{f}(\phi \vee \psi) \leftrightarrow \mathbf{f}\,\phi \vee \mathbf{f}\,\psi$
(ii)	$\mathbf{f}\,\neg\phi \leftrightarrow \neg\,\mathbf{f}\,\phi$
(iii)	$(\phi\ \mathbf{suf}\ \psi) \vee (\phi\ \mathbf{suf}\ \chi) \leftrightarrow \phi\ \mathbf{suf}\ (\psi \vee \chi)$
(iv)	$(\phi\ \mathbf{suf}\ \psi) \wedge (\chi\ \mathbf{suf}\ \omega) \leftrightarrow (\phi \wedge \chi)\ \mathbf{suf}\ (\psi \wedge \omega) \vee (\phi \wedge \chi)\ \mathbf{suf}\ (\phi \wedge \omega \wedge \phi\ \mathbf{suf}\ \psi) \vee (\phi \wedge \chi)\ \mathbf{suf}\ (\psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega)$
(v)	$\neg(\phi\ \mathbf{suf}\ \psi) \leftrightarrow \neg(\mathit{true}\ \mathbf{suf}\ \psi) \vee (\neg\psi)\ \mathbf{suf}\ (\neg\phi \wedge \neg\psi)$
(vi)	$\phi\ \mathbf{suf}\ \psi \leftrightarrow \mathbf{n}\,\psi \vee \mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$
(vii)	$\phi\ \mathbf{suf}\ (\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \leftrightarrow \mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$
(viii)	$\phi\ \mathbf{suf}\ (\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \leftrightarrow \phi\ \mathbf{suf}\ (\phi \wedge \mathbf{n}\,\psi)$
(ix)	$\mathbf{f}\,\mathbf{n}\,\mathit{true} \leftrightarrow \mathit{false}$
(x)	$\neg\phi \wedge \mathbf{f}\,\phi \to \mathbf{n}\,\mathit{true}$
(xi)	$p \leftrightarrow \mathbf{f}\,p$, where p is an atomic proposition
(xii)	$\mathbf{f}\,\phi \wedge \langle\alpha\rangle\psi \leftrightarrow \langle\alpha\rangle(\mathbf{f}\,\phi \wedge \psi)$
(xiii)	$\mathbf{n}\langle\alpha\rangle\phi \leftrightarrow \mathbf{n}\,\mathit{true} \wedge \langle\alpha\rangle\,\bar{\mathbf{n}}\,\phi$
(xiv)	$\langle\alpha\rangle\mathit{true} \to \mathbf{fin}$
(xv)	$((\mathbf{n}\,\phi \to \phi)\ \mathbf{suf}\ \phi) \to \mathbf{n}\,\phi$

Table 2. Axioms and rules of propositional process logic.

A formula in FOPL is defined just as in Definition 2.1, except that an atomic program is an assignment $u := e$, where $u \in \mathit{Var}$ is a variable and e is an expression; an atomic formula is a predicate of the form $e_1 \triangle e_2$, where $\triangle$ is one of the usual binary integer relations in $\{<, \leq, >, \geq, =\}$; and, given an FOPL formula ϕ, $\forall u.\phi(u)$ is also a formula quantified by the operator $\forall$ over a variable $u \in \mathit{Var}$. $\forall$ has the usual dual operator $\exists$. The evaluation of a predicate by s is defined in the usual way: $s(e_1 \triangle e_2) =_{df} s(e_1) \triangle s(e_2)$.

The *substitution* in FOPL is defined in the usual way. Given a formula ϕ, $\phi[e/u]$ denotes the formula obtained by replacing each free occurrence of the variable u (not bound by any quantifier) with the term e in ϕ. We always assume that a substitution is *admissible*, in the sense that the substitution does not affect the freeness of the other existing variables in ϕ except u.

The special theory $\mathfrak{A}$ fixes the Kripke frame in FOPL, namely $(\mathcal{S}_{fo}, \mathcal{I}_{fo})$, where $\mathcal{S}_{fo}$ is the set of all worlds in $\mathfrak{A}$, and $\mathcal{I}_{fo}$ interprets an assignment and a predicate in $\mathfrak{A}$ as follows:

1. $\mathcal{I}_{fo}(u := e) =_{df} \{ss' \mid s' = s[u \mapsto s(e)]\}$.
2. $\mathcal{I}_{fo}(e_1 \triangle e_2) =_{df} \{s \mid s(e_1 \triangle e_2)$ is true in theory $\mathfrak{A}\}$.

The semantics of FOPL is obtained by extending $(\mathcal{S}_{fo}, \mathcal{I}_{fo})$ to other programs and formulas as in Definition 2.2, except that, for a quantified formula of the form $\forall u.\phi$, we have

$$\mathcal{I}_{fo}(\forall u.\phi) =_{df} \{tr \mid \forall i \in \mathbb{Z}.(tr)[u \mapsto i] \in \mathcal{I}_{fo}(\phi)\},$$

where $(tr)[u \mapsto i]$ is the trace obtained by replacing each world s in tr with $s[u \mapsto i]$.

3 Overview

Having recalled the basic definitions of process logic, we now step back and ask: *why do we need a new proof framework, and how does our approach work?* This section gives an informal tour of the main ideas before the technical development begins.

Section 3.1 motivates our work by showing that classical and trace-based program logics can be embedded into process logic, making a complete proof system for it broadly useful. Section 3.2 introduces the core idea of the cyclic labelled approach and outlines the soundness and completeness arguments for the proposed labelled systems. Section 3.3 compares our approach with other recent program logics for reasoning about temporal properties.

3.1 Motivations of Process Logic

Process logic is highly expressive: its trace-based formulas subsume classical Hoare triples, dynamic-logic modalities, and the richer LTL-flavoured properties studied in recent traced program logics. Providing a *complete* proof system for it is therefore both theoretically significant and broadly applicable, yet non-trivial — especially at the first-order level. In the subsections below we make this motivation concrete by showing how the key logics from the literature each embed into process logic.

3.1.1 From State Properties to Trace Properties: Hoare Logic and Dynamic Logic. Because process logic evaluates formulas over execution *traces*, it is strictly more expressive than purely state-based logics. In particular, it subsumes the classical Hoare-logic specification $\{\phi\}\alpha\{\psi\}$, which asserts that if a program state satisfies the precondition ϕ then, after executing program α, the resulting state satisfies the postcondition ψ.

In process logic, this correctness condition is captured by the formula

$$\phi \rightarrow [\alpha]\Box(Ł_0 \rightarrow \psi),$$

where the auxiliary abbreviations are defined as follows [16]:

$$\Box\varphi =_{df} \neg(\Diamond\neg\varphi),$$
$$\Diamond\varphi =_{df} \mathit{true}\,\mathrm{U}\,\varphi,$$
$$Ł_0 =_{df} \neg\,\mathbf{n}\,\mathit{true}.$$

Here, $\Box\varphi$ ("always φ") means that φ holds at *every* state along the current trace; $\Diamond\varphi$ ("eventually φ") means that φ holds at *some* state along the trace; and $Ł_0$ ("last state") is a formula that holds precisely at the *final* state of a trace, since $\neg\,\mathbf{n}\,\mathit{true}$ is satisfied exactly when there is no next step. Thus $\Box(Ł_0 \rightarrow \psi)$ asserts that at the last state of every execution trace of α the formula ψ holds — which is exactly the Hoare postcondition.

Process logic also subsumes classical dynamic logic. In dynamic logic, every formula is a *state formula* — it is satisfied by a single world, not by a trace. The box modality $[\alpha]\phi$ therefore asserts that the world obtained after executing α (in any possible way) satisfies ϕ. Because process logic evaluates formulas over traces rather than over single states, the

equivalent process-logic statement must single out the *last* state of the execution trace. This is achieved by the formula

$$[\alpha]\Box(Ł_0 \to \phi),$$

which, by the same reasoning as above, says that at the final state of every execution trace of α the formula ϕ holds.

3.1.2 Existing Trace-Based Dynamic Logics and Their Limitations. Process logic also subsumes the various trace-semantic-based dynamic logics discussed in Section 1. For each, we briefly explain its core idea and show how its key formulas and rules can be expressed in process logic. This discussion motivates our own labelled framework, introduced in Section 3.2, by highlighting the limitations of each approach.

Dynamic Logic with Trace Modalities (DLT). DLT [4] extends FODL with a trace modality $[|\cdot|]$ that captures the intermediate worlds visited during program execution. In DLT, the underlying program model consists of while programs, and the proof system is proved to be complete for this class [4]. The formula $[|\alpha|]\phi$ means that ϕ (a state formula) holds at *every* intermediate world along every execution trace of α. In process logic, this is captured precisely by

$$[\alpha]\Box\phi,$$

using the "always" operator $\Box$ introduced earlier.

For sequential programs, [4] proposes the following compositional rule, which we call $([;]-\Box)$:

$$\frac{[|\alpha|]\phi \wedge [\alpha][|\beta|]\phi}{[|\alpha;\beta|]\phi}$$

Expressed as a process-logic axiom, this rule becomes:

$$[\alpha;\beta]\Box\phi \leftrightarrow [\alpha]\Box\phi \wedge [\alpha](\mathbf{last}[\beta]\Box\phi),$$

where $\mathbf{last}\,\varphi =_{df} \Diamond(\varphi \wedge Ł_0)$ for any formula φ. Intuitively, $\mathbf{last}\,\varphi$ means that φ holds *at the last state of the current trace*: $\Diamond(\varphi \wedge Ł_0)$ requires that there is some state on the trace that is simultaneously the final state ($Ł_0$) and satisfies φ. The axiom then says: every trace of $\alpha;\beta$ satisfies $\Box\phi$ if and only if every trace of α already satisfies $\Box\phi$, *and* at the end of every trace of α, every continuation trace of β also satisfies $\Box\phi$. In other words, the axiom splits the sequential execution into two parts and ensures that ϕ is maintained throughout both segments. In process logic, this rule is more complex than the simpler rule (4) of Table 2 for handling sequential programs.

The subsequent work [24] proposes the logic DLT_d, which adopts the trace modalities of DLT as an extension to differential dynamic logic [23], applying them to the more general regular programs. However, it remains open whether the system fragment of DLT in DLT_d is complete. Moreover, [35] shows that for regular programs the direction $[\alpha;\beta]\Box\phi \to [\alpha]\Box\phi \wedge [\alpha](\mathbf{last}[\beta]\Box\phi)$ does not hold: a test in $\alpha;\beta$ may block a trace of α from continuing as a trace of β, thereby invalidating the compositional split. Consequently, rule $([;]-\Box)$ is too strong and is thus insufficient to derive all valid trace properties for arbitrary regular programs.

Differential Temporal Dynamic Logic DLT_d^2. It was later observed in [24] that rule $([;]-\Box)$ cannot be applied to reasoning about the formula $[\alpha]\Diamond\phi$ for the "eventually" operator $\Diamond$: the rule breaks down because distributing the $\Diamond$ modality over a sequential composition does not follow the same pattern as $\Box$. To handle "eventually" formulas, [19] introduces the logic DLT_d^2, which enriches the formula $[\alpha]\Diamond\phi$ into the richer form $[\alpha](\psi \sqcup \Diamond\phi)$ and proposes a compositional rule named $([;]-\Diamond)$:

$$\frac{[\alpha]([\beta](\psi \sqcup \Diamond\phi) \sqcup \Diamond\phi)}{[\alpha;\beta](\psi \sqcup \Diamond\phi)}\ .$$

In process logic, the formula $[\alpha](\psi \sqcup \Diamond\phi)$ is captured as

$$[\alpha](\psi \vee \Diamond\phi),$$

which means that each trace of α satisfies either ψ or $\Diamond\phi$. And the rule $([;] - \Diamond)$ can be captured as the axiom:

$$[\alpha\,;\beta](\psi \vee \Diamond\phi) \leftrightarrow [\alpha](\mathbf{last}([\beta](\psi \vee \Diamond\phi)) \vee \Diamond\phi).$$

Informally, the axiom decomposes the "eventually" property of the sequential program $\alpha\,;\beta$: the combined execution satisfies $\psi \vee \Diamond\phi$ if and only if, at the end of every α-trace, either (i) $\Diamond\phi$ has already been witnessed within that α-trace, or (ii) the subsequent β-execution will witness $\psi \vee \Diamond\phi$ from that terminal state.

Later work [1] proves that DLT^2_d is complete for deterministic program behaviours, i.e., given a fixed starting world, there is only one possible execution trace. In regular programs, this is possible if the choice operator $\cup$ is used with mutually exclusive Boolean test conditions. For example, consider the program $\phi?\,;a \cup \neg\phi?\,;b$ for any ϕ, a, and b. However, the completeness for general regular programs with genuine nondeterminism remains open; moreover, it is not clear whether any fixed set of compositional rules suffices to handle all LTL operators uniformly.

Dynamic Trace Logic (DTL). Earlier work [2] proposes another variant of trace-based dynamic logic called dynamic trace logic (DTL). The program models of DTL are restricted to while programs, which are deterministic. Under this restriction, the proof system of DTL can derive every valid formula $[\alpha]\phi$ for an *arbitrary* LTL formula ϕ. The key insight is that compositional rules for a single deterministic trace work uniformly for all types of temporal properties. However, this argument breaks down for regular programs in general, where nondeterminism allows multiple possible execution traces and the compositional rules must handle branching.

3.2 Labelled System & Our Approach

3.2.1 A Labelled Approach for Process Logic. We adopt an alternative approach to provide a complete logical framework for process logic at both propositional and first-order levels. Our approach draws on the rich tradition of *labelled proof systems* for modal and dynamic logics (cf., e.g., [12, 22]), which augment formulas with explicit structural information—called *labels*—that records the relational or semantic context in which a formula is evaluated. By making this context part of the formula itself, labelled systems often admit cleaner, more modular inference rules than their unlabelled counterparts.

Following the main idea from prior work on labelled dynamic-logic systems [12, 34, 38], we introduce a label σ to capture the current execution-trace information accumulated during a derivation. A labelled process-logic formula takes the form $\sigma : [\alpha]\phi$, meaning that under the trace information recorded in σ, every execution trace of α satisfies the formula ϕ. We shall see that this labelled form is strictly more general than the plain formula $[\alpha]\phi$: when σ is a free trace variable with no additional constraints, $\sigma : [\alpha]\phi$ and $[\alpha]\phi$ share the same validity.

To see why labels help, consider the formula

$$\{x := 1\} : [x := 2\,;x := 3]\Box(x \geq 0),$$

which means that given the execution trace information $\{x := 1\}$ (recording that x was set to 1 at the first step), every continuation via program $x := 2\,;x := 3$ produces a whole trace satisfying $\Box(x \geq 0)$. By applying a labelled version of the sequential-composition rule, we can unfold this derivation step by step, accumulating execution information in the

label:

$$\dfrac{\dfrac{\dfrac{\{x := 1\} \diamond \{x := 2\} \diamond \{x := 3\} : \Box(x \geq 0)}{\{x := 1\} \diamond \{x := 2\} : [x := 3]\Box(x \geq 0)}}{\{x := 1\} : [x := 2][x := 3]\Box(x \geq 0)}}{\{x := 1\} : [x := 2\,; x := 3]\Box(x \geq 0)} \quad ,$$

where the label $\{x := 1\} \diamond \{x := 2\} \diamond \{x := 3\}$ records the full execution history. Once all programs have been unfolded, the verification goal reduces to checking whether this concrete label satisfies the temporal formula $\Box(x \geq 0)$ — a much simpler task. This contrasts with the traditional approach using the rule $([;] - \Box)$, which would split $[x := 2\,; x := 3]\Box(x \geq 0)$ into separate sub-goals $[x := 2]\Box(x \geq 0)$ and $[x := 3](\textbf{last}([x := 3]\Box(x \geq 0)))$ that must be handled independently.

The formal construction of the labelled systems for PPL and FOPL — namely G3PPL and G3FOPL — is carried out in Sections 4.1, 4.2, 7.1, and 7.2, respectively.

Similar to [12, 34], our labelled system for process logic does not directly support reasoning about iterative regular programs: when unwrapping the iterative structure, the same program form repeats during a derivation, producing an infinite derivation. The following derivation illustrates this:

$$\dfrac{\dfrac{\dfrac{

is discussed in detail in Section 5. For both G3PPL and G3FOPL, the soundness arguments follow the main strategy from [34], but adapted with non-trivial technical details specific to the trace semantics of process logic.

Completeness. We establish the completeness using different proof methods for G3PPL and G3FOPL, discussed in Sections 6 and 7.3 respectively. For G3PPL, the key idea is to show that the labelled system can simulate every axiom and rule of the established Hilbert-style proof system for PPL displayed in Table 2. We present the technical details for the critical "Necessitation" rule, the PDL axioms (rules (1)–(7), (MP), (Gen)), and one side of the process-logic-specific axiom (iii), leaving the proofs for the other process-logic-specific axioms (rules (i), (ii), (iv) – (xv), and the other side of rule (iii)) to the appendix. For G3FOPL, we adapt the classical completeness argument for FODL [15, 17] to the labelled setting with trace semantics, providing the necessary technical modifications.

3.3 Comparisons of Labelled Process Logic to Other Program Logics

In addition to the trace-based dynamic logics discussed in Section 3.1, several recent program logics address temporal reasoning about programs. These logical theories are accompanied by mechanized verification tools. Below, we compare them further in terms of theoretical aspects such as expressiveness, program structures, and proof techniques. This comparison shows that the labelled process logic that we propose is competitive with these works and is worth further development and mechanization, but it also has limitations.

[32] proposed a Hoare-style logic for reasoning about temporal properties for pointer while programs. It can be seen as a separation logic extended by CTL [10] formulas. These CTL formulas can be captured equivalently in labelled process logic by combining its modality $[\cdot]$ and LTL formulas. E.g., a specification "$\varphi \vdash \alpha : \mathrm{A}(\phi \,\mathrm{U}\, \psi)$" in [32] just means $X : \varphi \Rightarrow X : [\alpha]\phi \,\mathrm{U}\, \psi$ with $X \in \mathit{TVar}$ in labelled process logic, where the modality $[\alpha]$ captures the meaning of the for-all-paths symbol A in CTL formulas. The same applies to other CTL operators such as A X and E U. A cyclic proof system was built that adopted compositional rules similar to those in [2] but specialized to CTL formulas in order to decompose while programs. Compared to [32], labelled process logic cannot handle pointers; it is a more abstract logical framework. As discussed in Section 1, we take a different approach that avoids directly decomposing sequential programs by introducing trace labels. We also handle dynamic formulas more complex than Hoare triples and program models more general than while programs.

Temporal Interaction and Choice Logic (Ticl) [18] takes a monadic tree structure as its program model. Temporal properties in Ticl are expressed by the satisfaction of CTL formulas by monad trees. As above, labelled process logic can capture the meanings of all CTL formulas in Ticl using its $[\cdot]$ modality and LTL formulas. The regular programs adopted in labelled process logic are closer to the structures of practical programs and powerful enough to express the finite-trace behaviours of standard practical programs. By contrast, Ticl's monadic trees can express infinite-trace behaviours and program features not expressible by regular programs, including "return", "visibility", "silent events", and "bind". Yet the completeness of Ticl's proof system is not discussed in [18]. When dealing with loop structures, our work takes a cyclic-proof approach, whereas Ticl uses a well-founded proof system and the coinductive semantics of CTL formulas and monadic trees.

The Hoare-style program logics LiLi [20] and TaDA Live [13] both concentrate on temporal properties of concurrent while programs. LiLi offers a rely-guarantee-style proof system, in which a particular set of CTL properties that concern "progress properties of blocking objects" are handled in a modular way. TaDA Live is a concurrent separation logic for compositionally reasoning about the termination of blocking concurrent while programs. To this end, certain safety and liveness properties related to termination are discussed and proved. Compared to the two works, labelled process

logic cannot specify concurrent models with special features. However, it is a more general logical framework that can handle a wide range of temporal properties, not just those specific to certain program domains.

Other recent works, such as [33] and [21], also address the use of program logics to verify temporal properties of program traces. However, their approaches are based mainly on program refinement rather than direct reasoning in a program logic, as in our work.

Limitation of Labelled Process Logic. Compared to these works, labelled process logic has the main limitation that it can handle only finite traces because it adopts regular programs as its program model. The same limitation applies to the original process logic proposed in [16]. As stated in [17], in order to cover infinite traces, we need to introduce the omega loop α^ω, and build suitable rules for that operator. More work can be done in the future to address this extension.

4 Labelled Propositional Process Logic

This section develops the labelled propositional process logic and its proof system. We first define *labels* and *labelled formulas*, then present the labelled sequent rules and the cyclic proof condition required for regular-program iteration.

4.1 Labelled PPL Formulas

Definition 4.1 (Labels of PPL). *A label σ is a trace defined as follows:*

$$\sigma =_{df} \varepsilon \mid x \mid X \mid \sigma \diamond \sigma,$$

where x is a label variable for worlds; X is a label variable for traces of worlds; $\diamond$ is a concatenation operator between label variables; ε is called the "empty label", which satisfies $\sigma \diamond \varepsilon = \varepsilon \diamond \sigma = \sigma$ for any label σ.

Use $\mathbf{L}$ to denote the set of all labels. Use $WVar$ to express the set of all label variables for worlds; use $TVar$ to express the set of all label variables for traces. Define $LVar =_{df} WVar \cup TVar$ to be the set of all label variables.

The *substitution* on labels is defined in a natural way: for a label σ, $\sigma[\sigma'/U]$ returns the label obtained by replacing each free occurrence of a variable $U \in LVar$ with the label σ'. Since no quantifiers for label variables exist, all occurrences of label variables are free.

Definition 4.2 (Label Mappings of PPL). *Given a Kripke frame $(\mathcal{S}, \mathcal{I})$, a world mapping $\mathfrak{w}$ in PPL is a function from $WVar$ to $\mathcal{S}$.*

A label mapping $\mathfrak{m}$ is a function from $\mathbf{L}$ to the set $\mathcal{S}^*$ of world traces, defined as follows:

i $\mathfrak{m}$ *maps a variable $X \in TVar$ to a trace of worlds in $\mathcal{S}^*$.*
ii $\mathfrak{m}(\varepsilon) =_{df} \epsilon$.
iii $\mathfrak{m}(x) =_{df} \mathfrak{w}(x)$.
iv $\mathfrak{m}(\sigma_1 \diamond \sigma_2) =_{df} \mathfrak{m}(\sigma_1) \cdot \mathfrak{m}(\sigma_2)$.

Denote the set of all label mappings (w.r.t. $(\mathcal{S}, \mathcal{I})$) by $\mathbf{M}(\mathcal{S}, \mathcal{I})$ (simply $\mathbf{M}$).

Definition 4.3 (Label Relations). *Given an action a and label variables $x, y \in LVar$, xR_ay is called a "label relation".*

Definition 4.4 (Labelled PPL Formulas). *A "labelled PPL formula" is of the form*

$$\sigma : \phi,$$

where $\sigma \in \mathbf{L}$ and ϕ is a PPL formula.

We sometimes use the symbol τ to represent a labelled formula.

Definition 4.5 (Semantics of Labelled PPL Formulas & Label Relations). *Given a Kripke frame $(\mathcal{S}, \mathcal{I})$ and a set $\mathbf{M}$ of label mappings w.r.t. $(\mathcal{S}, \mathcal{I})$, the satisfaction relation $\mathfrak{m} \models_{\mathbf{M}} \sigma : \phi$ of a labelled formula $\sigma : \phi$ by a label mapping $\mathfrak{m}$ (w.r.t. $\mathbf{M}$) is defined by*

$$\mathfrak{m} \models_{\mathbf{M}} \sigma : \phi, \ \textit{if}\ \mathfrak{m}(\sigma) \models_{(\mathcal{S},\mathcal{I})} \phi.$$

The satisfaction relation $\mathfrak{m} \models_{\mathbf{M}} xR_ay$ of a label relation xR_ay by a label mapping $\mathfrak{m}$ (w.r.t. $\mathbf{M}$) is defined by

$$\mathfrak{m} \models_{\mathbf{M}} xR_ay, \ \textit{if}\ \mathfrak{m}(x)\mathfrak{m}(y) \in \mathcal{I}(a).$$

We write $\models_{\mathbf{M}}$ simply as $\models$ when $\mathbf{M}$ and the frame $(\mathcal{S}, \mathcal{I})$ are clear from context.

Relationship to PPL Formulas. A valid PPL formula ϕ can be captured by its equivalent labelled form $X : \phi$, with $X \in \mathit{TVar}$. Indeed, ϕ is valid if and only if every trace satisfies it: ϕ is valid iff for every trace tr, $tr \models \phi$, iff for every label mapping $\mathfrak{m}$, $\mathfrak{m}(X) \models \phi$, iff for every label mapping $\mathfrak{m}$, $\mathfrak{m} \models X : \phi$. Thus the validity of $X : \phi$ in labelled process logic implies the validity of ϕ in process logic, and vice versa.

4.2 A Proof System for Labelled PPL

A *sequent* is a logical argumentation of the form:

$$\Gamma \Rightarrow \Delta,$$

where Γ and Δ are finite sets of formulas, called the *left side* and the *right side* of the sequent, respectively. We use a dot $\cdot$ to represent Γ or Δ when it is the empty set. Intuitively, a sequent $\Gamma \Rightarrow \Delta$ means that if all formulas in Γ hold, then one of the formulas in Δ holds. We use ν to represent a sequent. An *inference rule* is of the form

$$\frac{\nu_1 \quad \dots \quad \nu_n}{\nu},$$

with the premises $\nu_1, \dots, \nu_n$ and the conclusion ν. The semantics of the rule is that the validity of sequents $\nu_1, \dots, \nu_n$ implies the validity of sequent ν. We call a formula pair (τ_1, τ_2) a *conclusion-premise* (CP) pair if τ_1 and τ_2 belong to sequents ν, ν_i for some $1 \leq i \leq n$ respectively. In this paper, we use a double-lined inference form:

$$\frac{\phi_1 \quad \dots \quad \phi_n}{\phi}$$

as a shorthand to represent both rules

$$\frac{\Gamma \Rightarrow \phi_1, \Delta \quad \dots \quad \Gamma \Rightarrow \phi_n, \Delta}{\Gamma \Rightarrow \phi, \Delta} \quad \text{and} \quad \frac{\Gamma, \phi_1 \Rightarrow \Delta \quad \dots \quad \Gamma, \phi_n \Rightarrow \Delta}{\Gamma, \phi \Rightarrow \Delta},$$

for arbitrary contexts Γ and Δ.

Given a proof system, i.e., a set of inference rules, a *proof* of a sequent ν is a finite tree constructed by applying these rules backwardly (from conclusions to premises). In this tree, each node is a sequent, and the root is ν. If a node is N and its children are $N_1, \dots, N_m$ ($m \geq 0$), then

$$\frac{N_1 \quad \dots \quad N_m}{N}$$

is an instance of an inference rule. Each leaf node is *terminal*, in the sense that it is the conclusion of an axiom. We say that a sequent is provable if it has a proof.

The proof system of labelled PPL is displayed in Tables 3, 4, and 5 and consists of three parts: the rules for regular programs (Table 3), which are extended from the G3 sequent calculus for modal logics [22]; the rules for trace formulas

Rule	Schematic form
$([a]R)$	$\dfrac{\Gamma, x\,\mathrm{R}_a\, y \Rightarrow \sigma \diamond x \diamond y : \phi, \Delta}{\Gamma \Rightarrow \sigma \diamond x : [a]\phi, \Delta}$, where $y \in \mathit{WVar}$ is fresh
$([a]L)$	$\dfrac{\Gamma, \sigma \diamond x \diamond y : \phi \Rightarrow \Delta}{\Gamma, \sigma \diamond x : [a]\phi, x\,\mathrm{R}_a\, y \Rightarrow \Delta}$, for some $y \in \mathit{WVar}$
$([\phi?]R)$	$\dfrac{\Gamma, x \diamond x : \psi \Rightarrow \sigma \diamond x \diamond x : \phi, \Delta}{\Gamma \Rightarrow \sigma \diamond x : [\psi?]\phi, \Delta}$
$([\phi?]L)$	$\dfrac{\Gamma, \sigma \diamond x \diamond x : \phi \Rightarrow \Delta}{\Gamma, \sigma \diamond x : [\psi?]\phi, x \diamond x : \psi \Rightarrow \Delta}$
$([;])$	$\dfrac{\sigma : [\alpha][\beta]\phi}{\sigma : [\alpha ; \beta]\phi}$
$([\cup])$	$\dfrac{\sigma : [\alpha]\phi \quad \sigma : [\beta]\phi}{\sigma : [\alpha \cup \beta]\phi}$
$([*])$	$\dfrac{\sigma : [\mathit{true}?]\phi \quad \sigma : [\alpha][\alpha^*]\phi}{\sigma : [\alpha^*]\phi}$
(sub)	$\dfrac{\Gamma \Rightarrow \Delta}{\Gamma[\sigma/U] \Rightarrow \Delta[\sigma/U]}$, where $U \in \mathit{LVar}$

Table 3. Rules of G3PPL for regular programs and label substitution.

Rule	Schematic form
(p)	$\dfrac{x : p}{x \diamond \sigma : p}$
$(\mathbf{f})$	$\dfrac{x : \phi}{x \diamond \sigma : \mathbf{f}\,\phi}$
$(\mathbf{suf}\ R)$	$\dfrac{\Gamma \Rightarrow \sigma : \psi, \sigma : \phi, \Delta \quad \Gamma \Rightarrow \sigma : \psi, \sigma : \phi\ \mathbf{suf}\ \psi, \Delta}{\Gamma \Rightarrow x \diamond \sigma : \phi\ \mathbf{suf}\ \psi, \Delta}$
$(\mathbf{suf}\ L)$	$\dfrac{\Gamma, \sigma : \psi \Rightarrow \Delta \quad \Gamma, \sigma : \phi, \sigma : \phi\ \mathbf{suf}\ \psi \Rightarrow \Delta}{\Gamma, x \diamond \sigma : \phi\ \mathbf{suf}\ \psi \Rightarrow \Delta}$
$(\mathbf{suf}\ \ x)$	$\dfrac{}{\Gamma, x : \phi\ \mathbf{suf}\ \psi \Rightarrow \Delta}$

Table 4. Rules of G3PPL for trace formulas.

(Table 4), which are specific to process logic; and the rules for the labelled versions of propositional formulas (Table 5), which are obtained by augmenting the classical sequent calculus for propositional logic. We call the resulting proof system G3PPL.

In a right rule of the form

$$\frac{\Gamma_1 \Rightarrow \phi_1, \Delta \quad \ldots \quad \Gamma_n \Rightarrow \phi_n, \Delta}{\Gamma \Rightarrow \phi, \Delta}\ ,$$

we call ϕ the "target formula" of the rule and call each ϕ_i a "replacement" of ϕ. For example, in Table 3, $\sigma \diamond x : [a]\phi$ is the target of the rule $([a]R)$, and $\sigma \diamond x \diamond y : \phi$ is the replacement of $\sigma \diamond x : [a]\phi$.

The rules for regular programs in Table 3 follow the standard labelled-sequent treatment of modal operators [22], adapted to the trace semantics of PPL. Rules $([a]R)$ and $([a]L)$ handle the box modality over atomic programs: to prove

Rule	Schematic form
(ax)	$\dfrac{}{\Gamma, \sigma : \phi \Rightarrow \sigma : \phi, \Delta}$
(cut)	$\dfrac{\Gamma \Rightarrow \sigma : \phi, \Delta \quad \Sigma, \sigma : \phi \Rightarrow \Lambda}{\Gamma, \Sigma \Rightarrow \Delta, \Lambda}\ (Cut)$
(wk)	$\dfrac{\dfrac{}{\sigma : \phi}}{}$
$(\neg R)$	$\dfrac{\Gamma, \sigma : \phi \Rightarrow \Delta}{\Gamma \Rightarrow \sigma : \neg\phi, \Delta}$
$(\neg L)$	$\dfrac{\Gamma \Rightarrow \sigma : \phi, \Delta}{\Gamma, \sigma : \neg\phi \Rightarrow \Delta}$
$(\wedge R)$	$\dfrac{\Gamma \Rightarrow \sigma : \phi, \Delta \quad \Gamma \Rightarrow \sigma : \psi, \Delta}{\Gamma \Rightarrow \sigma : \phi \wedge \psi, \Delta}$
$(\wedge L)$	$\dfrac{\Gamma, \sigma : \phi, \sigma : \psi \Rightarrow \Delta}{\Gamma, \sigma : \phi \wedge \psi \Rightarrow \Delta}$

Table 5. Primitive propositional and structural rules of G3PPL.

$\sigma \diamond x : [a]\phi$, rule $([a]R)$ introduces a fresh world variable y representing the successor of x under action a, reducing the goal to $\sigma \diamond x \diamond y : \phi$ with the relational atom $xR_a y$ in the antecedent; rule $([a]L)$ uses an existing relational atom to consume the modality from the left. Rules $([\phi?]R)$ and $([\phi?]L)$ handle tests: a test $\psi?$ produces a stuttering trace, so the rule appends x to itself and requires the test condition to be satisfied. Rules $([;])$, $([\cup])$, and $([*])$ unfold sequential composition, choice, and iteration, respectively, matching the standard PDL axioms (Table 2). Rule (sub) substitutes a label variable U with a concrete label σ throughout a sequent; it is the key mechanism for constructing back-links in cyclic proofs.

The rules for trace formulas in Table 4 capture the semantics of the temporal operators in process logic. Rule (p) states that an atomic proposition p evaluated on a trace $x \diamond \sigma$ depends only on the first world x: if $x : p$ holds, then so does $x \diamond \sigma : p$ for any σ. Rule $(\mathbf{f})$ similarly reduces the evaluation of $\mathbf{f}\,\phi$ on $x \diamond \sigma$ to the evaluation of ϕ at x alone.

Rules $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ are the key rules for the suffix connective, and both are read off the one-step unfolding

$$x \diamond \sigma : \phi\ \mathbf{suf}\ \psi \text{ iff } \sigma : \psi \vee (\phi \wedge \phi\ \mathbf{suf}\ \psi),$$

which states that a trace $x \diamond \sigma$ satisfies $\phi\ \mathbf{suf}\ \psi$ exactly when its suffix σ either already satisfies ψ, or satisfies ϕ while the same condition propagates along σ. Intuitively, rule $(\mathbf{suf}\ R)$ is what applying $(\vee R)$ and then $(\wedge R)$ to the right-hand side of this equivalence yields: to prove $x \diamond \sigma : \phi\ \mathbf{suf}\ \psi$ on the right of a sequent, it suffices to prove the two premises in which the target formula is replaced by $\sigma : \psi, \sigma : \phi$ and by $\sigma : \psi, \sigma : \phi\ \mathbf{suf}\ \psi$ respectively. Dually, rule $(\mathbf{suf}\ L)$ is what applying $(\vee L)$ and then $(\wedge L)$ yields: an assumption $x \diamond \sigma : \phi\ \mathbf{suf}\ \psi$ on the left splits into the case $\sigma : \psi$, in which the suffix already witnesses ψ, and the case $\sigma : \phi, \sigma : \phi\ \mathbf{suf}\ \psi$, in which the witness is postponed to a shorter suffix. Note that in both rules the label of the formula $\phi\ \mathbf{suf}\ \psi$ in the premise is strictly shorter than the label of the same formula in the conclusion; this is exactly what makes an application of $(\mathbf{suf}\ R)$ or $(\mathbf{fus}\ R)$ (Table 6) a progressing step in the cyclic condition given in Section 4.3.

Finally, rule $(\mathbf{suf}\ \ x)$ is an axiom that closes a branch when the trace reduces to a single world variable x: since x has no proper suffix, $x : \phi\ \mathbf{suf}\ \psi$ is vacuously false and the branch terminates.

$$\dfrac{\cdot \Rightarrow \tau_7 :: x : [a^*]p \ (bud) \qquad \dfrac{\dfrac{}{x : [true?]p, x : [a][a^*]p \Rightarrow x : [true?]p}\,(ax)}{x : [a^*]p \Rightarrow x : [true?]p}\,([*])}{\cdot \Rightarrow \tau_2 :: x : [true?]p}\,(cut) \qquad \dfrac{\dfrac{\dfrac{\cdot \Rightarrow \tau_6 :: x : [a^*]p \ (bud)}{\cdot \Rightarrow \tau_5 :: x \diamond y : [a^*]p}\,(sub)}{xR_a y \Rightarrow \tau_4 :: x \diamond y : [a^*]p}\,(wk)}{\cdot \Rightarrow \tau_3 :: x : [a][a^*]p}\,([a]R) \qquad ([*])$$
$$\cdot \Rightarrow \tau_1 :: x : [a^*]p$$

Fig. 1. An example of invalid cyclic derivation

The rules for labelled propositional formulas (Table 5) have the same meaning as their unlabelled counterparts in classical propositional logic.

Table 6 collects the derived rules for the non-primitive operators $\langle\cdot\rangle$, $\vee$, and $\rightarrow$; each of these rules follows directly from the primitive rules of Table 5 and is listed here for convenient reference in subsequent proofs. We also give the derived rules for the operator **fus**, which transform the negation of a formula ϕ **suf** ψ and can be derived from the rules for the operator **suf** in Table 5.

Soundness of the rules in G3PPL is stated in the following theorem.

Theorem 4.1. *Each rule in Tables 3, 4, and 5 for G3PPL is sound.*

The proof directly follows from the semantics of labelled PPL formulas. We omit it in this paper.

Note: In the derivations that follow in this paper, we occasionally omit one or more intermediate steps to save space. For example, the annotation (**suf** R, **suf** L) in Figure 2 indicates that instances of the rules (**suf** R) and (**suf** L) are each applied one or more times. Sometimes, we list only the main rule applied.

4.3 Cyclic Proof Structure of G3PPL

Because regular programs include iteration, the derivation of a formula in G3PPL does not always terminate, as the same formula forms may appear repeatedly.

Example 4.1. *In Figure 1, using the rules in G3PPL yields the repeated root node* $\cdot \Rightarrow x : [a^*]p$ *in two branches. Continuing to use the same rules leads to an infinite derivation that does not terminate.*

In the cyclic-proof approach, a *preproof* is a finite proof tree in which there is a special type of nonterminal leaf node, called a *bud* (cf. [9]). A bud is identical to one of its ancestors in its syntactic form in the tree. Together, a bud and one of its identical ancestors are called a *back-link*. A *derivation path* in a proof tree is expressed as a sequence of nodes $\nu_1\nu_2 \ldots \nu_m \ldots$ ($m \geq 1$) starting from the root node ν_1, where each node pair (ν_i, ν_{i+1}) ($i \geq 1$) belongs to an instance of a rule. In a preproof, there always exists an infinite derivation path over one or more derivation branches which contain back-links.

A preproof may not be a valid proof. As in Example 4.1, the whole derivation is a preproof, with the two leaf nodes back-linked to the root. However, for some choices of a, p, and the Kripke frame, the formula $x : [a^*]p$ concluded in the preproof may be false. For example, $x : [(true?)^*]false$ is false for every x. As also indicated in works such as [8, 9], this indicates that we need to specify a particular *cyclic condition* for G3PPL to ensure that a preproof has a sound conclusion.

Below we define the notion of derivation traces in G3PPL, then introduce the progressing derivation traces for G3PPL. Based on these concepts, we finally give the cyclic condition for G3PPL in Definition 4.8.

Rule	Schematic form
$(\langle a\rangle R)$	$\dfrac{\Gamma \Rightarrow \sigma \diamond x \diamond y : \phi, \Delta}{\Gamma, x\,\mathrm{R}_a\,y \Rightarrow \sigma \diamond x : \langle a\rangle\phi, \Delta}$, for some $y \in \mathit{WVar}$
$(\langle a\rangle L)$	$\dfrac{\Gamma, \sigma \diamond x \diamond y : \phi, x\,\mathrm{R}_a\,y \Rightarrow \Delta}{\Gamma, \sigma \diamond x : \langle a\rangle\phi \Rightarrow \Delta}$, where $y \in \mathit{WVar}$ is fresh
$(\langle \phi? \rangle R)$	$\dfrac{\Gamma \Rightarrow \sigma \diamond x \diamond x : \phi, \Delta}{\Gamma, x \diamond x : \psi \Rightarrow \sigma \diamond x : \langle \psi? \rangle \phi, \Delta}$
$(\langle \phi? \rangle L)$	$\dfrac{\Gamma, \sigma \diamond x \diamond x : \phi, x \diamond x : \psi \Rightarrow \Delta}{\Gamma, \sigma \diamond x : \langle \psi? \rangle \phi \Rightarrow \Delta}$
$(\langle ; \rangle)$	$\dfrac{\sigma : \langle \alpha \rangle \langle \beta \rangle \phi}{\sigma : \langle \alpha ; \beta \rangle \phi}$
$(\langle \cup \rangle R)$	$\dfrac{\Gamma \Rightarrow \sigma : \langle \alpha \rangle \phi, \sigma : \langle \beta \rangle \phi, \Delta}{\Gamma \Rightarrow \sigma : \langle \alpha \cup \beta \rangle \phi, \Delta}$
$(\langle \cup \rangle L)$	$\dfrac{\Gamma, \sigma : \langle \alpha \rangle \phi \Rightarrow \Delta \quad \Gamma, \sigma : \langle \beta \rangle \phi \rightarrow \Delta}{\Gamma, \sigma : \langle \alpha \cup \beta \rangle \phi \Rightarrow \Delta}$
$(\langle * \rangle R)$	$\dfrac{\Gamma \Rightarrow \sigma : \langle \mathit{true}? \rangle \phi, \sigma : \langle \alpha \rangle \langle \alpha^* \rangle \phi, \Delta}{\Gamma \Rightarrow \sigma : \langle \alpha^* \rangle \phi, \Delta}$
$(\langle * \rangle L)$	$\dfrac{\Gamma, \sigma : \langle \mathit{true}? \rangle \phi \Rightarrow \Delta \quad \Gamma, \sigma : \langle \alpha \rangle \langle \alpha^* \rangle \phi \Rightarrow \Delta}{\Gamma, \sigma : \langle \alpha^* \rangle \phi \Rightarrow \Delta}$
$(\vee R)$	$\dfrac{\Gamma \Rightarrow \sigma : \phi, \sigma : \psi, \Delta}{\Gamma \Rightarrow \sigma : \phi \vee \psi, \Delta}$
$(\vee L)$	$\dfrac{\Gamma, \sigma : \phi \Rightarrow \Delta \quad \Gamma, \sigma : \psi \Rightarrow \Delta}{\Gamma, \sigma : \phi \vee \psi \Rightarrow \Delta}$
$(\rightarrow R)$	$\dfrac{\Gamma, \sigma : \phi \Rightarrow \sigma : \psi, \Delta}{\Gamma \Rightarrow \sigma : \phi \rightarrow \psi, \Delta}$
$(\rightarrow L)$	$\dfrac{\Gamma \Rightarrow \sigma : \phi, \Delta \quad \Gamma, \sigma : \psi \Rightarrow \Delta}{\Gamma, \sigma : \phi \rightarrow \psi \Rightarrow \Delta}$
$(\mathbf{fus}\,R)$	$\dfrac{\Gamma \Rightarrow \sigma : \psi, \Delta \quad \Gamma \Rightarrow \sigma : \phi, \sigma : \phi\,\mathbf{fus}\,\psi, \Delta}{\Gamma \Rightarrow x \diamond \sigma : \phi\,\mathbf{fus}\,\psi, \Delta}$
$(\mathbf{fus}\,L)$	$\dfrac{\Gamma, \sigma : \psi, \sigma : \phi \Rightarrow \Delta \quad \Gamma, \sigma : \psi, \sigma : \phi\,\mathbf{fus}\,\psi \Rightarrow \Delta}{\Gamma, x \diamond \sigma : \phi\,\mathbf{fus}\,\psi \Rightarrow \Delta}$

Table 6. Derived rules used in G3PPL.

Definition 4.6 (Derivation Traces). *A "derivation trace" over a derivation path $\mu_1\mu_2\ldots\mu_k\nu_1\nu_2\ldots\nu_m\ldots$ ($k \geq 0$, $m \geq 1$) is a sequence $\tau_1\tau_2\ldots\tau_m\ldots$ of formulas, where each τ_i ($1 \leq i \leq m$) occurs on the right side of node ν_i and has one of the forms $\sigma : [\alpha]\phi$, $\sigma : \phi\,\mathbf{suf}\,\psi$, or $\sigma : \phi\,\mathbf{fus}\,\psi$, for some σ, α, ϕ, and ψ. Each CP pair (τ_i, τ_{i+1}) ($i \geq 1$) in the derivation (ν_i, ν_{i+1}) satisfies the following condition, according to the rule instance containing (ν_i, ν_{i+1}) in G3PPL:*

1. *If (ν_i, ν_{i+1}) belongs to an instance of the rule $([\alpha]R)$, $([\phi?R])$, $([;])$, $([\cup])$, $([*])$, then τ_i is the target formula and τ_{i+1} is one of its replacements;*
2. *If (ν_i, ν_{i+1}) belongs to an instance of the rule $(\mathbf{suf}\,R)$, then $\tau_i = x \diamond \sigma : \phi\,\mathbf{suf}\,\psi$ and $\tau_{i+1} = \sigma : \phi\,\mathbf{suf}\,\psi$;*
3. *If (ν_i, ν_{i+1}) belongs to an instance of the rule $(\mathbf{fus}\,R)$, then $\tau_i = x \diamond \sigma : \phi\,\mathbf{fus}\,\psi$ and $\tau_{i+1} = \sigma : \phi\,\mathbf{fus}\,\psi$;*

4. *If* (ν_i, ν_{i+1}) *belongs to an instance of rule* (sub)*, then* $\tau_i = \sigma[\sigma'/U] : [\alpha]\phi$ *and* $\tau_{i+1} = \sigma : [\alpha]\phi$ *for some* $\sigma, \sigma' \in \mathbf{L}$ *and formula* $[\alpha]\phi$*;*
5. *If* (ν_i, ν_{i+1}) *belongs to an instance of the other rules, then* $\tau_i = \tau_{i+1}$.

We call the CP pair (τ_i, τ_{i+1}) *a "trace pair" over the derivation* (ν_i, ν_{i+1})*, and call* τ_i *a "trace element".*

In Definition 4.6, the derived rule (**fus** R) needs to be considered because we allow a trace pair to appear only on the right side of a sequent.

Definition 4.7 (Progressing Derivation Traces of G3PPL). *In a preproof of system G3PPL, given a derivation trace* $\tau_1\tau_2\ldots\tau_m\ldots$ *over an infinite derivation path* $\ldots\nu_1\nu_2\ldots\nu_m\ldots$ $(m \geq 1)$ *starting from* τ_1 *in node* ν_1*, a trace pair* (τ_i, τ_{i+1}) $(1 \leq i \leq m)$ *of the derivation* (ν_i, ν_{i+1}) *is called a "progressing step" if* (ν_i, ν_{i+1}) *belongs to an instance of one of the following rules:* $([a]R)$*,* $([\phi?]R)$*,* (**suf** R) *and* (**fus** R)*.*

If a derivation trace has an infinite number of progressing steps, we say that the trace is "progressing". We say that a derivation path is "progressing" if it has a progressing derivation trace.

Example 4.2. *In Figure 1, we call the two derivation branches with buds the "left cycle" and the "right cycle", respectively. The derivation trace* $\tau_1\tau_3\tau_4\tau_5\tau_6(\tau_1)\ldots$ *over the infinite derivation path that covers only the right cycle is a progressing one because every pair* (τ_3, τ_4) *appearing in the trace is a progressing step.*

Definition 4.8 (Cyclic Proofs of G3PPL). *A preproof of G3PPL is called a "cyclic proof" if there is a progressing derivation trace over every infinite derivation path.*

Example 4.3. *In Example 4.2, any derivation path that only covers the left cycle or covers the left cycle for an infinite number of times does not have a derivation trace, since any pair* $(\tau_2, \tau_7(\tau_1))$ *over an instance of the rule* (cut) *is not a trace pair (according to Definition 4.6). For this reason, the whole derivation (as a preproof) is not a cyclic proof.*

Section 5 shows that a cyclic proof of G3PPL always leads to a valid conclusion.

In G3PPL, we also say that a sequent is provable if there is a cyclic proof of it. For a labelled PPL formula τ, we write G3PPL $\vdash \tau$ (or simply $\vdash \tau$) if the sequent $\cdot \Rightarrow \tau$ is provable in G3PPL.

5 Soundness of G3PPL

We analyze the soundness of G3PPL for any PPL formula. By the equivalence between PPL formulas and their labelled counterparts established in Section 4.1, it suffices to analyze soundness (and, in later sections, completeness) for the labelled form $X : \phi$ with $X \in TVar$, for any PPL formula ϕ.

Theorem 5.1 (Soundness of G3PPL). *For any labelled PPL formula* $X : \phi$ *with variable* $X \in TVar$*, if* $\vdash X : \phi$*, then* $\models X : \phi$.

Main Idea of the Proof of Theorem 5.1. We mainly follow the idea behind [8, 34] to prove Theorem 5.1. We proceed by contradiction. Assume that the conclusion of a cyclic proof is invalid. Then, by the soundness of the rules of G3PPL and a special selection procedure specified by Lemma 5.1, we can find an infinite derivation path all of whose nodes are invalid. By Definition 4.8 it contains a progressing trace, which would lead to an infinitely descending chain in some *well-founded set* (introduced below). But this violates the definition of the well-foundedness itself.

Our proof follows the proof strategy of [34] for a more general parameterized dynamic logic, but relaxes the termination condition used there to accommodate the explicit regular-program model. The key technical contribution

is the introduction of *counter-example sequences* for PPL formulas (Definition 5.3) and Lemma 5.1. Counter-example sequences offer a semantics-related, well-founded measure, while Lemma 5.1 describes a mechanism for finding this invalid derivation path and a related progressing trace.

Below, we first introduce the concept of well-foundedness and the well-founded relation on which we rely, and then present the proof skeleton for Theorem 5.1. Other proof details are given in Appendix A.1.

Definition 5.1 (Well-foundedness). *Given a set S and a partial-order relation $\preceq$ on S, $\preceq$ is called a* well-founded relation *over S if, for any element a in S, there is no infinite descending sequence $a \succ a_1 \succ a_2 \succ \ldots$ in S. The set S is called a* well-founded *set w.r.t. $\preceq$.*

For our soundness proof, we rely on the well-known well-founded set of natural numbers with the usual order $\leq$.

Proof Skeleton. Below we give the main skeleton of the proof of Theorem 5.1, deferring the details of the proof of Lemma 5.1 to Appendix A.1.

Following the main idea above, we introduce two kinds of sequences: *execution sequences* (Definition 5.2), which record the full intermediate-trace history generated during the execution of a regular program, and *counter-example sequences* (Definition 5.3), which collect the execution history that witnesses the failure of a PPL formula at a given trace. We then state Lemma 5.1, the key technical step in the proof of Theorem 5.1: along each step of an invalid derivation path, the minimum length of the relevant counter-example sequences does not increase, and it strictly decreases whenever the step is progressing.

Definition 5.2 (Execution Sequences). *An "execution sequence" (simply "sequence") is a sequence of traces. Given a regular program α and a starting trace tr, the set $ex(tr, \alpha)$ of execution sequences of α is defined as follows:*

$$ex(tr, \alpha) =_{df} \{(tr \cdot s_1)(tr \cdot s_1 s_2) \ldots (tr \cdot s_1 \ldots s_n) \mid s_1 s_2 \ldots s_n \in \mathcal{I}(\alpha), n > 1\}.$$

Each element of the sequence is enclosed in a pair of parentheses: $(\ldots)$.

We define the length of a sequence $(tr_1)(tr_2)\ldots(tr_n)$ $(n \geq 0)$ as the sum of the lengths of all of its traces:

$$|(tr_1)(tr_2)\ldots(tr_n)| =_{df} |tr_1| + |tr_2| + \cdots + |tr_n|.$$

We often use *seq* to denote an execution sequence.

Intuitively, an execution sequence of a program α starting from a trace tr records the full sequence of intermediate traces generated step by step during the execution of α. Rather than collapsing the execution into a single final trace, it retains each intermediate trace as a separate element, so that the sequence $(tr \cdot s_1)(tr \cdot s_1 s_2) \cdots (tr \cdot s_1 \cdots s_n)$ explicitly captures the history of how the execution progresses from tr through the successive transitions of α. The length of such a sequence is defined as the total number of states accumulated across all its component traces.

Example 5.1. *Let a_1 and a_2 be two actions, and suppose that $s_1 s_2 s_3 \in \mathcal{I}(a_1 ; a_2)$, with $s_1 s_2 \in \mathcal{I}(a_1)$ and $s_2 s_3 \in \mathcal{I}(a_2)$. Then $ex(s_1, a_1 ; a_2) = \{(s_1)(s_1 s_2)(s_1 s_2 s_3)\}$.*

Definition 5.3 (Counter-example Sequences). *Given a Kripke frame $(\mathcal{S}, \mathcal{I})$ and a dynamic formula ϕ, the set of "counter-example sequences" of ϕ beginning with a trace tr, denoted by $CES(tr, \phi)$, is defined according to the form of ϕ as follows:*

1. *$CES(tr, [\alpha]\psi) =_{df} \{seq \cdot seq' \mid seq \in ex(tr, \alpha), seq' \in CES(seq_e, \psi)\}$;*
2. *$CES(tr, \phi) =_{df} \{(tr) \mid tr \not\models \phi\}$, where ϕ is of any other form.*

Intuitively, item 1 captures the following idea. To construct a counter-example to $[\alpha]\psi$ starting from a trace tr, one first selects an execution sequence $seq \in ex(tr, \alpha)$, which records all intermediate traces generated step by step during the execution of α from tr, and then appends a counter-example sequence $seq' \in CES(seq_e, \psi)$ for ψ beginning from the last trace seq_e of seq. The resulting concatenation $seq \cdot seq'$ constitutes a counter-example sequence for $[\alpha]\psi$ starting from tr, and the full set $CES(tr, [\alpha]\psi)$ consists of all such concatenations. Item 2 assigns the singleton sequence (tr) to every formula not of the form $[\alpha]\psi$ that fails at tr.

Example 5.2. *Let $\mathcal{I}(\alpha)_s$ be the set of traces in $\mathcal{I}(\alpha)$ that start with the world s. In Example 4.2 (see Figure 1), for the formula $\tau_3 = x : [a][a^*]p$, let s_1 be an arbitrary world. Suppose that, for some worlds s_2, s_3, and s_4, $\mathcal{I}(a)_{s_1} = \{s_1s_2\}$ and $\mathcal{I}(a^*)_{s_2} = \{s_2s_2, s_2s_3\}$. Then, by Definition 5.2, $ex(s_1, a) = \{(s_1)(s_1s_2)\}$ and $ex(s_1s_2, a^*) = \{(s_1s_2)(s_1s_2s_2), (s_1s_2)(s_1s_2s_3)\}$. Based on $ex(s_1, a)$ and $ex(s_1s_2, a^*)$, we can compute: $CES(s_1, [a][a^*]p) = \{(s_1)(s_1s_2)(s_1s_2s_2), (s_1)(s_1s_2)(s_1s_2s_3)\}$ by Definition 5.3.*

Definition 5.4 (Minimum Length of Execution Sequences). *For a set A of execution sequences, $\min(A)$ denotes the length of the shortest sequence in A.*

We call $\mathfrak{m} \in \mathbf{M}$ a *counter-example mapping* of a node ν if it is one of the mappings that make ν invalid.

Lemma 5.1. *In a cyclic proof, let ν be an invalid conclusion of an instance of a rule, where $\mathfrak{m}$ is a counter-example mapping, and let $\tau =_{df} \sigma : \phi$ be an invalid trace element on the right side of ν. Then there exists an invalid premise ν' of the instance, a counter-example mapping $\mathfrak{m}'$ of ν' and an invalid formula $\tau' =_{df} \sigma' : \phi'$ on the right side of ν', such that (τ, τ') is a trace pair over (ν, ν'), and $\min(CES(\mathfrak{m}'(\sigma'), \phi')) \leq \min(CES(\mathfrak{m}(\sigma), \phi))$. Moreover, if (τ, τ') is a progressing step, then $\min(CES(\mathfrak{m}'(\sigma'), \phi')) < \min(CES(\mathfrak{m}(\sigma), \phi))$.*

The proof of this lemma is given in Appendix A.1. It proceeds by a case analysis on the derivation rule giving rise to (ν, ν'). For each progressing instance of $([a]R)$, $([\phi?]R)$, $(\mathbf{suf}\, R)$, or $(\mathbf{fus}\, R)$, the argument identifies a counter-example sequence for the premise whose length is strictly smaller than the minimum length for the conclusion; hence $\min(CES(\mathfrak{m}'(\sigma'), \phi')) < \min(CES(\mathfrak{m}(\sigma), \phi))$. For a non-progressing instance, it selects an invalid premise for which the corresponding minimum length does not increase, yielding $\min(CES(\mathfrak{m}'(\sigma'), \phi')) \leq \min(CES(\mathfrak{m}(\sigma), \phi))$.

Example 5.3. *Based on the setting in Example 5.2, Definition 5.3 also allows us to compute the set CES for the formula $\tau_4 = x \diamond y : [a^*]p$ in Figure 1 w.r.t. the trace s_1s_2: $CES(s_1s_2, [a^*]p) = \{(s_1s_2)(s_1s_2s_2), (s_1s_2)(s_1s_2s_3)\}$, in which sequences $(s_1s_2)(s_1s_2s_2)$ and $(s_1s_2)(s_1s_2s_3)$ are proper suffixes of sequences $(s_1)(s_1s_2)(s_1s_2s_2)$ and $(s_1)(s_1s_2)(s_1s_2s_3)$ in $CES(s_1, [a][a^*]p)$ respectively. Let $\mathfrak{m}$ be a mapping such that $\mathfrak{m}(x) = s_1$. By $\mathcal{I}(a)_{s_1}$ and Definition 4.5, we have $\mathfrak{m}(y) = s_2$. Therefore, for the progressing step (τ_3, τ_4) in the derivation of Figure 1, we have $\min(CES(\mathfrak{m}(x \diamond y), [a^*]p)) = \min(CES(s_1s_2, [a^*]p)) = 2 < 3 = \min(CES(s_1, [a][a^*]p)) = \min(CES(\mathfrak{m}(x), [a][a^*]p))$.*

Proof of Theorem 5.1. Let $\nu = (\cdot \Rightarrow \sigma : \phi)$. For contradiction, suppose $\not\models \sigma : \phi$; that is, ν is invalid. By the soundness of the rules in G3PPL, using Lemma 5.1, from ν, we can either conclude that some axiom is invalid, which contradicts its soundness, or obtain an invalid derivation path P. From a certain node of P, there exists a derivation trace $T = (\sigma_1 : \phi_1)(\sigma_2 : \phi_2) \ldots (\sigma_i : \phi_i) \ldots (i \geq 1)$ over P such that

$$\min(CES(\mathfrak{m}_1(\sigma_1), \phi_1)) \geq \min(CES(\mathfrak{m}_2(\sigma_2), \phi_2)) \geq \cdots \geq \min(CES(\mathfrak{m}_i(\sigma_i), \phi_i)) \geq \cdots .$$

Since P is progressing, Definition 4.7 implies that it must contain infinitely many instances of the rules $([a]R)$, $([\phi?]R)$, $(\mathbf{suf}\, R)$, or $(\mathbf{fus}\, R)$. By Definition 4.6, this means that in T, those trace pairs over the instances of these rules are

progressing steps. Therefore, by Lemma 5.1, the chain above must contain infinitely many strict inequalities $>$. This contradicts the well-foundedness of the natural numbers under the relevant strict order.

□

6 Completeness of G3PPL

We analyze the completeness of G3PPL for any PPL formula ϕ, captured by its equivalent labelled form $X : \phi$ with $X \in \mathit{TVar}$.

Theorem 6.1 (Completeness of G3PPL). *Given a labelled PPL formula $X : \phi$ with $X \in \mathit{TVar}$, if $\models X : \phi$, then $\vdash X : \phi$.*

Main Idea of the Proof of Theorem 6.1 The proof mainly follows the standard approach for establishing the completeness of labelled proof systems (cf. [8, 12]): we show that G3PPL is capable of deriving every axiom and rule of the proof system of PPL (Table 2). Since the proof system of PPL is complete [17], this suffices to conclude the completeness of G3PPL.

Below we explain how to derive each rule in Table 2.

Necessitation Lemma. We first establish a Necessitation Lemma (Lemma 6.1) as a generalization of rule (*Gen*) in Table 2. This lemma is required when deriving the other rules. We then derive the PDL rules, following the proof strategy of [12] but adapting it to the trace labels of process logic. Finally, we derive the rules specific to PPL. This final step is the core of the completeness proof for the process-logic part.

For a set A of formulas, we use $X : A$ to denote the set $\{X : \phi \mid \phi \in A\}$; we use $\mathit{op}\, A$, where $\mathit{op} \in \{[\alpha], \langle\alpha\rangle\}$, to denote the set $\{\mathit{op}\,\phi \mid \phi \in A\}$.

Lemma 6.1 (Necessitation). *For any sequent $X : \mathit{op}\,\Gamma \Rightarrow X : \mathit{op}\,\Delta$, where $\mathit{op} \in \{[\alpha], \langle\alpha\rangle\}$, there is an "almost" cyclic derivation of it: every infinite derivation path has a progressing derivation trace, except that the derivation may have leaves of the form $X : \Gamma \Rightarrow X : \Delta$ that are neither terminal nodes nor buds. Moreover, there is a progressing trace along each derivation path from the root to such a leaf.*

Below, we use (*nec*) to represent the derivation of Lemma 6.1.

Proof of Lemma 6.1. We consider only the case when *op* is of the form $[\alpha]$; the case when *op* is $\langle\alpha\rangle$ then follows immediately, since $\langle\alpha\rangle\phi = \neg[\alpha]\neg\phi$ and the rules $(\neg L)$ and $(\neg R)$ reduce any sequent involving $\langle\alpha\rangle$ to the corresponding sequent involving $[\alpha]$. And we only consider the case in which Γ and Δ each contain exactly one formula. The general case follows by the same argument applied independently to each formula in Γ and Δ using the same set of rules.

Let $\Gamma = \{\phi\}$, $\Delta = \{\varphi\}$. We prove by induction on the structure of regular programs.

When α is an atomic program a, without loss of generality, let $X = Y \diamond x$. We have the following derivation:

$$\dfrac{\dfrac{\dfrac{\dfrac{Y \diamond x : \phi \Rightarrow Y \diamond x : \varphi}{Y \diamond y : \phi \Rightarrow Y \diamond y : \varphi}\,(sub)}{Y \diamond x \diamond y : \phi \Rightarrow Y \diamond x \diamond y : \varphi}\,(sub)}{x\,\mathrm{R}_a\,y, Y \diamond x : [a]\phi \Rightarrow Y \diamond x \diamond y : \varphi}\,([a]L)}{Y \diamond x : [a]\phi \Rightarrow Y \diamond x : [a]\varphi}\,([a]R)$$

The derivation above is progressing since rule $([a]R)$ is applied (see Definition 4.7). In the first application of rule (*sub*), we instantiate the label variable by $Y \diamond x \diamond y = (Y \diamond y)[Y \diamond x/Y]$. In the second application of (*sub*), we instantiate by $y = x[y/x]$.

The case when α is a test $\psi?$ is similar. We omit it here.

When α is a sequence program $\alpha_1\,;\alpha_2$, we have

$$\dfrac{\dfrac{\dfrac{X:\phi \Rightarrow X:\varphi}{X:[\alpha_2]\phi \Rightarrow X:[\alpha_2]\varphi}\,IH}{X:[\alpha_1][\alpha_2]\phi \Rightarrow X:[\alpha_1][\alpha_2]\varphi}\,IH}{X:[\alpha_1\,;\alpha_2]\phi \Rightarrow X:[\alpha_1\,;\alpha_2]\varphi}\,([;]),$$

where the steps *IH* use the induction hypothesis on α_1 and α_2, respectively. By the induction hypothesis, the derivation is progressing.

When α is a choice program $\alpha_1 \cup \alpha_2$, we have

$$\dfrac{\dfrac{\dfrac{X:\phi \Rightarrow X:\varphi}{X:[\alpha_1]\phi \Rightarrow X:[\alpha_1]\varphi}\,IH}{X:[\alpha_1]\phi, X:[\alpha_2]\phi \Rightarrow X:[\alpha_1]\varphi}\,(wk) \quad \dfrac{\dfrac{X:\phi \Rightarrow X:\varphi}{X:[\alpha_2]\phi \Rightarrow X:[\alpha_2]\varphi}\,IH}{X:[\alpha_1]\phi, X:[\alpha_2]\phi \Rightarrow X:[\alpha_2]\varphi}\,(wk)}{X:[\alpha_1\cup\alpha_2]\phi \Rightarrow X:[\alpha_1\cup\alpha_2]\varphi}\,([\cup]),$$

where the steps *IH* use the induction hypothesis on α_1 and α_2, respectively. From the induction hypothesis, we see that the two derivation branches are progressing.

When α is a star program α_1^*, let $X = Y \diamond x$. We have

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{Y\diamond x:\phi \Rightarrow Y\diamond x:\varphi}{Z\diamond x:\phi \Rightarrow Z\diamond x:\varphi}\,(sub)}{Y\diamond x\diamond x:\phi \Rightarrow Y\diamond x\diamond x:\varphi}\,(sub)}{Y\diamond x\diamond x:\phi, Y\diamond x:[\alpha_1][\alpha_1^*]\phi \Rightarrow Y\diamond x\diamond x:\varphi}\,(wk)}{Y\diamond x:[true?]\phi, Y\diamond x:[\alpha_1][\alpha_1^*]\phi \Rightarrow Y\diamond x:[true?]\varphi}\,([\phi?]L,[\phi?]R)}{Y\diamond x:[\alpha_1^*]\phi \Rightarrow Y\diamond x:[true?]\varphi}\,([*]) \quad \text{Cont.}}{Y\diamond x:[\alpha_1^*]\phi \Rightarrow Y\diamond x:[\alpha_1^*]\varphi}\,([*]),$$

where Cont. is the derivation as follows:

$$\dfrac{\dfrac{\dfrac{Y\diamond x:[\alpha_1^*]\phi \Rightarrow Y\diamond x:[\alpha_1^*]\varphi\ (\text{bud})}{Y\diamond x:[\alpha_1][\alpha_1^*]\phi \Rightarrow Y\diamond x:[\alpha_1][\alpha_1^*]\varphi}\,IH}{Y\diamond x:[true?]\phi, Y\diamond x:[\alpha_1][\alpha_1^*]\phi \Rightarrow Y\diamond x:[\alpha_1][\alpha_1^*]\varphi}\,(wk)}{\text{Cont.}: Y\diamond x:[\alpha_1^*]\phi \Rightarrow Y\diamond x:[\alpha_1][\alpha_1^*]\varphi}\,[*].$$

The left derivation branch is progressing because the rule $([\phi?]R)$ is applied; on the right derivation branch (from "Cont."), the induction-hypothesis step *IH* on α_1 shows that the derivation path from the root node to its bud is progressing. □

Proving the Rules for PDL. The Necessitation lemma is in fact a general form of the rule (*Gen*). Thus from it we can trivially derive (*Gen*). For the other rules of PDL shown in Table 2, the derivations of rules (3)–(6) follow directly by applying the rules $([\cup])$, $([;])$, $([\phi?])$, and $([*])$ of G3PPL. The derivation of rule (*MP*) is a special case of rule (*cut*) of G3PPL. Below, we only show the derivations for the rules (1), (2), (7) in G3PPL. The key idea is to apply the Necessitation lemma to reduce a goal to either terminal nodes or some buds so that a cyclic proof can be formed.

The derivation for formula $X : [\alpha](\phi \to \varphi) \to ([\alpha]\phi \to [\alpha]\varphi)$ with $X \in \mathit{TVar}$ (corresponding to rule (1)) is shown as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{X:\phi \Rightarrow X:\phi, X:\varphi}\,(ax) \quad \dfrac{}{X:\varphi, X:\phi \Rightarrow X:\varphi}\,(ax)}{X:(\phi\to\varphi), X:\phi \Rightarrow X:\varphi}\,(\to L)}{X:[\alpha](\phi\to\varphi), X:[\alpha]\phi \Rightarrow X:[\alpha]\varphi}\,nec}{X:[\alpha](\phi\to\varphi) \Rightarrow X:([\alpha]\phi \to [\alpha]\varphi)}\,(\to R)}{\cdot \Rightarrow X:[\alpha](\phi\to\varphi) \to ([\alpha]\phi \to [\alpha]\varphi)}\,(\to R)$$

The derivation for formula $X : [\alpha](\phi \wedge \varphi) \leftrightarrow ([\alpha]\phi \wedge [\alpha]\varphi)$ with $X \in \mathit{TVar}$ (corresponding to rule (2)) is shown as follows for both directions separately:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{X:\phi, X:\varphi \Rightarrow X:\phi}\,(ax)}{X:(\phi\wedge\varphi) \Rightarrow X:\phi}\,(\wedge L)}{X:[\alpha](\phi\wedge\varphi) \Rightarrow X:[\alpha]\phi}\,nec \quad \dfrac{\dfrac{\dfrac{}{X:\phi, X:\varphi \Rightarrow X:\varphi}\,(ax)}{X:(\phi\wedge\varphi) \Rightarrow X:\varphi}\,(\wedge L)}{X:[\alpha](\phi\wedge\varphi) \Rightarrow X:[\alpha]\varphi}\,nec}{X:[\alpha](\phi\wedge\varphi) \Rightarrow X:([\alpha]\phi \wedge [\alpha]\varphi)}\,(\wedge R)}{\cdot \Rightarrow X:[\alpha](\phi\wedge\varphi) \to ([\alpha]\phi \wedge [\alpha]\varphi)}\,(\to R)\quad,$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{X:\phi, X:\varphi \Rightarrow X:\phi}\,(ax) \quad \dfrac{}{X:\phi, X:\varphi \Rightarrow X:\varphi}\,(ax)}{X:\phi, X:\varphi \Rightarrow X:(\phi\wedge\varphi)}\,(\wedge R)}{X:[\alpha]\phi, X:[\alpha]\varphi \Rightarrow X:[\alpha](\phi\wedge\varphi)}\,nec}{X:([\alpha]\phi \wedge [\alpha]\varphi) \Rightarrow X:[\alpha](\phi\wedge\varphi)}\,(\wedge L)}{\cdot \Rightarrow X:([\alpha]\phi \wedge [\alpha]\varphi) \to [\alpha](\phi\wedge\varphi)}\,(\to R)$$

The derivation for the formula $X : \phi \wedge [\alpha^*](\phi \to [\alpha]\phi) \to [\alpha^*]\phi$ with $X \in \mathit{TVar}$ (corresponding to rule (7)) is shown as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{\text{Lemma 6.2}}{X:\phi, X:[\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\mathit{true}?]\phi}\quad \text{Cont.}}{X:\phi, X:[\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha^*]\phi\ \text{(companion)}}\,([*])}{X:\phi\wedge[\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha^*]\phi}\,(\wedge L)}{\cdot \Rightarrow X:\phi\wedge[\alpha^*](\phi\to[\alpha]\phi) \to [\alpha^*]\phi}\,(\to R)\quad,$$

where Cont. is the derivation shown below:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{X:\phi, X:[\alpha][\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha][\alpha^*]\phi, X:\phi}\,(ax) \quad \text{Cont. 2}}{X:\phi, X:(\phi\to[\alpha]\phi), X:[\alpha][\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha][\alpha^*]\phi}\,(\to L)}{X:\phi, X:[\mathit{true}?](\phi\to[\alpha]\phi), X:[\alpha][\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha][\alpha^*]\phi}\,(cut, lem)}{\text{Cont.} : X:\phi, X:[\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha][\alpha^*]\phi}\,[*]\quad,$$

Cont. 2 is the derivation shown below:

$$\dfrac{\dfrac{X:\phi, X:[\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha^*]\phi\ \text{(bud)}}{X:[\alpha]\phi, X:[\alpha][\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha][\alpha^*]\phi}\,nec}{\text{Cont. 2} : X:\phi, X:[\alpha]\phi, X:[\alpha][\alpha^*](\phi\to[\alpha]\phi) \Rightarrow X:[\alpha][\alpha^*]\phi}\,(wk).$$

The "bud" back-links to the "companion". The derivation above also relies on Lemma 6.2 below, tagged as the derivation step "lem". Its proof is given in Appendix 6.

$$\dfrac{\dfrac{\dfrac{\surd}{iii.a11 - a13}\,(\vee R,ax) \quad \dfrac{(iii.a141)\ (\text{bud})}{(iii.a14)}\,(wk)}{(iii.a1)\ (\text{comp.})}\,(\textbf{suf}\,R,\textbf{suf}\,L) \quad \dfrac{\dfrac{\surd}{iii.a21 - a23}\,(\vee R,ax) \quad \dfrac{(iii.a241)\ (\text{bud})}{(iii.a24)}\,(wk)}{(iii.a2)\ (\text{comp.})}\,(\textbf{suf}\,R,\textbf{suf}\,L)}{(iii.a)}\,(\vee L)$$

Fig. 2. Derivation process of critical nodes for sequent (iii.a)

Lemma 6.2. *For any $X \in TVar$ and formula ϕ, sequents $X : \phi \Rightarrow X : [true?]\phi$ and $X : [true?]\phi \Rightarrow X : \phi$ are provable in G3PPL.*

The derivation step "(cut, *lem*)" above is shorthand for first applying (cut) (with the additional formula $X : (\phi \to [\alpha]\phi)$) and then applying Lemma 6.2 to prove $\ldots X : [true?](\phi \to [\alpha]\phi) \ldots \Rightarrow \ldots X : (\phi \to [\alpha]\phi) \ldots$. The derivation path from the node "companion" to the node "bud" is progressing because the Necessitation lemma (*nec*) is applied.

Proving the Rules Specific to PPL. It remains to prove the special rules (i)–(xv) for PPL in Table 2. The derivations for rules (i)–(xv) follow a uniform pattern: each axiom is lifted to its labelled form, the structural rules of G3PPL together with the trace rules $(\textbf{suf}\,R)$, $(\textbf{suf}\,L)$, $(\textbf{fus}\,R)$, $(\textbf{fus}\,L)$ and $(\textbf{f})$ are used to unfold the temporal connectives and program structures on the label, and the resulting propositional residuals are closed by (ax). Each rule is applied backwardly (from the conclusion to the premises). Cyclic constructions arise for rules involving the suffix operator and programs, where a companion–bud pair is needed; progressiveness is guaranteed at each back-link by a prior application of $(\textbf{suf}\,R)$, $(\textbf{fus}\,R)$, or the Necessitation lemma. Full derivations, except for one direction of rule (iii) given below, appear in Appendix A.2.

As a representative example, below we give the derivation for one direction of rule (iii).

Proof of One Direction of Proposition A.3. Let $X \in TVar$. We need to prove

$$(iii.a)\ X : (\phi\ \textbf{suf}\ \psi) \vee (\phi\ \textbf{suf}\ \chi) \Rightarrow X : \phi\ \textbf{suf}(\psi \vee \chi).$$

Without loss of generality, let $X = x \diamond Y$. By $(\vee L)$, we need

$(iii.a1)$ $x \diamond Y : (\phi\ \textbf{suf}\ \psi) \Rightarrow x \diamond Y : \phi\ \textbf{suf}(\psi \vee \chi)$.
$(iii.a2)$ $x \diamond Y : (\phi\ \textbf{suf}\ \chi) \Rightarrow x \diamond Y : \phi\ \textbf{suf}(\psi \vee \chi)$.

Note that, by $(\textbf{suf}\,x)$, the case in which $Y = \varepsilon$ is trivial. Thus, we consider only the case in which $Y \neq \varepsilon$.

For (iii.a1), by applying $(\textbf{suf}\,R)$ and $(\textbf{suf}\,L)$ on both sides, we have

$(iii.a11)$ $Y : \psi \Rightarrow Y : \psi \vee \chi, Y : \phi$, which can be proved by $(\vee R)$ and (ax).
$(iii.a12)$ $Y : \phi, Y : \phi\ \textbf{suf}\ \psi \Rightarrow Y : \psi \vee \chi, Y : \phi$, which can be proved by (ax).
$(iii.a13)$ $Y : \psi \Rightarrow Y : \psi \vee \chi, Y : \phi\ \textbf{suf}(\psi \vee \chi)$, which can be proved by $(\vee R)$ and (ax).
$(iii.a14)$ $Y : \phi, Y : \phi\ \textbf{suf}\ \psi \Rightarrow Y : \psi \vee \chi, Y : \phi\ \textbf{suf}(\psi \vee \chi)$.

From $(iii.a14)$, by (wk), we have

$$(iii.a141)\quad Y : \phi\ \textbf{suf}\ \psi \Rightarrow Y : \phi\ \textbf{suf}(\psi \vee \chi),$$

which is exactly (iii.a1) (by replacing Y with X). The derivation path from (iii.a1) to (iii.a141) is progressing since during the process $(\textbf{suf}\,R)$ is applied.

The case of (iii.a2) is similar. Applying $(\textbf{suf}\,R)$ and $(\textbf{suf}\,L)$ yields

$(iii.a21)$ $Y : \chi \Rightarrow Y : \psi \vee \chi, Y : \phi$, which can be proved by $(\vee R)$ and (ax).
$(iii.a22)$ $Y : \phi, Y : \phi \,\mathbf{suf}\, \chi \Rightarrow Y : \psi \vee \chi, Y : \phi$, which can be proved by (ax).
$(iii.a23)$ $Y : \chi \Rightarrow Y : \psi \vee \chi, Y : \phi \,\mathbf{suf}(\psi \vee \chi)$, which can be proved by $(\vee R)$ and (ax).
$(iii.a24)$ $Y : \phi, Y : \phi \,\mathbf{suf}\, \chi \Rightarrow Y : \psi \vee \chi, Y : \phi \,\mathbf{suf}(\psi \vee \chi)$.

From $(iii.a24)$, by (wk), we have

$$(iii.a241)\quad Y : \phi \,\mathbf{suf}\, \chi \Rightarrow Y : \phi \,\mathbf{suf}(\psi \vee \chi),$$

which is exactly (iii.a2) (by replacing Y with X). The derivation path from (iii.a2) to (iii.a241) is progressing because $(\mathbf{suf}\, R)$ is applied during the derivation from (iii.a2) to (iii.a24), with $(x \diamond Y : \phi \,\mathbf{suf}(\psi \vee \chi), Y : \phi \,\mathbf{suf}(\psi \vee \chi))$ as the progressing step.

□

Figure 2 shows the derivation process for the critical nodes appearing in the above proof, from which we can clearly see how the nodes terminate and how the back-links are formed. All the proofs for the rest of the rules in Appendix A.2 follow similar derivation processes.

Note: In the proofs in Appendices A.2 and A.3, unless explicitly stated otherwise, we omit the case in which a label variable is the empty label ε because these cases are trivial.

7 Labelled First-Order Process Logic

At the first-order level of process logic, assignments generate program-state information that must be tracked explicitly throughout derivations. We address this by replacing the "trace-world variables" (*WVar*) of PPL labels with *program updates*, which record the cumulative effect of assignments as structured label components. The following subsections define update-based labels and labelled FOPL formulas, present the proof system G3FOPL, and analyze its soundness and relative completeness.

7.1 Labelled FOPL Formulas

Program updates, introduced in [2] as a first-order label structure for capturing program configurations during derivations, provide the foundation for the labelled treatment of FOPL. An update $\mathcal{U}$ records the accumulated effect of assignment steps as an explicit substitution applied to the current world variable, making the program configuration available inside the proof system without requiring rule-specific axioms for each assignment form.

Definition 7.1 (Program Updates). *A program update $\mathcal{U}$ in FOPL is defined as a first-order configuration as follows:*

$$\mathcal{U} =_{df} x \mid \{u := e\}\mathcal{U},$$

where $x \in WVar$, $u \in Var$ and e is an expression in $\mathfrak{A}$.

We use $\mathbf{U}$ to represent the set of all updates in FOPL. $\{u := e\}$ captures the effect of assigning the value of e to the variable u in the current context.

The labels of FOPL enrich those of PPL with the explicit structures of program updates $\mathcal{U}$. We give the definitions of labels and label mappings of FOPL in the following definitions.

Definition 7.2 (Labels of FOPL). *A label σ in FOPL is a trace defined as follows:*

$$\sigma =_{df} \varepsilon \mid \mathcal{U} \mid X \mid \sigma \diamond \sigma,$$

where $X \in TVar$, ε is the empty label, $\diamond$ is the concatenation operator, and $\mathcal{U}$ is a program update.

We use $\mathbf{L}_{fo}$ to denote the set of all labels of FOPL.

Definition 7.3 (Label mappings of FOPL). *A world mapping $\mathfrak{w}$ in FOPL is a function from $\mathbf{U}$ to $\mathcal{S}_{fo}$, defined inductively as follows:*

1. *$\mathfrak{w}$ maps a variable $x \in WVar$ to a world in $\mathcal{S}_{fo}$.*
2. *$\mathfrak{w}(\{u := e\}\mathcal{U}) =_{df} \mathfrak{w}(\mathcal{U})[u \mapsto \mathfrak{w}(\mathcal{U})(e)]$.*

A label mapping $\mathfrak{m}$ is a function from $\mathbf{L}_{fo}$ to $\mathcal{S}_{fo}^*$, defined as follows:

i *$\mathfrak{m}$ maps a variable $X \in TVar$ to a trace of worlds in $\mathcal{S}_{fo}^*$.*
ii *$\mathfrak{m}(\varepsilon) =_{df} \epsilon$.*
iii *$\mathfrak{m}(\mathcal{U}) =_{df} \mathfrak{w}(\mathcal{U})$.*
iv *$\mathfrak{m}(\sigma_1 \diamond \sigma_2) =_{df} \mathfrak{m}(\sigma_1) \cdot \mathfrak{m}(\sigma_2)$.*

We use $\mathbf{M}_{fo}$ to denote the set of all label mappings from $\mathbf{L}_{fo}$ to $\mathcal{S}_{fo}^*$.

In FOPL, label relations are no longer needed since actions are explicit assignments. A *labelled FOPL formula* is defined exactly as for PPL (Definition 4.4), but with $\mathbf{L}$ replaced by $\mathbf{L}_{fo}$.

The semantics of labelled FOPL formulas is introduced as follows.

Definition 7.4 (Semantics of Labelled FOPL Formulas). *Given the Kripke frame $(\mathcal{S}_{fo}, \mathcal{I}_{fo})$ of FOPL and the set $\mathbf{M}_{fo}$ of label mappings, the satisfaction relation $\mathfrak{m} \models_{\mathbf{M}_{fo}} \sigma : \phi$ of a labelled formula $\sigma : \phi$ by a label mapping $\mathfrak{m}$ (w.r.t. $\mathbf{M}_{fo}$) is defined such that*

$$\mathfrak{m} \models_{\mathbf{M}_{fo}} \sigma : \phi, \text{ if } \mathfrak{m}(\sigma) \models_{(\mathcal{S}_{fo}, \mathcal{I}_{fo})} \phi.$$

7.2 System G3FOPL

The proof system G3FOPL for labelled FOPL formulas extends G3PPL by specializing the rules for atomic programs and first-order arithmetic formulas, while keeping all other rules unchanged. As shown in Table 7, the rule $([u := e])$ replaces the pair $([a]R)$ and $([a]L)$ in G3PPL: it appends the update $\{u := e\}$ to the label, recording the assignment effect in the label structure rather than substituting into the formula. The rule (at) replaces rule (p) for the evaluation of arithmetic predicates at an update: it strips the trailing label σ and evaluates $e_1 \triangle e_2$ at the update $\mathcal{U}$ alone. The rule (ter) closes a derivation branch when all remaining formulas are arithmetic sequents of the form $\mathcal{U} : e_1 \triangle e_2$; such sequents can be discharged by an external arithmetic decision procedure or SMT solver. The rules $(\forall R)$ and $(\forall L)$ handle universal quantification in the standard way as in FODL.

All other rules of G3FOPL are inherited from G3PPL with labels interpreted in $\mathbf{L}_{fo}$, and with each world variable x in those rules replaced by a program update $\mathcal{U} \in \mathbf{U}$. For example, rule $(\mathbf{suf}\ R)$ in G3FOPL takes the form

$$\frac{\Gamma \Rightarrow \sigma : \psi, \sigma : \phi, \Delta \quad \Gamma \Rightarrow \sigma : \psi, \sigma : \phi\ \mathbf{suf}\ \psi, \Delta}{\Gamma \Rightarrow \mathcal{U} \diamond \sigma : \phi\ \mathbf{suf}\ \psi, \Delta}\ (\mathbf{suf}\ R),$$

with $\sigma \in \mathbf{L}_{fo}$, following the same trace-reduction principle as in G3PPL.

Table 8 lists the derived rules for the non-primitive operators $\langle\cdot\rangle$ and $\exists$ in G3FOPL. Each follows directly from the rules in Table 7.

The soundness of the G3FOPL rules follows directly from the semantics of labelled FOPL formulas.

Theorem 7.1. *Each rule of G3FOPL is sound.*

Rule	Schematic form
Inherited rules	All rules of G3PPL except $([a]R)$ and $([a]L)$, with labels interpreted in $\mathbf{L}_{fo}$
$([u := e])$	$\dfrac{\sigma \diamond \mathcal{U} \diamond \{u := e\}\mathcal{U} : \phi}{\sigma \diamond \mathcal{U} : [u := e]\phi}$
(at)	$\dfrac{\mathcal{U} : e_1 \triangle e_2}{\mathcal{U} \diamond \sigma : e_1 \triangle e_2}$, where $\triangle \in \{<, \leq, >, \geq, =\}$
(ter)	$\dfrac{}{\Gamma \Rightarrow \Delta}$, where each formula of $\Gamma \Rightarrow \Delta$ is of the form $\mathcal{U} : e_1 \triangle e_2$, and the formula $\bigwedge_{\phi \in \Gamma} \phi \rightarrow \bigvee_{\psi \in \Delta} \psi$ is valid
$(\forall R)$	$\dfrac{\Gamma \Rightarrow \sigma : \phi[v/u], \Delta}{\Gamma \Rightarrow \sigma : \forall u.\phi, \Delta}$, where $v \in \mathit{Var}$ is fresh w.r.t. Γ, Δ, ϕ
$(\forall L)$	$\dfrac{\Gamma, \sigma : \phi[e/u] \Rightarrow \Delta}{\Gamma, \sigma : \forall u.\phi \Rightarrow \Delta}$, where e is a term

Table 7. Rules specific to labelled first-order process logic.

Rule	Schematic form
$(\langle u := e\rangle)$	$\dfrac{\sigma \diamond \mathcal{U} \diamond \{u := e\}\mathcal{U} : \phi}{\sigma \diamond \mathcal{U} : \langle u := e\rangle\phi}$
$(\exists R)$	$\dfrac{\Gamma \Rightarrow \phi[e/u], \Delta}{\Gamma \Rightarrow \exists u.\phi, \Delta}$, where e is a term
$(\exists L)$	$\dfrac{\Gamma, \phi[v/u] \Rightarrow \Delta}{\Gamma, \exists u.\phi \Rightarrow \Delta}$, where $v \in \mathit{Var}$ is fresh w.r.t. Γ, Δ, ϕ

Table 8. Derived rules used in G3FOPL.

The cyclic proof structure of G3FOPL is defined in the same way as that of G3PPL, except that, for its derivation traces and progressing derivation traces (Definitions 4.6 and 4.7), trace pairs and progressing steps no longer correspond to instances of the rule $([a]R)$ but to instances of its counterpart $([u := e])$ in G3FOPL.

Similarly, in G3FOPL, we say that a sequent is also provable when there is a cyclic proof of it. For a labelled FOPL formula τ, we write G3FOPL $\vdash \tau$ (or simply $\vdash \tau$) if the sequent $\cdot \Rightarrow \tau$ is provable in G3FOPL.

7.3 Soundness and Relative Completeness of G3FOPL

We discuss the soundness and completeness of the proof system G3FOPL with respect to all FOPL formulas. The soundness proof for G3FOPL follows the same structure as that of G3PPL, adapting the cases for the rules $([u := e])$, $(\forall R)$, and $(\forall L)$ that are specific to the first-order setting. The completeness of G3FOPL is obtained by adapting the standard completeness argument for FODL [17] to the trace semantics of FOPL.

Theorem 7.2 (Soundness of G3FOPL). *For any labelled FOPL formula $X : \phi$, where $X \in \mathit{TVar}$, if $\vdash X : \phi$, then $\models X : \phi$.*

The proof of Theorem 7.2 proceeds along the same lines as the proof of soundness for G3PPL. The argument is modified in two respects. First, the case for progressing steps corresponding to the rule $([a]R)$ in G3PPL is replaced by the case for the rule $([u := e])$ in G3FOPL; the proof carries through essentially unchanged because the update rule records the effect of the assignment in the label and the same multiset-ordering argument applies. Second, the cases for

the rules ($\forall R$) and ($\forall L$) must be handled; these are straightforward and proceed similarly to the case for the rule (*sub*) in G3PPL. We omit the proof of Theorem 7.2 in this paper.

Theorem 7.3 (Completeness of G3FOPL). *For a labelled FOPL formula $X : \phi$, where $X \in TVar$ is a label variable, if $\models X : \phi$, then $\vdash X : \phi$.*

The completeness of G3FOPL adapts the main approach for the completeness of FODL [15, 17] to trace formulas and trace semantics of FOPL. The argument proceeds in two steps. We first establish two auxiliary lemmas—Lemma 7.1 and Lemma 7.2—that are required for the main completeness argument. The full proof of Theorem 7.3 then follows from these two lemmas and the structure of FOPL formulas.

Lemma 7.1. *For any FOPL formula ϕ, there exists a pure arithmetical first-order logic (FOL) formula $\psi^\flat$ such that $\models \psi^\flat \leftrightarrow \phi$.*

We usually write a formula ϕ as $\phi^\flat$ to indicate that it is a pure arithmetical FOL formula.

We omit the proof of Lemma 7.1. Here is an informal explanation. It is well known that temporal properties can be encoded in first-order form over suitable trace positions (cf. [28]). On the other hand, any FODL formula can be expressed by a pure arithmetical FOL formula [15, 17]. The result of Lemma 7.1 follows directly by combining these two facts.

Lemma 7.2. *For any FOPL formulas ϕ and ψ, label variable X, and $op \in \{[\alpha], \langle\alpha\rangle\}$ for any program α, if $\models X : \phi^\flat \Rightarrow X : op\,\psi^\flat$, then $\vdash X : \phi^\flat \Rightarrow X : op\,\psi^\flat$.*

The proof of this lemma is given in Appendix A.3.

The proof of Theorem 7.3 follows the main skeleton of the proof for FODL [15], but uses a refined induction that also considers the lengths of the labels of FOPL formulas.

Proof of Theorem 7.3. For a labelled formula $X : \phi$, ϕ is semantically equivalent to a formula in conjunctive normal form, $C_1 \wedge \cdots \wedge C_n$ ($n \geq 1$). Each clause C_i ($1 \leq i \leq n$) is a disjunction of literals: $C_i = l_{i,1} \vee \cdots \vee l_{i,m_i}$, where $l_{i,j}$ ($1 \leq i \leq n, 1 \leq j \leq m_i$) is an atomic FODL formula or its negation. By the rule for FOL formulas (Table 5 plus the rules ($\forall R$) and ($\forall L$)), to prove formula $X : \phi$, it is enough to show that for each clause C_i, $\models X : C_i$ implies $\vdash X : C_i$. We prove the result by simultaneous induction on both the number n of occurrences of the modalities $[\cdot]$ and $\langle\cdot\rangle$ in C_i and the length of the label X.

If $n = 0$ and the length of X is 1, there are no occurrences of $[\alpha]$ or $\langle\alpha\rangle$ in C_i, so C_i is an FOL formula. By the rule (*ter*), we immediately have $\vdash X : C_i$.

Suppose $n > 0$ and the length of X is $N > 0$. Without loss of generality, assume that C_i has one of the following forms: $\mathbf{f}\,\psi$, $\psi_1\ \mathbf{suf}\ \psi_2$, or $\psi_1 \vee op\,\psi_2$.

If $C_i = \psi_1\ \mathbf{suf}\ \psi_2$ (the case for $C_i = \mathbf{f}\,\psi$ is similar), without loss of generality, let $X = \mathcal{U} \diamond Y$ for some $\mathcal{U} \in \mathbf{U}$. Since $\models \mathcal{U} \diamond Y : \psi_1\ \mathbf{suf}\ \psi_2$ entails $\models Y : \psi_2 \vee (\psi_1 \wedge \psi_1\ \mathbf{suf}\ \psi_2)$ by the semantics of **suf**, we have (a) $\models Y : \psi_2$ or $\models Y : \psi_1$ and (b) $\models Y : \psi_2$ or $\models Y : \psi_1\ \mathbf{suf}\ \psi_2$. By the induction hypothesis (the length of Y is $N - 1$), we obtain that (1) $\vdash Y : \psi_2$ or $\vdash Y : \psi_1$, and (2) $\vdash Y : \psi_2$ or $\vdash Y : \psi_1\ \mathbf{suf}\ \psi_2$. Therefore, by ($\mathbf{suf}\ R$), $\vdash \mathcal{U} \diamond Y : \psi_1\ \mathbf{suf}\ \psi_2$.

If $C_i = \psi_1 \vee op\,\psi_2$, where $op \in \{[\alpha], \langle\alpha\rangle\}$, proving $\vdash X : (\psi_1 \vee op\,\psi_2)$ is equivalent to proving

$$\vdash X : \neg\psi_1 \Rightarrow X : op\,\psi_2$$

by $(\vee R)$ and $(\neg R)$. By Lemma 7.1, there exist pure arithmetical FOL formulas $\phi_1^\flat$ and $\phi_2^\flat$ such that $\models \phi_1^\flat \leftrightarrow \neg\psi_1$ and $\models \phi_2^\flat \leftrightarrow \psi_2$. Therefore, we have both $\models X : \phi_1^\flat \Rightarrow X : op\,\phi_2^\flat$ and $\models (X : \phi_2^\flat \Rightarrow X : \psi_2)$. From $\models X : \phi_1^\flat \Rightarrow X : op\,\phi_2^\flat$, by Lemma 7.2, we have

$$\vdash X : \phi_1^\flat \Rightarrow X : op\,\phi_2^\flat. \tag{1}$$

By the induction hypothesis on n, from $\models X : \neg\psi_1 \Rightarrow X : \phi_1^\flat$ (because $\models \phi_1^\flat \leftrightarrow \neg\psi_1$), we have

$$\vdash X : \neg\psi_1 \Rightarrow X : \phi_1^\flat. \tag{2}$$

Therefore, by (1) and (2), using (cut), we get

$$\vdash X : \neg\psi_1 \Rightarrow X : op\,\phi_2^\flat. \tag{3}$$

On the other hand, from $\models (X : \phi_2^\flat \Rightarrow X : \psi_2)$, by the induction hypothesis on n, we have $\vdash X : \phi_2^\flat \Rightarrow X : \psi_2$. By the Necessitation lemma, we have

$$\vdash X : op\,\phi_2^\flat \Rightarrow X : op\,\psi_2. \tag{4}$$

The result is directly obtained by (3) and (4).

□

8 Conclusion

This paper firstly develops a complete labelled proof-theoretic framework for process logic. The distinguishing idea is that the labelling discipline makes trace-execution information explicit in derivations. Thus, rather than distributing a temporal property over the subprograms of a program, the framework can defer reasoning about that property until after the program has been executed. Our central contribution is a uniform, modular, and cyclic logical framework at both the propositional level (G3PPL) and the first-order level (G3FOPL). We make a full analysis and prove the soundness and completeness/relative completeness for G3PPL and G3FOPL respectively. Together, these results show that labelled process logic provides a promising framework for reasoning about trace-based program properties.

One direction for future work is to extend labelled process logic with the omega-iteration operator α^ω to cover temporal properties over infinite program traces [17]. We will also explore applications of G3FOPL to practical programs and languages, develop automated proof-search strategies, and mechanize the labelled frameworks in Rocq (formerly Coq) [7].

References

[1] Hammad Ahmad and Jean-Baptiste Jeannin. 2021. A Program Logic to Verify Signal Temporal Logic Specifications of Hybrid Systems. In *Proceedings of the 24th ACM International Conference on Hybrid Systems: Computation and Control (HSCC 2021)*. ACM, Nashville, TN, USA, 1–11. https://doi.org/10.1145/3447928.3456648

[2] Bernhard Beckert and Daniel Bruns. 2013. Dynamic Logic with Trace Semantics. In *CADE 2013*. Springer Berlin Heidelberg, 315–329.

[3] Bernhard Beckert, Vladimir Klebanov, and Benjamin Weiß. 2016. *Dynamic Logic for Java*. Springer International Publishing, 49–106.

[4] Bernhard Beckert and Steffen Schlager. 2001. A Sequent Calculus for First-Order Dynamic Logic with Trace Modalities. In *Automated Reasoning*. Springer Berlin Heidelberg, Berlin, Heidelberg, 626–641.

[5] Mario Benevides, Leandro Gomes, and Bruno Lopes. 2025. Towards determinism in PDL: relations and proof theory. *Journal of Logic and Computation* 35, 5 (07 2025), exae022. https://doi.org/10.1093/logcom/exae022 arXiv:https://academic.oup.com/logcom/article-pdf/35/5/exae022/57794320/exae022.pdf

[6] Mario R. F. Benevides and L. Menasché Schechter. 2008. A Propositional Dynamic Logic for CCS Programs. In *Logic, Language, Information and Computation*. Springer Berlin Heidelberg, Berlin, Heidelberg, 83–97.

[7] Yves Bertot and Pierre Castéran. 2004. *Interactive Theorem Proving and Program Development - Coq'Art: The Calculus of Inductive Constructions*. Springer. 1–472 pages.

[8] James Brotherston, Richard Bornat, and Cristiano Calcagno. 2008. Cyclic proofs of program termination in separation logic. *SIGPLAN Not.* 43, 1 (2008), 101–112.

[9] James Brotherston and Alex Simpson. 2007. Complete Sequent Calculi for Induction and Infinite Descent. In *LICS 2007*. 51–62.

[10] Edmund M. Clarke and E. Allen Emerson. 1981. Design and Synthesis of Synchronization Skeletons Using Branching-Time Temporal Logic. In *Logics of Programs (Lecture Notes in Computer Science, Vol. 131)*, Dexter Kozen (Ed.). Springer, 52–71. https://doi.org/10.1007/BFb0025774

[11] Canh Minh Do, Tsubasa Takagi, and Kazuhiro Ogata. 2024. Automated Quantum Protocol Verification Based on Concurrent Dynamic Quantum Logic. *ACM Trans. Softw. Eng. Methodol.* (Dec. 2024). Just Accepted.

[12] Simon Docherty and Reuben N. S. Rowe. 2019. A Non-wellfounded, Labelled Proof System for Propositional Dynamic Logic. In *TABLEAUX 2019*. Springer International Publishing, 335–352.

[13] Emanuele D'Osualdo, Julian Sutherland, Azadeh Farzan, and Philippa Gardner. 2021. TaDA Live: Compositional Reasoning for Termination of Fine-Grained Concurrent Programs. *ACM Transactions on Programming Languages and Systems* 43, 4 (2021), 16:1–16:134. https://doi.org/10.1145/3477082

[14] Daniel Drodt and Reiner Hähnle. 2026. RustyDL: A Program Logic for Rust. In *Formal Methods*, Augusto Sampaio and Marielle Stoelinga (Eds.). Springer Nature Switzerland, Cham, 357–377.

[15] David Harel. 1979. *First-Order Dynamic Logic*. Lecture Notes in Computer Science (LNCS), Vol. 68. Springer.

[16] David Harel, Dexter Kozen, and Rohit Parikh. 1982. Process Logic: Expressiveness, Decidability, Completeness. *J. Comput. System Sci.* 25, 2 (1982), 144–170.

[17] David Harel, Dexter Kozen, and Jerzy Tiuryn. 2000. *Dynamic Logic*. MIT Press.

[18] Eleftherios Ioannidis, Yannick Zakowski, Steve Zdancewic, and Sebastian Angel. 2025. Structural Temporal Logic for Mechanized Program Verification. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1148–1175. arXiv:2410.14906 [cs.PL] https://arxiv.org/abs/2410.14906

[19] Jean-Baptiste Jeannin and André Platzer. 2014. dTL2: Differential Temporal Dynamic Logic with Nested Temporalities for Hybrid Systems. In *Automated Reasoning*. Springer International Publishing, Cham, 292–306.

[20] Hongjin Liang and Xinyu Feng. 2016. A Program Logic for Concurrent Objects under Fair Scheduling. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '16)*. Association for Computing Machinery, 385–399. https://doi.org/10.1145/2837614.2837635

[21] Egor Namakonov, Justus Fasse, Bart Jacobs, Lars Birkedal, and Amin Timany. 2026. Lawyer: Modular Obligations-Based Liveness Reasoning in Higher-Order Impredicative Concurrent Separation Logic. *Proceedings of the ACM on Programming Languages* 10, OOPSLA1, Article 132 (2026), 26 pages. https://doi.org/10.1145/3798240

[22] Sara Negri. 2005. Proof Analysis in Modal Logic. *Journal of Philosophical Logic* 34, 5–6 (2005), 507–544. https://doi.org/10.1007/s10992-005-2267-3

[23] André Platzer. 2007. Differential Dynamic Logic for Verifying Parametric Hybrid Systems.. In *TABLEAUX 2007* (2007-09-20). Springer Berlin Heidelberg, 216–232.

[24] André Platzer. 2007. A Temporal Dynamic Logic for Verifying Hybrid System Invariants. In *Logical Foundations of Computer Science*. Springer Berlin Heidelberg, Berlin, Heidelberg, 457–471.

[25] André Platzer. 2015. Differential Game Logic. *ACM Trans. Comput. Log.* 17, 1 (2015), 1:1–1:52. https://doi.org/10.1145/2817824

[26] André Platzer. 2018. *Logical Foundations of Cyber-Physical Systems*. Springer.

[27] André Platzer and Jan-David Quesel. 2008. KeYmaera: A Hybrid Theorem Prover for Hybrid Systems (System Description). In *Automated Reasoning*. Springer Berlin Heidelberg, Berlin, Heidelberg, 171–178.

[28] Amir Pnueli. 1977. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (SFCS 1977)*. 46–57. https://doi.org/10.1109/SFCS.1977.32

[29] Wolfgang Reif. 1995. *The Kiv-approach to software verification*. Springer Berlin Heidelberg, Berlin, Heidelberg, 339–368.

[30] K. Rustan and M. Leino. 2007. *Verification of Object-Oriented Software. The KeY Approach*. Lecture Notes in Computer Science (LNCS), Vol. 4334. Springer.

[31] Colin Stirling and David Walker. 1991. Local model checking in the modal mu-calculus. *Theor. Comput. Sci.* 89, 1 (1991), 161–177.

[32] Gadi Tellez and James Brotherston. 2020. Automatically Verifying Temporal Properties of Pointer Programs with Cyclic Proof. *Journal of Automated Reasoning* 64, 3 (2020), 555–578.

[33] Amin Timany, Simon Oddershede Gregersen, Léo Stefanesco, Jonas Kastberg Hinrichsen, Léon Gondelman, Abel Nieto, and Lars Birkedal. 2024. Trillium: Higher-Order Concurrent and Distributed Separation Logic for Intensional Refinement. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 241–272. https://doi.org/10.1145/3632851

[34] Yuanrui Zhang. 2025. Parameterized Dynamic Logic – Towards A Cyclic Logical Framework for General Program Specification and Verification. arXiv:2404.18098 [cs.LO] https://arxiv.org/abs/2404.18098

[35] Yuanrui Zhang and Zhiming Liu. 2024. A dynamic logic with branching modalities. *Journal of Logical and Algebraic Methods in Programming* 136 (2024), 100921. https://doi.org/10.1016/j.jlamp.2023.100921

[36] Yuanrui Zhang, Frédéric Mallet, and Zhiming Liu. 2022. A dynamic logic for verification of synchronous models based on theorem proving. *Front. Comput. Sci.* 16, 4 (2022), 3 pages.

[37] Yuanrui Zhang, Hengyang Wu, Yixiang Chen, and Frédéric Mallet. 2021. A clock-based dynamic logic for the verification of CCSL specifications in synchronous systems. *Science of Computer Programming* 203 (2021), 102591.

[38] Yuanrui Zhang and Zhibin Yang. 2026. A Generic Dynamic Logic for Program Reasoning Based on Operational Semantics. In *Dependable Software Engineering. Theories, Tools, and Applications*, Amir Goharshady and Christoph Haase (Eds.). Springer Nature Singapore, Singapore, 93–112.

A Other Proofs of the Main Theorems

A.1 Soundness Proofs for G3PPL

Proof of Lemma 5.1. We consider only the cases in which the derivation step is an instance of one of the rules $([a]R)$, $([\cup])$, (sub), and $(\mathbf{suf}\ R)$. The cases for instances of the rules $([\phi?]R)$, $([*])$, and $(\mathbf{fus}\ R)$ are similar to those for $([a]R)$, $([\cup])$, and $(\mathbf{suf}\ R)$, respectively. The other cases are trivial.

Case for rule $([a]R)$**:** If the rule $([a]R)$ is applied to ν, let $\nu = (\Gamma \Rightarrow \tau, \Delta)$ with $\tau = \sigma \diamond x : [a]\phi$, then ν' is the only premise $\Gamma, xR_ay \Rightarrow \sigma \diamond x \diamond y : \phi, \Delta$ of ν. By Definition 4.6, $\tau' = \sigma \diamond x \diamond y : \phi$. By the invalidity of ν, we get that $\mathfrak{m} \not\models \tau$ and ν' is invalid. From $\mathfrak{m} \not\models \tau$, we have that $CES(\mathfrak{m}(\sigma \diamond x), [a]\phi) \neq \emptyset$. Because ν' is invalid, we know that $\mathfrak{m} \not\models \tau'$ and $\mathfrak{m} \models xR_ay$. By Definition 5.3, for any sequence $(tr_1)(tr_2)\dots(tr_n) \in CES(\mathfrak{m}(\sigma \diamond x), [a]\phi)$ with $tr_1 = \mathfrak{m}(\sigma \diamond x)$, $(tr_2)\dots(tr_n) \in CES(\mathfrak{m}(\sigma \diamond x \diamond y), \phi)$ with $tr_2 = \mathfrak{m}(\sigma \diamond x \diamond y)$ and $(tr_1)(tr_2) \in ex(tr_1, a)$ (since $\mathfrak{m} \models xR_ay$). Moreover, since $CES(\mathfrak{m}(\sigma \diamond x), [a]\phi) \neq \emptyset$, we thus have $\min(CES(\mathfrak{m}(\sigma \diamond x), [a]\phi)) > \min(CES(\mathfrak{m}(\sigma \diamond x \diamond y, \phi))$.

Case for rule $([\cup])$**:** If the rule $([\cup])$ is applied to ν, let $\tau = \sigma : [\alpha \cup \beta]\phi$. By Definition 5.2 and 5.3, $CES(\mathfrak{m}(\sigma), [\alpha \cup \beta]\phi) = CES(\mathfrak{m}(\sigma), [\alpha]\phi) \cup CES(\mathfrak{m}(\sigma), [\beta]\phi)$. By the invalidity of ν, $\mathfrak{m} \not\models \tau$, so $CES(\mathfrak{m}(\sigma), [\alpha \cup \beta]\phi) \neq \emptyset$. Assume that the shortest sequence in $CES(\mathfrak{m}(\sigma), [\alpha \cup \beta]\phi)$ belongs to $CES(\mathfrak{m}(\sigma), [\alpha]\phi)$, and choose $\tau' = \sigma : [\alpha]\phi$ as the replacement of the target formula τ; let ν' be the premise containing τ'. Clearly, $\min(CES(\mathfrak{m}(\sigma), [\alpha \cup \beta]\phi)) = \min(CES(\mathfrak{m}(\sigma), [\alpha]\phi))$. Since $CES(\mathfrak{m}(\sigma), [\alpha \cup \beta]\phi) \neq \emptyset$, we also have $CES(\mathfrak{m}(\sigma), [\alpha]\phi) \neq \emptyset$; therefore, $\mathfrak{m} \not\models \sigma : [\alpha]\phi$, which means that ν' is invalid.

Case for rule $(\boldsymbol{sub})$**:** If the rule (sub) is applied to ν, then ν has the unique premise ν'. Thus the invalidity of ν implies that of ν'. Let $\tau = \sigma[\sigma'/U] : [\alpha]\phi$, where σ' is a label and $U \in LVar$ is a label variable. Choose $\tau' = \sigma : [\alpha]\phi$. (τ, τ') is then a trace pair over (ν, ν') by Definition 4.6. Let $\mathfrak{m}'$ be the label mapping such that $\mathfrak{m}'(U) =_{df} \mathfrak{m}(\sigma')$; $\mathfrak{m}'(V) =_{df} \mathfrak{m}(V)$ for any other $V \in LVar$. By the definition of substitution on labels (see Section 4.1), we have $\mathfrak{m}'(\sigma) = \mathfrak{m}(\sigma[\sigma'/U])$. Hence $CES(\mathfrak{m}(\sigma[\sigma'/U]), \phi) = CES(\mathfrak{m}'(\sigma), \phi)$. So $\min(CES(\mathfrak{m}(\sigma[\sigma'/U]), \phi)) = \min(CES(\mathfrak{m}'(\sigma), \phi))$.

Case for rule $(\mathbf{suf}\ R)$**:** If the rule $(\mathbf{suf}\ R)$ is applied to the node ν, let $\tau = x \diamond \sigma : (\phi\ \mathbf{suf}\ \psi)$ be the target formula of ν. By the invalidity of ν, $\mathfrak{m} \not\models \tau$, so $CES(\mathfrak{m}(x \diamond \sigma), \phi\ \mathbf{suf}\ \psi) = \{(\mathfrak{m}(x)\mathfrak{m}(\sigma))\}$. According to the rule $(\mathbf{suf}\ R)$, since $\mathfrak{m} \not\models \tau$, if $\mathfrak{m} \models \sigma : \phi\ \mathbf{suf}\ \psi$, then $\mathfrak{m}$ must not satisfy $\sigma : \phi$. In this case, by Definition 4.6, $(\tau, \sigma : \phi)$ is not a trace pair over the derivation (violating that τ is a trace element). If $\mathfrak{m} \not\models \sigma : \phi\ \mathbf{suf}\ \psi$, let $\tau' = \sigma : \phi\ \mathbf{suf}\ \psi$, and let ν' be the premise containing τ'. (τ, τ') is then a trace pair over (ν, ν'). By Definition 5.3, clearly $CES(\mathfrak{m}(\sigma), \phi\ \mathbf{suf}\ \psi) = \{(\mathfrak{m}(\sigma))\}$. Thus, $\min(CES(\mathfrak{m}(x \diamond \sigma), \phi\ \mathbf{suf}\ \psi)) = |(\mathfrak{m}(x)\mathfrak{m}(\sigma))| > |(\mathfrak{m}(\sigma))| = \min(CES(\mathfrak{m}(\sigma), \phi\ \mathbf{suf}\ \psi))$, according to the definition of the length of an execution sequence in Definition 5.2. Moreover, from $\mathfrak{m} \not\models \sigma : \phi\ \mathbf{suf}\ \psi$, we immediately obtain the invalidity of ν'.

□

A.2 Completeness Proofs of G3PPL

Lemma 6.2 is subsequently used in the proof of Lemma A.2 in Section A.3.

Proof of Lemma 6.2. We only prove $X : \phi \Rightarrow X : [true?]\phi$. The other direction is similar.

Without loss of generality, let $X = Y \diamond x$. By $([\phi?]R)$ and (wk), we need

$$(A2)\quad Y \diamond x : \phi \Rightarrow Y \diamond x \diamond x : \phi$$

We prove $(A2)$ by induction on the structure of formula ϕ.

The base step is when ϕ is an atomic formula. Then $(A2)$ can be trivially derived by applying (p) and (ax).

For the inductive step, consider $\phi = \mathbf{f}\,\psi$ and $\phi = \phi_1 \,\mathbf{suf}\, \phi_2$. The case for $\phi = \mathbf{f}\,\psi$ is similar to the base step, and we omit it here.

If $\phi = \phi_1 \,\mathbf{suf}\, \phi_2$, let $Y = y \diamond Z$. Then from (A2), by applying ($\mathbf{suf}\, R$) and ($\mathbf{suf}\, L$), we need to prove:

(A2.1) $Z \diamond x : \phi_2 \Rightarrow Z \diamond x \diamond x : \phi_2, Z \diamond x \diamond x : \phi_1$.
(A2.2) $Z \diamond x : \phi_2 \Rightarrow Z \diamond x \diamond x : \phi_2, Z \diamond x \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2$.
(A2.3) $Z \diamond x : \phi_1, Z \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2 \Rightarrow Z \diamond x \diamond x : \phi_2, Z \diamond x \diamond x : \phi_1$
(A2.4) $Z \diamond x : \phi_1, Z \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2 \Rightarrow Z \diamond x \diamond x : \phi_2, Z \diamond x \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2$

For ($A2.1$) and ($A2.2$), by (wk) on the right, we obtain

$$(A2.21)\quad Z \diamond x : \phi_2 \Rightarrow Z \diamond x \diamond x : \phi_2.$$

By first replacing Z with Y (using (sub)) and then applying the induction hypothesis to ϕ_2, (A2.21) is provable.

($A2.3$) is similar. By (wk) we obtain

$$(A2.31)\quad Z \diamond x : \phi_1 \Rightarrow Z \diamond x \diamond x : \phi_1,$$

which can be proved by first replacing Z with Y and then applying the induction hypothesis to ϕ_1.

For ($A2.4$), after (wk) on both sides, we obtain

$$(A2.41)\quad Z \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2 \Rightarrow Z \diamond x \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2.$$

(A2.41) is exactly (A2) after variable replacement. Moreover, from (A2) to (A2.41) the derivation path is progressing since from A2 to A2.4, rule ($\mathbf{suf}\, R$) is applied with ($Y \diamond x \diamond x : \phi, Z \diamond x \diamond x : \phi_1 \,\mathbf{suf}\, \phi_2$) the progressing step.

□

Below, we show that each rule specific to process logic in PPL (Table 2) is provable in G3PPL. We first describe each rule as a proposition and then prove it (Propositions A.1–A.15).

Proposition A.1 (Rule (i)). *Rule (i):* $\mathbf{f}(\phi \vee \psi) \leftrightarrow \mathbf{f}\,\phi \vee \mathbf{f}\,\psi$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We only prove

$$(i.a)\quad X : \mathbf{f}(\phi \vee \psi) \Rightarrow X : \mathbf{f}\,\phi \vee \mathbf{f}\,\psi.$$

The other direction is similar.

Without loss of generality, let $X = x \diamond Y$. Then by ($\mathbf{f}$), we need

$$(i.a1)\quad x : \phi \vee \psi \Rightarrow x : \phi, x : \psi,$$

which is proved trivially by ($\vee L$) and (ax). □

Proposition A.2 (Rule (ii)). *Rule (ii):* $\mathbf{f}\,\neg\phi \leftrightarrow \neg\,\mathbf{f}\,\phi$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We only prove

$$(ii.a)\quad X : \mathbf{f}\,\neg\phi \Rightarrow X : \neg\,\mathbf{f}\,\phi.$$

The other direction is similar.

Without loss of generality, let $X = x \diamond Y$. By ($\mathbf{f}$) and ($\neg R$) we obtain

$$x : \neg\phi, x : \phi \Rightarrow \cdot,$$

which is trivially proved by (ax). □

Proposition A.3 (Rule (iii)). *Rule (iii):* $(\phi \,\mathbf{suf}\, \psi) \vee (\phi \,\mathbf{suf}\, \chi) \leftrightarrow \phi \,\mathbf{suf}(\psi \vee \chi)$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We need to prove

(iii.a) $X : (\phi \,\mathbf{suf}\, \psi) \vee (\phi \,\mathbf{suf}\, \chi) \Rightarrow X : \phi \,\mathbf{suf}(\psi \vee \chi)$.
(iii.b) $X : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow X : (\phi \,\mathbf{suf}\, \psi) \vee (\phi \,\mathbf{suf}\, \chi)$.

Part (iii.a) is proved in Section 6; here, we prove only (iii.b). That is, it remains to prove

$$(iii.b1)\quad X : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow X : (\phi \,\mathbf{suf}\, \psi), X : (\phi \,\mathbf{suf}\, \chi)$$

by $(\vee R)$. Without loss of generality, let $X = x \diamond Y$. By applying $(\mathbf{suf}\, L)$ on the left, we split (iii.b1) into

$(iii.b11)$ $Y : \psi \vee \chi \Rightarrow x \diamond Y : (\phi \,\mathbf{suf}\, \psi), x \diamond Y : (\phi \,\mathbf{suf}\, \chi)$.
$(iii.b12)$ $Y : \phi, Y : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow x \diamond Y : (\phi \,\mathbf{suf}\, \psi), x \diamond Y : (\phi \,\mathbf{suf}\, \chi)$.

For (iii.b11), by applying $(\mathbf{suf}\, R)$ on both $x \diamond Y : (\phi \,\mathbf{suf}\, \psi)$ and $x \diamond Y : (\phi \,\mathbf{suf}\, \chi)$, we obtain

(iii.b111) $Y : \psi \vee \chi \Rightarrow Y : \psi, Y : \phi, Y : \chi, Y : \phi$.
(iii.b112) $Y : \psi \vee \chi \Rightarrow Y : \psi, Y : \phi, Y : \chi, Y : \phi \,\mathbf{suf}\, \chi$.
(iii.b113) $Y : \psi \vee \chi \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi, Y : \chi, Y : \phi$.
(iii.b114) $Y : \psi \vee \chi \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi, Y : \chi, Y : \phi \,\mathbf{suf}\, \chi$.

Since the right side of each of (iii.b111)–(iii.b114) contains both $Y : \psi$ and $Y : \chi$, these sequents can be proved by (ax) for the different cases on the left side.

For (iii.b12), by applying $(\mathbf{suf}\, R)$ on both $x \diamond Y : (\phi \,\mathbf{suf}\, \psi)$ and $x \diamond Y : (\phi \,\mathbf{suf}\, \chi)$, we obtain

(iii.b121) $Y : \phi, Y : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow Y : \psi, Y : \phi, Y : \chi, Y : \phi$.
(iii.b122) $Y : \phi, Y : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow Y : \psi, Y : \phi, Y : \chi, Y : \phi \,\mathbf{suf}\, \chi$.
(iii.b123) $Y : \phi, Y : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi, Y : \chi, Y : \phi$.
(iii.b124) $Y : \phi, Y : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi, Y : \chi, Y : \phi \,\mathbf{suf}\, \chi$.

Each of (iii.b121)–(iii.b123) can be proved directly by applying (ax) because both sides contain $Y : \phi$.

For (iii.b124), by (wk), we have

$$(iii.b1241)\quad Y : \phi \,\mathbf{suf}(\psi \vee \chi) \Rightarrow Y : \phi \,\mathbf{suf}\, \psi, Y : \phi \,\mathbf{suf}\, \chi,$$

which is exactly (iii.b1) (replacing Y with X). The derivation from (iii.b1) to (iii.b1241) is progressing since from (iii.b12) to (iii.b124) rule $(\mathbf{suf}\, R)$ is applied with $(x \diamond Y : (\phi \,\mathbf{suf}\, \chi), Y : \phi \,\mathbf{suf}\, \chi)$ the progressing step. This makes the whole preproof a cyclic proof.

□

Proposition A.4 (Rule (iv)). *Rule (iv):* $(\phi \,\mathbf{suf}\, \psi) \wedge (\chi \,\mathbf{suf}\, \omega) \leftrightarrow (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \omega) \vee (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \vee (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega)$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We prove that

$(iv.a)$ $X : (\phi \,\mathbf{suf}\, \psi) \wedge (\chi \,\mathbf{suf}\, \omega) \Rightarrow X : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \omega) \vee (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \vee (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega)$.
$(iv.b)$ $X : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \omega) \vee (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \vee (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega) \Rightarrow X : (\phi \,\mathbf{suf}\, \psi) \wedge (\chi \,\mathbf{suf}\, \omega)$.

Case for proving $(iv.a)$: Let

$$A =_{df} (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \omega),$$
$$B =_{df} (\phi \wedge \chi)\ \mathbf{suf}(\phi \wedge \omega \wedge \phi\ \mathbf{suf}\ \psi),$$
$$C =_{df} (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega).$$

By applying the rules $(\wedge L)$ and $(\vee R)$ on all formulas on both sides respectively, we obtain the sequent:

$$(iv.a1)\ \ X : \phi\ \mathbf{suf}\ \psi, X : \chi\ \mathbf{suf}\ \omega \Rightarrow X : A, X : B, X : C.$$

Let $X = y \diamond Y$. By applying the rules $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides, we obtain the following 32 cases:

$(iv.a1.1)\ W_1 \Rightarrow S_1$	$(iv.a1.9)\ W_2 \Rightarrow S_1$	$(iv.a1.17)\ W_3 \Rightarrow S_1$	$(iv.a1.25)\ W_4 \Rightarrow S_1$
$(iv.a1.2)\ W_1 \Rightarrow S_2$	$(iv.a1.10)\ W_2 \Rightarrow S_2$	$(iv.a1.18)\ W_3 \Rightarrow S_2$	$(iv.a1.26)\ W_4 \Rightarrow S_2$
$(iv.a1.3)\ W_1 \Rightarrow S_3$	$(iv.a1.11)\ W_2 \Rightarrow S_3$	$(iv.a1.19)\ W_3 \Rightarrow S_3$	$(iv.a1.27)\ W_4 \Rightarrow S_3$
$(iv.a1.4)\ W_1 \Rightarrow S_4$	$(iv.a1.12)\ W_2 \Rightarrow S_4$	$(iv.a1.20)\ W_3 \Rightarrow S_4$	$(iv.a1.28)\ W_4 \Rightarrow S_4$
$(iv.a1.5)\ W_1 \Rightarrow S_5$	$(iv.a1.13)\ W_2 \Rightarrow S_5$	$(iv.a1.21)\ W_3 \Rightarrow S_5$	$(iv.a1.29)\ W_4 \Rightarrow S_5$
$(iv.a1.6)\ W_1 \Rightarrow S_6$	$(iv.a1.14)\ W_2 \Rightarrow S_6$	$(iv.a1.22)\ W_3 \Rightarrow S_6$	$(iv.a1.30)\ W_4 \Rightarrow S_6$
$(iv.a1.7)\ W_1 \Rightarrow S_7$	$(iv.a1.15)\ W_2 \Rightarrow S_7$	$(iv.a1.23)\ W_3 \Rightarrow S_7$	$(iv.a1.31)\ W_4 \Rightarrow S_7$
$(iv.a1.8)\ W_1 \Rightarrow S_8$	$(iv.a1.16)\ W_2 \Rightarrow S_8$	$(iv.a1.24)\ W_3 \Rightarrow S_8$	$(iv.a1.32)\ W_4 \Rightarrow S_8$

where

$$W_1 =_{df} Y : \psi, Y : \omega,$$
$$W_2 =_{df} Y : \phi, Y : \phi\ \mathbf{suf}\ \psi, Y : \omega,$$
$$W_3 =_{df} Y : \psi, Y : \chi, Y : \chi\ \mathbf{suf}\ \omega,$$
$$W_4 =_{df} Y : \phi, Y : \phi\ \mathbf{suf}\ \psi, Y : \chi, Y : \chi\ \mathbf{suf}\ \omega.$$

$S_1, \ldots, S_8$ are defined as follows:

$$S_1 =_{df} A', A_1, B', B_1, C', C_1,$$
$$S_2 =_{df} A', A_1, B', B_1, C', C_2,$$
$$S_3 =_{df} A', A_1, B', B_2, C', C_1,$$
$$S_4 =_{df} A', A_1, B', B_2, C', C_2,$$
$$S_5 =_{df} A', A_2, B', B_1, C', C_1,$$
$$S_6 =_{df} A', A_2, B', B_1, C', C_2,$$
$$S_7 =_{df} A', A_2, B', B_2, C', C_1,$$
$$S_8 =_{df} A', A_2, B', B_2, C', C_2,$$

where X' and X_i are obtained from X ($i \in \{1, 2\}$, $X \in \{A, B, C\}$). The symbols are defined as follows:

$A' =_{df} Y : \psi \wedge \omega$	$B' =_{df} Y : \phi \wedge \omega \wedge \phi\ \mathbf{suf}\ \psi$	$C' =_{df} Y : \psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega$
$A_1 =_{df} Y : \phi \wedge \chi$	$B_1 =_{df} Y : \phi \wedge \chi$	$C_1 =_{df} Y : \phi \wedge \chi$
$A_2 =_{df} Y : (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \omega)$	$B_2 =_{df} Y : (\phi \wedge \chi)\ \mathbf{suf}(\phi \wedge \omega \wedge \phi\ \mathbf{suf}\ \psi)$	$C_2 =_{df} Y : (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega).$

The proofs of the cases from (iv.a1.1) to (iv.a1.24) are trivial and are all terminal branches (without buds), since in each of these cases we can find a formula on the left side that directly matches a subformula of a right-side disjunct

among A', B', C', allowing the branch to be closed by $(\wedge R)$ and (ax) without any application of the rules $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$. For example, for

$$(iv.a1.16)\ \ Y:\phi, Y:\phi\ \mathbf{suf}\ \psi, Y:\omega \Rightarrow A', A_2, B', B_2, C', C_2,$$

by $(\wedge R)$ on B', we can obtain:

(iv.a1.16.1) $Y:\phi, Y:\phi\ \mathbf{suf}\ \psi, Y:\omega \Rightarrow A', A_2, Y:\phi, C', C_2.$
(iv.a1.16.2) $Y:\phi, Y:\phi\ \mathbf{suf}\ \psi, Y:\omega \Rightarrow A', A_2, Y:\omega, C', C_2.$
(iv.a1.16.3) $Y:\phi, Y:\phi\ \mathbf{suf}\ \psi, Y:\omega \Rightarrow A', A_2, Y:\phi\ \mathbf{suf}\ \psi, C', C_2.$

Each of (iv.a1.16.1) - (iv.a1.16.3) can be proved directly by (ax).

In the cases from (iv.a1.25) to (iv.a1.31), the left side is W_4, which contains $Y:\phi$ and $Y:\chi$, and the right side contains one of A_1, B_1, or C_1, each equal to $Y:\phi\wedge\chi$. Therefore, these cases can be proved by $(\wedge R)$ and (ax). For example, for (iv.a.31), by $(\wedge R)$ on C_1 on the right side, we obtain:

(iv.a.31.1) $Y:\phi, Y:\phi\ \mathbf{suf}\ \psi, Y:\chi, Y:\chi\ \mathbf{suf}\ \omega \Rightarrow A', A_2, B', B_2, C', Y:\phi,$
(iv.a.31.2) $Y:\phi, Y:\phi\ \mathbf{suf}\ \psi, Y:\chi, Y:\chi\ \mathbf{suf}\ \omega \Rightarrow A', A_2, B', B_2, C', Y:\chi.$

Both cases can be proved directly by (ax).

Lastly, for the case (iv.a1.32), since it has A_2, B_2 and C_2, by applying (wk) we obtain:

$$(iv.a1.32.1)\ Y:\phi\ \mathbf{suf}\ \psi, Y:\chi\ \mathbf{suf}\ \omega \Rightarrow A_2, B_2, C_2.$$

After replacing Y with X (i.e., by applying (sub)), (iv.a1.32.1) is exactly (iv.a1). Note that along the derivation path from (iv.a1) to (iv.a1.32.1), rule $(\mathbf{suf}\ R)$ is applied, which guarantees the whole preproof is a cyclic proof.

Case for proving $(iv.b)$: By applying the rules $(\vee L)$ and $(\wedge R)$ on the left and right sides of the sequent respectively, we split (iv.b) into 6 cases as follows:

(iv.b1) $X:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\omega) \Rightarrow X:(\phi\ \mathbf{suf}\ \psi).$
(iv.b2) $X:(\phi\wedge\chi)\ \mathbf{suf}(\phi\wedge\omega\wedge\phi\ \mathbf{suf}\ \psi) \Rightarrow X:(\phi\ \mathbf{suf}\ \psi).$
(iv.b3) $X:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\chi\wedge\chi\ \mathbf{suf}\ \omega) \Rightarrow X:(\phi\ \mathbf{suf}\ \psi).$
(iv.b4) $X:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\omega) \Rightarrow X:(\chi\ \mathbf{suf}\ \omega).$
(iv.b5) $X:(\phi\wedge\chi)\ \mathbf{suf}(\phi\wedge\omega\wedge\phi\ \mathbf{suf}\ \psi) \Rightarrow X:(\chi\ \mathbf{suf}\ \omega).$
(iv.b6) $X:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\chi\wedge\chi\ \mathbf{suf}\ \omega) \Rightarrow X:(\chi\ \mathbf{suf}\ \omega).$

Without loss of generality, let $X = y\diamond Y$.

Case (iv.b1): By applying the rules $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides, we can have

(iv.b11) $Y:\psi\wedge\omega \Rightarrow Y:\psi, Y:\phi.$
(iv.b12) $Y:\psi\wedge\omega \Rightarrow Y:\psi, Y:\phi\ \mathbf{suf}\ \psi.$
(iv.b13) $Y:\phi\wedge\chi, Y:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\omega) \Rightarrow Y:\psi, Y:\phi$
(iv.b14) $Y:\phi\wedge\chi, Y:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\omega) \Rightarrow Y:\psi, Y:\phi\ \mathbf{suf}\ \psi$

The cases (iv.b11)–(iv.b13) can be proved trivially by $(\wedge L)$ and (ax).

For the case (iv.b14), by (wk), we have

$$(iv.b141)\ Y:(\phi\wedge\chi)\ \mathbf{suf}(\psi\wedge\omega) \Rightarrow Y:\phi\ \mathbf{suf}\ \psi,$$

which is exactly (iv.b1), forming a back-link. The derivation path from (iv.b1) to (iv.b141) is progressing since rule $(\mathbf{suf}\ R)$ is applied from (iv.b1) to (iv.b14) and $(X:(\phi\ \mathbf{suf}\ \psi), Y:\phi\ \mathbf{suf}\ \psi)$ is the progressing step.

Case (iv.b2): By applying the rules (**suf** R) and (**suf** L) on both sides, we can have

(iv.b21) $Y : \phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi \Rightarrow Y : \psi, Y : \phi$.
(iv.b22) $Y : \phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi$.
(iv.b23) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \Rightarrow Y : \psi, Y : \phi$.
(iv.b24) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi$.

The cases for (iv.b21) - (iv.b23) can be proved by ($\wedge L$) and (ax) easily as both sides contain the same formula.

For the case (iv.b24), by (wk) we have

$$(iv.b241)\quad Y : (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \Rightarrow Y : \phi \,\mathbf{suf}\, \psi,$$

which is exactly (iv.b2), forming a back-link. The derivation path from (iv.b2) to (iv.b241) is progressing since rule (**suf** R) is applied from (iv.b2) to (iv.b24), with $(X : (\phi \,\mathbf{suf}\, \psi), Y : \phi \,\mathbf{suf}\, \psi)$ the progressing step.

Case (iv.b3): By applying the rules (**suf** R) and (**suf** L), we obtain

(iv.b31) $Y : \psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega \Rightarrow Y : \psi, Y : \phi$.
(iv.b32) $Y : \psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi$.
(iv.b33) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega) \Rightarrow Y : \psi, Y : \phi$.
(iv.b34) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega) \Rightarrow Y : \psi, Y : \phi \,\mathbf{suf}\, \psi$.

The cases (iv.b31)–(iv.b33) can be proved by ($\wedge L$) and (ax).

For the case (iv.b34), by (wk) we get

$$(iv.b341)\quad Y : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \chi \wedge \chi \,\mathbf{suf}\, \omega) \Rightarrow Y : \phi \,\mathbf{suf}\, \psi,$$

which is exactly (iv.b3), forming a back-link. The derivation path from (iv.b3) to (iv.b341) is progressing since rule (**suf** R) is applied from (iv.b3) to (iv.b34), where $(X : (\phi \,\mathbf{suf}\, \psi), Y : \phi \,\mathbf{suf}\, \psi)$ is the progressing step.

Case (iv.b4): By applying the rules (**suf** R) and (**suf** L), we have

(iv.b41) $Y : \psi \wedge \omega \Rightarrow Y : \omega, Y : \chi$.
(iv.b42) $Y : \psi \wedge \omega \Rightarrow Y : \omega, Y : \chi \,\mathbf{suf}\, \omega$.
(iv.b43) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \omega) \Rightarrow Y : \omega, Y : \chi$.
(iv.b44) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \omega) \Rightarrow Y : \omega, Y : \chi \,\mathbf{suf}\, \omega$.

The cases (iv.b41)–(iv.b43) can be proved trivially using ($\wedge L$) and (ax).

For the case (iv.b44), by (wk) we obtain

$$(iv.b441)\quad Y : (\phi \wedge \chi) \,\mathbf{suf}(\psi \wedge \omega) \Rightarrow Y : \chi \,\mathbf{suf}\, \omega,$$

which is exactly (iv.b4), forming a back-link. The derivation path from (iv.b4) to (iv.b441) is progressing since rule (**suf** R) is applied from (iv.b4) to (iv.b44), with $(X : (\chi \,\mathbf{suf}\, \omega), Y : \chi \,\mathbf{suf}\, \omega)$ the progressing step.

Case (iv.b5): By applying the rules (**suf** R) and (**suf** L), we have

(iv.b51) $Y : \phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi \Rightarrow Y : \omega, Y : \chi$.
(iv.b52) $Y : \phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi \Rightarrow Y : \omega, Y : \chi \,\mathbf{suf}\, \omega$.
(iv.b53) $Y : (\phi \wedge \chi), Y : (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \Rightarrow Y : \omega, Y : \chi$.
(iv.b54) $Y : (\phi \wedge \chi), Y : (\phi \wedge \chi) \,\mathbf{suf}(\phi \wedge \omega \wedge \phi \,\mathbf{suf}\, \psi) \Rightarrow Y : \omega, Y : \chi \,\mathbf{suf}\, \omega$.

The cases (iv.b51)–(iv.b53) can be proved trivially using ($\wedge L$) and (ax).

For the case (iv.b54), we obtain by applying (wk):

$$(iv.b541)\ \ Y : (\phi \wedge \chi)\ \mathbf{suf}(\phi \wedge \omega \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \chi\ \mathbf{suf}\ \omega,$$

which is exactly (iv.b5), forming a back-link. The derivation path from (iv.b5) to (iv.b541) is progressing because, from (iv.b5) to (iv.b54), the rule ($\mathbf{suf}\ R$) is applied, with $(X : (\chi\ \mathbf{suf}\ \omega), Y : \chi\ \mathbf{suf}\ \omega)$ as the progressing step.

Case (iv.b6): By applying the rules ($\mathbf{suf}\ R$) and ($\mathbf{suf}\ L$), we can have

(iv.b61) $Y : \psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega \Rightarrow Y : \omega, Y : \chi$.
(iv.b62) $Y : \psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega \Rightarrow Y : \omega, Y : \chi\ \mathbf{suf}\ \omega$.
(iv.b63) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega) \Rightarrow Y : \omega, Y : \chi$.
(iv.b64) $Y : \phi \wedge \chi, Y : (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega) \Rightarrow Y : \omega, Y : \chi\ \mathbf{suf}\ \omega$.

The cases (iv.b61)–(iv.b63) can be proved trivially by $(\wedge L)$ and (ax).

For the case (iv.b64), by (wk), we get

$$(iv.b641)\ \ Y : (\phi \wedge \chi)\ \mathbf{suf}(\psi \wedge \chi \wedge \chi\ \mathbf{suf}\ \omega) \Rightarrow Y : \chi\ \mathbf{suf}\ \omega.$$

This is exactly (iv.b6), forming a back-link. The derivation path from (iv.b6) to (iv.b641) is progressing, since rule ($\mathbf{suf}\ R$) is applied from (iv.b6) to (iv.b64), where $(X : (\chi\ \mathbf{suf}\ \omega), Y : \chi\ \mathbf{suf}\ \omega)$ is the progressing step.

These 6 cases show that the whole derivation from (iv.b) is a cyclic proof. Thus we conclude the proof of (iv.b).

□

Proposition A.5 (Rule (v)). *Rule (v):* $\neg(\phi\ \mathbf{suf}\ \psi) \leftrightarrow \neg(\mathit{true}\ \mathbf{suf}\ \psi) \vee (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We need to prove

$(v.a)$ $X : \neg(\phi\ \mathbf{suf}\ \psi) \Rightarrow X : \neg(\mathit{true}\ \mathbf{suf}\ \psi) \vee (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.
$(v.b)$ $X : \neg(\mathit{true}\ \mathbf{suf}\ \psi) \vee (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi) \Rightarrow X : \neg(\phi\ \mathbf{suf}\ \psi)$.

Case for proving $(v.a)$: By applying $(\neg L)$ on the left and $(\vee R)$, $(\neg R)$ on the right, we obtain the sequent:

$$(v.a1)\ \ X : (\mathit{true}\ \mathbf{suf}\ \psi) \Rightarrow X : (\phi\ \mathbf{suf}\ \psi), X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi).$$

Without loss of generality, let $X = y \diamond Y$. By applying the rules ($\mathbf{suf}\ L$) on $X : (\mathit{true}\ \mathbf{suf}\ \psi)$ and ($\mathbf{suf}\ R$) on $X : \phi\ \mathbf{suf}\ \psi$, we have:

(v.a11) $Y : \psi \Rightarrow Y : \psi, Y : \phi, X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.
(v.a12) $Y : \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi, X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.
(v.a13) $Y : \mathit{true}, Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi, X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.
(v.a14) $Y : \mathit{true}, Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi, X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.

(v.a11) and (v.a12) can be directly proved by (ax).

For (v.a13) and (v.a14), by ($\mathbf{suf}\ R$) on $X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$, we further need the following sequents:

v.a131 $Y : \mathit{true}, Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi, Y : (\neg\phi \wedge \neg\psi), Y : (\neg\psi)$.
v.a132 $Y : \mathit{true}, Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi, Y : (\neg\phi \wedge \neg\psi), Y : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.
v.a141 $Y : \mathit{true}, Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi, Y : (\neg\phi \wedge \neg\psi), Y : (\neg\psi)$.
v.a142 $Y : \mathit{true}, Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi, Y : (\neg\phi \wedge \neg\psi), Y : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi)$.

(v.a131) and (v.a141) can be trivially closed by (ax) since they both have $Y : \psi$ and $Y : (\neg\psi)$ on their right sides. Sequent (v.a132) can be closed because it contains $Y : \psi$, $Y : \phi$, and $Y : (\neg\phi \wedge \neg\psi)$ on the right at the same time.

For (v.a142), by (wk), we need:

$$(v.a1421)\ Y : \mathit{true}\ \mathbf{suf}\ \psi \Rightarrow Y : \phi\ \mathbf{suf}\ \psi, Y : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi).$$

It has the same form as (v.a1), thus forming a back-link. The derivation path from (v.a1) to (v.a1421) is progressing since rule ($\mathbf{suf}\ R$) is applied.

Case for proving $(v.b)$: By applying $(\vee L)$ on the left, we split it into 2 cases:

(v.b1) $X : \neg(\mathit{true}\ \mathbf{suf}\ \psi) \Rightarrow X : \neg(\phi\ \mathbf{suf}\ \psi)$.
(v.b2) $X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi) \Rightarrow X : \neg(\phi\ \mathbf{suf}\ \psi)$.

Case (v.b1): By $(\neg L)$ and $(\neg R)$:

$$(v.b11)\ \ X : \phi\ \mathbf{suf}\ \psi \Rightarrow X : \mathit{true}\ \mathbf{suf}\ \psi.$$

Without loss of generality, let $X = y \diamond Y$. By applying ($\mathbf{suf}\ R$) and ($\mathbf{suf}\ L$) on both sides, we have:

(v.b111) $Y : \psi \Rightarrow Y : \psi, Y : \mathit{true}$.
(v.b112) $Y : \psi \Rightarrow Y : \psi, Y : \mathit{true}\ \mathbf{suf}\ \psi$.
(v.b113) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{true}$.
(v.b114) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{true}\ \mathbf{suf}\ \psi$.

(v.b111) and (v.b112) can be trivially proved by (ax). (v.b113) can be proved since $Y : \mathit{true}$ is on the right side.

For (v.b114), by (wk) we need:

$$(v.b1141)\ \ Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \mathit{true}\ \mathbf{suf}\ \psi,$$

which is exactly (v.b11), forming a back-link. The derivation path is progressing since rule ($\mathbf{suf}\ R$) is applied from (v.b11) to (v.b114), with $(X : \mathit{true}\ \mathbf{suf}\ \psi, Y : \mathit{true}\ \mathbf{suf}\ \psi)$ the progressing step.

Case (v.b2): By the introduction of the operator **fus**, it is equivalent to prove:

$$(v.b21)\ \ X : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi) \Rightarrow X : \neg\phi\ \mathbf{fus}\ \neg\psi.$$

By applying ($\mathbf{suf}\ L$) on the left and ($\mathbf{fus}\ R$) on the right, we have

(v.b211) $Y : \neg\phi \wedge \neg\psi \Rightarrow Y : \neg\psi$.
(v.b212) $Y : \neg\phi \wedge \neg\psi \Rightarrow Y : \neg\phi, Y : \neg\phi\ \mathbf{fus}\ \neg\psi$.
(v.b213) $Y : \neg\psi, Y : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi) \Rightarrow Y : \neg\psi$.
(v.b214) $Y : \neg\psi, Y : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi) \Rightarrow Y : \neg\phi, Y : \neg\phi\ \mathbf{fus}\ \neg\psi$.

(v.b211), (v.b212) and (v.b213) are trivially proved by $(\wedge L)$ and (ax).

For (v.b214), by (wk), we obtain

$$(v.b2141)\ \ Y : (\neg\psi)\ \mathbf{suf}(\neg\phi \wedge \neg\psi), Y : \phi \Rightarrow Y : \neg\phi\ \mathbf{fus}\ \neg\psi,$$

which is exactly (v.b21), forming a back-link. The derivation path is progressing since rule ($\mathbf{fus}\ R$) is applied, with $(X : \neg\phi\ \mathbf{fus}\ \neg\psi, Y : \neg\phi\ \mathbf{fus}\ \neg\psi)$ the progressing step.

□

Proposition A.6 (Rule (vi)). *Rule (vi):* $\phi\ \mathbf{suf}\ \psi \leftrightarrow \mathbf{n}\,\psi \vee \mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. Recall that $\mathbf{n}\,\theta =_{df} \mathit{false}\ \mathbf{suf}\ \theta$. We need to prove

$$(vi.a)\ \ X : \phi\ \mathbf{suf}\ \psi \Rightarrow X : \mathbf{n}\,\psi \vee \mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)).$$

$(vi.b)$ $X : \mathbf{n}\,\psi \vee \mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \Rightarrow X : \phi\ \mathbf{suf}\ \psi$.

Case for proving $(vi.a)$: By applying $(\vee R)$, we need:

$$(vi.a1)\ \ X : \phi\ \mathbf{suf}\ \psi \Rightarrow X : \mathit{false}\ \mathbf{suf}\ \psi, X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$$

Without loss of generality, let $X = y \diamond Y$. By applying $(\mathbf{suf}\ L)$ and $(\mathbf{suf}\ R)$ on $X : \mathit{false}\ \mathbf{suf}\ \psi$, we have

(vi.a11) $Y : \psi \Rightarrow Y : \psi, Y : \mathit{false}, X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.
(vi.a12) $Y : \psi \Rightarrow Y : \psi, Y : \mathit{false}\ \mathbf{suf}\ \psi, X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.
(vi.a13) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{false}, X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.
(vi.a14) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{false}\ \mathbf{suf}\ \psi, X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.

(vi.a11) and (vi.a12) can be proved directly by (ax).

For (vi.a13), by $(\mathbf{suf}\ R)$ on $X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$, we need:

(vi.a131) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{false}, Y : (\phi \wedge \phi\ \mathbf{suf}\ \psi), Y : \mathit{false}$.
(vi.a132) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{false}, Y : (\phi \wedge \phi\ \mathbf{suf}\ \psi), Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi)$.

Both (vi.a131) and (vi.a132) can be proved by $(\wedge R)$ and (ax) easily since they contain $Y : \phi$, $Y : \phi\ \mathbf{suf}\ \psi$ on the left side and $Y : (\phi \wedge \phi\ \mathbf{suf}\ \psi)$ on the right side.

For (vi.a14), by $(\mathbf{suf}\ R)$ on $X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$, we need:

(vi.a141) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{false}\ \mathbf{suf}\ \psi, Y : (\phi \wedge \phi\ \mathbf{suf}\ \psi), Y : \mathit{false}$.
(vi.a142) $Y : \phi, Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \mathit{false}\ \mathbf{suf}\ \psi, Y : (\phi \wedge \phi\ \mathbf{suf}\ \psi), Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi)$.

Similar to the above, both (vi.a141) and (vi.a142) can be proved by $(\wedge R)$ and (ax) easily since they contain $Y : \phi$, $Y : \phi\ \mathbf{suf}\ \psi$ on the left side and $Y : (\phi \wedge \phi\ \mathbf{suf}\ \psi)$ on the right side.

Case for proving $(vi.b)$: By $(\vee L)$, we split into 2 cases:

(vi.b1) $X : \mathit{false}\ \mathbf{suf}\ \psi \Rightarrow X : \phi\ \mathbf{suf}\ \psi$.
(vi.b2) $X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \Rightarrow X : \phi\ \mathbf{suf}\ \psi$.

Without loss of generality, let $X = y \diamond Y$.

Case (vi.b1): By $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides:

(vi.b11) $Y : \psi \Rightarrow Y : \psi, Y : \phi$.
(vi.b12) $Y : \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi$.
(vi.b13) $Y : \mathit{false}, Y : \mathit{false}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi$.
(vi.b14) $Y : \mathit{false}, Y : \mathit{false}\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi$.

(vi.b11) and (vi.b12) are trivially proved by (ax). Sequents (vi.b13) and (vi.b14) can be proved easily because they both have $Y : \mathit{false}$ on the left side.

Case (vi.b2): By $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides:

(vi.b21) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi$.
(vi.b22) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi$.
(vi.b23) $Y : \mathit{false}, Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \psi, Y : \phi$.
(vi.b24) $Y : \mathit{false}, Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \psi, Y : \phi\ \mathbf{suf}\ \psi$.

(vi.b21) and (vi.b22) are trivially proved by $(\wedge L)$ and (ax). Sequents (vi.b23) and (vi.b24) can be proved easily because they both have $Y : \mathit{false}$ on the left side.

□

Proposition A.7 (Rule (vii)). *Rule (vii):* $\phi\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \leftrightarrow \mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. Recall that $\mathbf{n}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) =_{df} \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$. We need to prove

$(vii.a)$ $X : \phi\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \Rightarrow X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.

$(vii.b)$ $X : \mathit{false}\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \Rightarrow X : \phi\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.

Without loss of generality, let $X = y \diamond Y$.

Case for proving $(vii.a)$: By $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides:

(vii.a1) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \mathit{false}$.

(vii.a2) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \mathit{false}\ \mathbf{suf}\ \phi \wedge \phi\ \mathbf{suf}\ \psi$.

(vii.a3) $Y : \phi, Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \mathit{false}$.

(vii.a4) $Y : \phi, Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \mathit{false}\ \mathbf{suf}\ \phi \wedge \phi\ \mathbf{suf}\ \psi$.

Sequents (vii.a1) and (vii.a2) can be proved trivially by (ax) because they contain $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi$ on both sides.

For (vii.a3), since $Y : \phi$ is on the left, by $(\wedge R)$ on $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi$, the branch that has $Y : \phi$ on the right can be trivially closed by (ax). So after (wk) to eliminate $Y : \phi$ on the left and $Y : \mathit{false}$ on the right, we only need:

$$(vii.a31)\quad Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi\ \mathbf{suf}\ \psi.$$

From (vii.a31), let $Y = z \diamond Z$. By $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$, we have

(vii.a311) $Z : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \psi, Z : \phi$.

(vii.a312) $Z : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \psi, Z : \phi\ \mathbf{suf}\ \psi$.

(vii.a313) $Z : \phi, Z : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Z : \psi, Z : \phi$.

(vii.a314) $Z : \phi, Z : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Z : \psi, Z : \phi\ \mathbf{suf}\ \psi$.

The cases (vii.a311) to (vii.a314) can be trivially proved by $(\wedge L)$ and (ax).

For (vii.a314), by (wk):

$$(vii.a3141)\ Z : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Z : \phi\ \mathbf{suf}\ \psi,$$

which matches (vii.a31), forming a back-link. The derivation path from (vii.a31) to (vii.a3141) is progressing, as the rule $(\mathbf{suf}\ R)$ is applied from (vii.a31) to (vii.a314), where $(Y : \phi\ \mathbf{suf}\ \psi, Z : \phi\ \mathbf{suf}\ \psi)$ is the progressing step.

For (vii.a4), by (wk) we obtain:

$$(vii.a41)\quad Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \mathit{false}\ \mathbf{suf}\ \phi \wedge \phi\ \mathbf{suf}\ \psi,$$

which is exactly (vii.a), forming a back-link. The derivation path from (vii.a) to (vii.a41) is progressing, as the rule $(\mathbf{suf}\ R)$ is applied from (vii.a) to (vii.a4), where $(X : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi), Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi))$ is the progressing step.

Case for proving $(vii.b)$: By $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides:

(vii.b1) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \phi$.

(vii.b2) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi)$.

(vii.b3) $Y : \mathit{false}, Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \phi$.

(vii.b4) $Y : \mathit{false}, Y : \mathit{false}\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi \wedge \phi\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi)$.

The cases for (vii.b1) and (vii.b2) can be trivially closed by (ax). The cases (vii.b3) and (vii.b4) can be proved easily because they contain $Y : \mathit{false}$ on the right side.

□

Proposition A.8 (Rule (viii)). *Rule (viii):* $\phi\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \leftrightarrow \phi\ \mathbf{suf}(\phi \wedge \mathbf{n}\,\psi)$ *is derived in G3PPL.*

Proof. Let $X \in TVar$. Recall that $\mathbf{n}\,\psi =_{df} \mathit{false}\ \mathbf{suf}\ \psi$. We need to prove

$(viii.a)$ $X : \phi\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi)) \Rightarrow X : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

$(viii.b)$ $X : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi) \Rightarrow X : \phi\ \mathbf{suf}(\phi \wedge (\phi\ \mathbf{suf}\ \psi))$.

Without loss of generality, let $X = x \diamond Y$.

Case for proving $(viii.a)$: By applying the rules $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on both sides:

(viii.a1) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi$.

(viii.a2) $Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

(viii.a3) $Y : \phi, Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi$.

(viii.a4) $Y : \phi, Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

The cases (viii.a1) and (viii.a3) can be trivially proved by (ax) since they contain $Y : \phi$ on both sides.

For (viii.a2), by $(\wedge R)$ on $Y : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi$, the branch that contains $Y : \phi$ on the right side can be easily closed since there is a $Y : \phi$ on the left side. Thus, the only branch that needs to be proved is:

$$Y : \phi \wedge \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi).$$

By $(\wedge L)$ and (wk), we need:

$$(viii.a21)\ \ Y : \phi\ \mathbf{suf}\ \psi \Rightarrow Y : \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi).$$

Without loss of generality, let $Y = z \diamond Z$. Then, by $(\mathbf{suf}\ R)$ and $(\mathbf{suf}\ L)$ on $Y : \mathit{false}\ \mathbf{suf}\ \psi$ and $Y : \phi\ \mathbf{suf}\ \psi$ respectively, we obtain

viii.a211 $Z : \psi \Rightarrow Z : \psi, Z : \mathit{false}, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

viii.a212 $Z : \psi \Rightarrow Z : \psi, Z : \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

viii.a213 $Z : \phi, Z : \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \psi, Z : \mathit{false}, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

viii.a214 $Z : \phi, Z : \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \psi, Z : \mathit{false}\ \mathbf{suf}\ \psi, Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$.

Where (viii.a211) and (viii.a212) can be closed directly by (ax).

For (viii.a213), by applying the rule $(\mathbf{suf}\ R)$ on the right, we need

viii.a2131 $Z : \phi, Z : \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \psi, Z : \mathit{false}, Z : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi, Z : \phi$, which is closed by (ax).

viii.a2132 $Z : \phi, Z : \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \psi, Z : \mathit{false}, Z : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi, Z : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi)$, from which by (wk) and $(\wedge R)$ on $Z : \phi \wedge \mathit{false}\ \mathbf{suf}\ \psi$, we still need to prove:

$$(viii.a21321)\ Z : \phi\ \mathbf{suf}\ \psi \Rightarrow Z : \mathit{false}\ \mathbf{suf}\ \psi, Z : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi),$$

which is exactly (viii.a21), forming a back-link. The derivation path from (viii.a21) to (viii.a21321) is progressing as the rule $(\mathbf{suf}\ R)$ is applied with suitable progressing step.

(viii.a214) is similar to the case (viii.a213). The only exception is that on the right side $Z : \mathit{false}\ \mathbf{suf}\ \psi$ is weakened instead of $Z : \mathit{false}$. We omit its proof here.

For (viii.a4), by (wk):

$$(viii.a41)\ Y : \phi\ \mathbf{suf}(\phi \wedge \phi\ \mathbf{suf}\ \psi) \Rightarrow Y : \phi\ \mathbf{suf}(\phi \wedge \mathit{false}\ \mathbf{suf}\ \psi),$$

which is exactly (viii.a), forming a back-link. The derivation path from (viii.a) to (viii.a41) is progressing as the rule (**suf** R) is applied.

Case for proving $(viii.b)$: By applying (**suf** R) and (**suf** L) on both sides, we get

(viii.b1) $Y : \phi \wedge \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Y : \phi \wedge \phi\,\mathbf{suf}\,\psi, Y : \phi$.
(viii.b2) $Y : \phi \wedge \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Y : \phi \wedge \phi\,\mathbf{suf}\,\psi, Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi)$.
(viii.b3) $Y : \phi, Y : \phi\,\mathbf{suf}(\phi \wedge \mathit{false}\,\mathbf{suf}\,\psi) \Rightarrow Y : \phi \wedge \phi\,\mathbf{suf}\,\psi, Y : \phi$.
(viii.b4) $Y : \phi, Y : \phi\,\mathbf{suf}(\phi \wedge \mathit{false}\,\mathbf{suf}\,\psi) \Rightarrow Y : \phi \wedge \phi\,\mathbf{suf}\,\psi, Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi)$.

The cases (viii.b1) and (viii.b3) can be proved trivially by (ax).

For case (viii.b2), by $(\wedge R)$ on $Y : \phi \wedge \phi\,\mathbf{suf}\,\psi$, the branch that contains $Y : \phi$ on the right side can be easily closed since there is a $Y : \phi$ on the left side. Thus, the only branch that needs to be proved is:

$$Y : \phi \wedge \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Y : \phi\,\mathbf{suf}\,\psi, Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi).$$

By $(\wedge L)$ and (wk), we need:

$$(viii.b21)\ Y : \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Y : \phi\,\mathbf{suf}\,\psi, Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi)$$

From (viii.b21), by (wk) on the right side we drop $Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi)$, so that it suffices to prove

$$(viii.b211)\ \ Y : \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Y : \phi\,\mathbf{suf}\,\psi.$$

Without loss of generality, let $Y = z \diamond Z$. Then, by (**suf** L) on $Y : \mathit{false}\,\mathbf{suf}\,\psi$ and (**suf** R) on $Y : \phi\,\mathbf{suf}\,\psi$, we obtain

(viii.b2111) $Z : \psi \Rightarrow Z : \psi, Z : \phi$.
(viii.b2112) $Z : \psi \Rightarrow Z : \psi, Z : \phi\,\mathbf{suf}\,\psi$.
(viii.b2113) $Z : \mathit{false}, Z : \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Z : \psi, Z : \phi$.
(viii.b2114) $Z : \mathit{false}, Z : \mathit{false}\,\mathbf{suf}\,\psi \Rightarrow Z : \psi, Z : \phi\,\mathbf{suf}\,\psi$.

The cases (viii.b2111) and (viii.b2112) are closed directly by (ax), since they carry $Z : \psi$ on both sides. The cases (viii.b2113) and (viii.b2114) are closed as well, since they both carry $Z : \mathit{false}$ on the left side.

For the case (viii.b4), by (wk) on both sides, we need:

$$(viii.b41)\ \ Y : \phi\,\mathbf{suf}(\phi \wedge \mathit{false}\,\mathbf{suf}\,\psi) \Rightarrow Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi),$$

which is exactly (viii.b), forming a back-link. The derivation path from (viii.b) to (viii.b41) is progressing, as the rule (**suf** R) is applied from (viii.b) to (viii.b4), where $(X : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi), Y : \phi\,\mathbf{suf}(\phi \wedge \phi\,\mathbf{suf}\,\psi))$ is the progressing step. This makes the whole derivation from (viii.b) a cyclic proof.

□

Proposition A.9 (Rule (ix)). *Rule (ix):* $\mathbf{f}\,\mathbf{n}\,\mathit{true} \leftrightarrow \mathit{false}$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. Recall that $\mathbf{n}\,\mathit{true} =_{df} \mathit{false}\,\mathbf{suf}\,\mathit{true}$. We need to prove

$(ix.a)$ $X : \mathbf{f}(\mathit{false}\,\mathbf{suf}\,\mathit{true}) \Rightarrow X : \mathit{false}$.
$(ix.b)$ $X : \mathit{false} \Rightarrow X : \mathbf{f}(\mathit{false}\,\mathbf{suf}\,\mathit{true})$.

Case for proving $(ix.a)$: Without loss of generality, let $X = x \diamond Y$. By the rule $(\mathbf{f})$ on $x \diamond Y : \mathbf{f}(\mathit{false}\,\mathbf{suf}\,\mathit{true})$:

$$x : \mathit{false}\,\mathbf{suf}\,\mathit{true} \Rightarrow x \diamond Y : \mathit{false}.$$

By (**suf** x), the sequent is closed.

Case for proving $(ix.b)$: This is trivially closed since $X : \mathit{false}$ is on the left side.

□

Proposition A.10 (Rule (x)). *Rule (x):* $\neg\phi \wedge \mathbf{f}\,\phi \longrightarrow \mathbf{n}\ \mathit{true}$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We need to prove

$$X : \neg\phi \wedge \mathbf{f}\,\phi \Rightarrow X : \mathbf{n}\ \mathit{true}.$$

That is, $X : \neg\phi \wedge \mathbf{f}\,\phi \Rightarrow X : \mathit{false}\,\mathbf{suf}\ \mathit{true}$.

Without loss of generality, let $X = x \diamond Y$. By $(\wedge L)$ on the left:

$$x \diamond Y : \neg\phi, x \diamond Y : \mathbf{f}\,\phi \Rightarrow x \diamond Y : \mathit{false}\,\mathbf{suf}\ \mathit{true}.$$

By the rule (**f**) on $x \diamond Y : \mathbf{f}\,\phi$:

$$x \diamond Y : \neg\phi, x : \phi \Rightarrow x \diamond Y : \mathit{false}\,\mathbf{suf}\ \mathit{true}.$$

If $Y = \varepsilon$, from the left we have both $x : \neg\phi$ and $x : \phi$, which can be closed by (ax).

If $Y \neq \varepsilon$, by (**suf** R) on $x \diamond Y : \mathit{false}\,\mathbf{suf}\ \mathit{true}$ on the right side, we obtain the two branches:

(x.1) $x \diamond Y : \neg\phi, x : \phi \Rightarrow Y : \mathit{true}, Y : \mathit{false}$.

(x.2) $x \diamond Y : \neg\phi, x : \phi \Rightarrow Y : \mathit{true}, Y : \mathit{false}\,\mathbf{suf}\ \mathit{true}$.

Both branches are terminal, since each of them carries $Y : \mathit{true}$ on the right side, which is always provable.

□

Proposition A.11 (Rule (xi)). *Rule (xi):* $p \leftrightarrow \mathbf{f}\,p$*, where* p *is an atomic proposition, is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We need to prove

$(xi.a)$ $X : p \Rightarrow X : \mathbf{f}\,p$.

$(xi.b)$ $X : \mathbf{f}\,p \Rightarrow X : p$.

We prove only (xi.a). The other direction is similar.

Case for proving $(xi.a)$: Without loss of generality, let $X = x \diamond Y$. By the rule (**f**) on $x \diamond Y : \mathbf{f}\,p$ on the right:

$$x \diamond Y : p \Rightarrow x : p.$$

By the rule (p) on $x \diamond Y : p$ on the left:

$$x : p \Rightarrow x : p,$$

which is immediately closed by (ax). □

Proposition A.12 (Rule (xii)). *Rule (xii):* $\mathbf{f}\,\phi \wedge \langle\alpha\rangle\psi \leftrightarrow \langle\alpha\rangle(\mathbf{f}\,\phi \wedge \psi)$ *is derived in G3PPL.*

Proof. Let $X \in \mathit{TVar}$. We prove the following two directions by structural induction on α. Throughout, recall that $\langle\alpha\rangle\theta =_{df} \neg[\alpha]\neg\theta$.

$(xii.a)$ $X : \mathbf{f}\,\phi \wedge \langle\alpha\rangle\psi \Rightarrow X : \langle\alpha\rangle(\mathbf{f}\,\phi \wedge \psi)$.

$(xii.b)$ $X : \langle\alpha\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha\rangle\psi$.

Case for proving $(xii.a)$: By $(\wedge L)$ on the left, we need to show:

$$X : \mathbf{f}\,\phi, X : \langle\alpha\rangle\psi \Rightarrow X : \langle\alpha\rangle(\mathbf{f}\,\phi \wedge \psi).$$

Case $\alpha = a$ **(atomic action).** Let $X = Y \diamond x$. We need:

$$(xii.a.a)\ \ Y \diamond x : \mathbf{f}\,\phi, Y \diamond x : \langle a\rangle\psi \Rightarrow Y \diamond x : \langle a\rangle(\mathbf{f}\,\phi \wedge \psi)$$

By $(\langle a\rangle R)$ and $(\langle a\rangle L)$, it suffices to show, for a new y:

$$Y \diamond x : \mathbf{f}\,\phi, Y \diamond x \diamond y : \psi \Rightarrow Y \diamond x \diamond y : \mathbf{f}\,\phi \wedge \psi$$

By $(\wedge R)$ on the right, we need $Y \diamond x \diamond y : \mathbf{f}\,\phi$ and $Y \diamond x \diamond y : \psi$. The latter is on the left. For the former, let $Y = z \diamond Z$. By $(\mathbf{f})$ on $Y \diamond x : \mathbf{f}\,\phi$ on the left, we obtain $z : \phi$; by $(\mathbf{f})$ on $Y \diamond x \diamond y : \mathbf{f}\,\phi$ on the right, we need $z : \phi$, which is available. This case is closed.

Case $\alpha = \chi?$ **(test).** Let $X = Y \diamond x$. We need:

$$(xii.a.\chi?)\ \ Y \diamond x : \mathbf{f}\,\phi, Y \diamond x : \langle\chi?\rangle\psi \Rightarrow Y \diamond x : \langle\chi?\rangle(\mathbf{f}\,\phi \wedge \psi)$$

By $(\langle\phi?\rangle L)$ on the left, we get $x \diamond x : \chi$ and $Y \diamond x \diamond x : \psi$, then by $(\langle\phi?\rangle R)$ on the right, we obtain:

$$Y \diamond x : \mathbf{f}\,\phi, Y \diamond x \diamond x : \psi \Rightarrow Y \diamond x \diamond x : (\mathbf{f}\,\phi \wedge \psi).$$

As in the case $\alpha = a$ above, we can close the proof.

Case $\alpha = \alpha_1 ; \alpha_2$ **(sequential composition).** We need to prove:

$$(xii.a.;)\ \ X : \mathbf{f}\,\phi, X : \langle\alpha_1 ; \alpha_2\rangle\psi \Rightarrow X : \langle\alpha_1 ; \alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$$

By $(\langle;\rangle)$, we need:

$$(xii.a.;.1)\ \ X : \mathbf{f}\,\phi, X : \langle\alpha_1\rangle\langle\alpha_2\rangle\psi \Rightarrow X : \langle\alpha_1\rangle\langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$$

From (xii.a.;.1), by applying (cut), we need:

(xii.a.;.11) $X : \mathbf{f}\,\phi, X : \langle\alpha_1\rangle\langle\alpha_2\rangle\psi \Rightarrow X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \langle\alpha_2\rangle\psi)$.
(xii.a.;.12) $X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \langle\alpha_2\rangle\psi) \Rightarrow X : \langle\alpha_1\rangle\langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$.

By the induction hypothesis on α_1, we close (xii.a.;.11). For (xii.a.;.12), by Necessitation Lemma we then need:

$$X : (\mathbf{f}\,\phi \wedge \langle\alpha_2\rangle\psi) \Rightarrow X : \langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$$

But this is straightforward by induction hypothesis on α_2.

Case $\alpha = \alpha_1 \cup \alpha_2$ **(choice).** We need to prove

$$(xii.a.\cup)\ \ X : \mathbf{f}\,\phi, X : \langle\alpha_1 \cup \alpha_2\rangle\psi \Rightarrow X : \langle\alpha_1 \cup \alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$$

By $(\langle\cup\rangle L)$ and $(\langle\cup\rangle R)$, we need to prove:

(xii.a.∪.1) $X : \mathbf{f}\,\phi, X : \langle\alpha_1\rangle\psi \Rightarrow X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \psi), X : \langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$
(xii.a.∪.2) $X : \mathbf{f}\,\phi, X : \langle\alpha_2\rangle\psi \Rightarrow X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \psi), X : \langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi)$

Both follow directly by weakening and the induction hypothesis. For example, from (xii.a.∪.1), by (wk), we need:

$$X : \mathbf{f}\,\phi, X : \langle\alpha_1\rangle\psi \Rightarrow X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \psi),$$

which is provable by induction hypothesis on α_1.

Case $\alpha = \alpha_1^*$ **(iteration).** We need to prove:

$$(xii.a.*)\ \ X : \mathbf{f}\,\phi, X : \langle\alpha_1^*\rangle\psi \Rightarrow X : \langle\alpha_1^*\rangle(\mathbf{f}\,\phi \wedge \psi)$$

By $(\langle * \rangle L)$, we split (xii.a.$*$) into:

(xii.a.$*$.1) $X : \mathbf{f}\,\phi, X : \langle \mathit{true}? \rangle \psi \Rightarrow X : \langle \alpha_1^* \rangle (\mathbf{f}\,\phi \wedge \psi)$

(xii.a.$*$.2) $X : \mathbf{f}\,\phi, X : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \psi \Rightarrow X : \langle \alpha_1^* \rangle (\mathbf{f}\,\phi \wedge \psi)$

Case (xii.a.$*$.1): By $(\langle * \rangle R)$ on the right, it suffices to show

$$X : \mathbf{f}\,\phi, X : \langle \mathit{true}? \rangle \psi \Rightarrow X : \langle \mathit{true}? \rangle (\mathbf{f}\,\phi \wedge \psi).$$

Let $X = Y \diamond x$. By $(\langle \phi? \rangle L)$ and $(\langle \phi? \rangle R)$, we have:

$$X : \mathbf{f}\,\phi, Y \diamond x \diamond x : \psi \Rightarrow Y \diamond x \diamond x : (\mathbf{f}\,\phi \wedge \psi).$$

This again is the case proved above when $\alpha = a$.

Case (xii.a.$*$.2): By $(\langle * \rangle R)$ on the right, it suffices to show

$$(\mathit{xii.a.*.21}) \quad X : \mathbf{f}\,\phi, X : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \psi \Rightarrow X : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle (\mathbf{f}\,\phi \wedge \psi).$$

From (xii.a.$*$.21), by (cut), we need to prove:

(xii.a.$*$.211) $X : \mathbf{f}\,\phi, X : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \psi \Rightarrow X : \langle \alpha_1 \rangle (\mathbf{f}\,\phi \wedge \langle \alpha_1^* \rangle \psi)$. It can be closed directly by induction hypothesis on α_1.

(xii.a.$*$.212) $X : \langle \alpha_1 \rangle (\mathbf{f}\,\phi \wedge \langle \alpha_1^* \rangle \psi) \Rightarrow X : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle (\mathbf{f}\,\phi \wedge \psi)$.

For (xii.a.$*$.212), by Necessitation Lemma, we obtain:

$$(\mathit{xii.a.*.2121}) \quad X : (\mathbf{f}\,\phi \wedge \langle \alpha_1^* \rangle \psi) \Rightarrow X : \langle \alpha_1^* \rangle (\mathbf{f}\,\phi \wedge \psi),$$

which is exactly (xii.a.$*$) (after applying rule $(\wedge L)$). The derivation path from (xii.a.$*$) to (xii.a.*.2121) is progressing because the Necessitation lemma is applied on this path.

Case for proving $(\mathit{xii.b})$: That is, $X : \langle \alpha \rangle (\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle \alpha \rangle \psi$.

Case $\alpha = a$ **(atomic action).** Let $X = Y \diamond x$. We need:

$$(\mathit{xii.b.a}) \quad Y \diamond x : \langle a \rangle (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \mathbf{f}\,\phi \wedge \langle a \rangle \psi$$

By $(\wedge R)$ on the right, we split into:

(xii.b.a.1) $Y \diamond x : \langle a \rangle (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \mathbf{f}\,\phi$.

By $(\langle a \rangle L)$, it suffices to show

$$x \,\mathrm{R}_a\, y, Y \diamond x \diamond y : (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \mathbf{f}\,\phi$$

with a new y. By $(\wedge L)$, we have $Y \diamond x \diamond y : \mathbf{f}\,\phi$. Let $Y = z \diamond Z$. By $(\mathbf{f})$ on $Y \diamond x \diamond y : \mathbf{f}\,\phi$ on the left, we get $z : \phi$. By $(\mathbf{f})$ on $Y \diamond x : \mathbf{f}\,\phi$ on the right, we need $z : \phi$, which is available. This case is closed.

(xii.b.a.2) $Y \diamond x : \langle a \rangle (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \langle a \rangle \psi$.

By the Necessitation lemma, we obtain $X : \mathbf{f}\,\phi \wedge \psi \Rightarrow X : \psi$, which is trivially derivable by $(\wedge L)$ and (ax).

Case $\alpha = \chi?$ **(test).** Let $X = Y \diamond x$. We need:

$$(\mathit{xii.b.}\chi?) \quad Y \diamond x : \langle \chi? \rangle (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \mathbf{f}\,\phi \wedge \langle \chi? \rangle \psi$$

By $(\langle \phi? \rangle L)$ on the left, we get $x \diamond x : \chi$ and $Y \diamond x \diamond x : \mathbf{f}\,\phi \wedge \psi$. By $(\wedge R)$ on the right:

(xii.b.χ?.1) $x \diamond x : \chi, Y \diamond x \diamond x : \mathbf{f}\,\phi \wedge \psi \Rightarrow Y \diamond x : \mathbf{f}\,\phi$. This case is similar to the proof of (xii.b.a.1) above.

(xii.b.χ?.2) $x \diamond x : \chi, Y \diamond x \diamond x : \mathbf{f}\,\phi \wedge \psi \Rightarrow Y \diamond x : \langle \chi? \rangle \psi$.

By $(\langle\phi?\rangle R)$ on the right, we need:

$$Y \diamond x \diamond x : \mathbf{f}\,\phi \wedge \psi \Rightarrow Y \diamond x \diamond x : \psi.$$

This is straightforward by $(\wedge L)$ and (ax).

Case $\alpha = \alpha_1 ; \alpha_2$ **(sequential composition).** We need:

$$(xii.b.;)\;\; X : \langle\alpha_1 ; \alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1 ; \alpha_2\rangle\psi$$

By $(\langle;\rangle)$, this becomes:

$$X : \langle\alpha_1\rangle\langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1\rangle\langle\alpha_2\rangle\psi$$

By (cut), we need to prove:

(xii.b.;.1) $X : \langle\alpha_1\rangle\langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \langle\alpha_2\rangle\psi)$. This is closed by the induction hypothesis for α_1 (with $\langle\alpha_2\rangle\psi$ in place of ψ).

(xii.b.;.2) $X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \langle\alpha_2\rangle\psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1\rangle\langle\alpha_2\rangle\psi$. By Necessitation Lemma we have

$$X : \langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_2\rangle\psi,$$

which is also closed by the induction hypothesis on α_2.

Case $\alpha = \alpha_1 \cup \alpha_2$ **(choice).** We need:

$$(xii.b.\cup)\;\; X : \langle\alpha_1 \cup \alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1 \cup \alpha_2\rangle\psi$$

By $(\langle\cup\rangle L)$, we need to prove:

(xii.b.∪.1) $X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1 \cup \alpha_2\rangle\psi$.

(xii.b.∪.2) $X : \langle\alpha_2\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1 \cup \alpha_2\rangle\psi$.

Take (xii.b.∪.1) as an example. The other case is similar. by (cut) we need:

(xii.b.∪.11) $X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1\rangle\psi$. It is closed by induction hypothesis on α_1.

(xii.b.∪.12) $X : \mathbf{f}\,\phi \wedge \langle\alpha_1\rangle\psi \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1 \cup \alpha_2\rangle\psi$. This can be easily proved by $(\wedge L)$, $(\wedge R)$, $(\langle\cup\rangle R)$ and (ax).

Case $\alpha = \alpha_1^*$ **(iteration).** We need:

$$(xii.b.*)\;\; X : \langle\alpha_1^*\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1^*\rangle\psi$$

By $(\langle*\rangle L)$, we need to consider two cases:

(xii.b.∗.1) $X : \langle \mathit{true}?\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1^*\rangle\psi$.

(xii.b.∗.2) $X : \langle\alpha_1\rangle\langle\alpha_1^*\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1^*\rangle\psi$.

For (xii.b.∗.1), let $X = Y \diamond x$. Then, by $(\langle\phi?\rangle L)$, $Y : \langle \mathit{true}?\rangle(\mathbf{f}\,\phi \wedge \psi)$ gives $x \diamond x : \mathit{true}$, $Y \diamond x \diamond x : (\mathbf{f}\,\phi \wedge \psi)$. By $(\wedge R)$, we need to prove:

(xii.b.∗.11) $x \diamond x : \mathit{true}$, $Y \diamond x \diamond x : (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \mathbf{f}\,\phi$. This can be obtained by applying $(\mathbf{f})$ on both sides since they both contain $\mathbf{f}\,\phi$ (after letting $Y = z \diamond Z$).

(xii.b.∗.12) $x \diamond x : \mathit{true}$, $Y \diamond x \diamond x : (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x : \langle\alpha_1^*\rangle\psi$. By $(\langle*\rangle R)$ on $Y \diamond x : \langle\alpha_1^*\rangle\psi$ and then $(\langle\phi?\rangle R)$, we obtain

$$Y \diamond x \diamond x : (\mathbf{f}\,\phi \wedge \psi) \Rightarrow Y \diamond x \diamond x : \psi$$

which is clearly closed since both sides contain $Y \diamond x \diamond x : \psi$.

For (xii.b.$*$.2), by (cut) we need to prove:

(xii.b.*.21) $X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \langle\alpha_1^*\rangle\psi) \Rightarrow X : \mathbf{f}\,\phi \wedge \langle\alpha_1\rangle\langle\alpha_1^*\rangle\psi$. This can be directly closed by the induction hypothesis for α_1.

(xii.b.*.22) $X : \langle\alpha_1\rangle\langle\alpha_1^*\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : \langle\alpha_1\rangle(\mathbf{f}\,\phi \wedge \langle\alpha_1^*\rangle\psi)$. By the Necessitation lemma, this gives

$$(xii.b.*.221)\quad X : \langle\alpha_1^*\rangle(\mathbf{f}\,\phi \wedge \psi) \Rightarrow X : (\mathbf{f}\,\phi \wedge \langle\alpha_1^*\rangle\psi),$$

which is exactly (xii.b.$*$). The derivaiton path from (xii.b.$*$) to (xii.b.$*$.221) is thus progressing because the Necessitation lemma is applied along this path.

□

Proposition A.13 (Rule (xiii)). *Rule (xiii):* $\mathbf{n}\langle\alpha\rangle\phi \leftrightarrow \mathbf{n}\ true \wedge \langle\alpha\rangle\,\bar{\mathbf{n}}\,\phi$ *is derived in G3PPL.*

Proof. Let $X \in TVar$. Recall that $\mathbf{n}\,\theta =_{df} false\ \mathbf{suf}\ \theta$ and $\bar{\mathbf{n}}\,\theta =_{df} \neg\,\mathbf{n}\,\neg\theta = \neg(false\ \mathbf{suf}\ \neg\theta)$. We prove the following two directions by structural induction on α.

$(xiii.a)$ $X : \mathbf{n}\langle\alpha\rangle\phi \Rightarrow X : \mathbf{n}\ true \wedge \langle\alpha\rangle\,\bar{\mathbf{n}}\,\phi$.

$(xiii.b)$ $X : \mathbf{n}\ true \wedge \langle\alpha\rangle\,\bar{\mathbf{n}}\,\phi \Rightarrow X : \mathbf{n}\langle\alpha\rangle\phi$.

Every trace formula occurring in this proof is of the form $false\ \mathbf{suf}\ \theta$. Below we therefore record, once and for all, how the rules ($\mathbf{suf}\ L$) and ($\mathbf{suf}\ R$) act on such a formula. Both derived steps are used repeatedly in the rest of this proof.

($\mathbf{n}\,L$) Applying ($\mathbf{suf}\ L$) to $x \diamond \sigma : false\ \mathbf{suf}\ \theta$ produces the two premises $\Gamma, \sigma : \theta \Rightarrow \Delta$ and $\Gamma, \sigma : false, \sigma : false\ \mathbf{suf}\ \theta \Rightarrow \Delta$. The latter is terminal, since it carries $\sigma : false$ on the left side. Hence it suffices to replace $x \diamond \sigma : false\ \mathbf{suf}\ \theta$ on the left side by $\sigma : \theta$.

($\mathbf{n}\,R$) Applying ($\mathbf{suf}\ R$) to $x \diamond \sigma : false\ \mathbf{suf}\ \theta$ produces the two premises $\Gamma \Rightarrow \sigma : \theta, \sigma : false, \Delta$ and $\Gamma \Rightarrow \sigma : \theta, \sigma : false\ \mathbf{suf}\ \theta, \Delta$. Both of them carry $\sigma : \theta$ on the right side, so after (wk) it suffices to replace $x \diamond \sigma : false\ \mathbf{suf}\ \theta$ on the right side by $\sigma : \theta$.

Intuitively, ($\mathbf{n}\,L$) and ($\mathbf{n}\,R$) express that on a trace $x \diamond \sigma$ the formula $\mathbf{n}\,\theta$ simply asserts θ on the immediate suffix σ, the second disjunct of the unfolding being blocked by *false*.

Case for proving $(xiii.a)$: That is, $X : false\ \mathbf{suf}\langle\alpha\rangle\phi \Rightarrow X : (false\ \mathbf{suf}\ true) \wedge \langle\alpha\rangle\neg(false\ \mathbf{suf}\ \neg\phi)$.

Without loss of generality, let $X = x \diamond Y$. By applying ($\mathbf{n}\,L$) to $x \diamond Y : false\ \mathbf{suf}\langle\alpha\rangle\phi$ on the left, we obtain:

$$Y : \langle\alpha\rangle\phi \Rightarrow x \diamond Y : (false\ \mathbf{suf}\ true) \wedge \langle\alpha\rangle\neg(false\ \mathbf{suf}\ \neg\phi)$$

By ($\wedge R$) on the right, we split into:

(xiii.a.1) $Y : \langle\alpha\rangle\phi \Rightarrow x \diamond Y : false\ \mathbf{suf}\ true$. By ($\mathbf{n}\,R$) on $x \diamond Y : false\ \mathbf{suf}\ true$ on the right, we need $Y : true$, which is always provable.

(xiii.a.2) $Y : \langle\alpha\rangle\phi \Rightarrow x \diamond Y : \langle\alpha\rangle\neg(false\ \mathbf{suf}\ \neg\phi)$.

For the case (xiii.a.2), we proceed by structural induction on α.

Case $\alpha = a$ **(atomic action).** Let $Y = Z \diamond y$. We need:

$$Z \diamond y : \langle a\rangle\phi \Rightarrow x \diamond Z \diamond y : \langle a\rangle\neg(false\ \mathbf{suf}\ \neg\phi).$$

By ($\langle a\rangle L$) on the left, we get $y\,\mathrm{R}_a\,z'$ and $Z \diamond y \diamond z' : \phi$, with z' fresh. By ($\langle a\rangle R$) on the right (with the same z'), we need $x \diamond Z \diamond y \diamond z' : \neg(false\ \mathbf{suf}\ \neg\phi)$. By ($\neg R$):

$$Z \diamond y \diamond z' : \phi, x \diamond Z \diamond y \diamond z' : false\ \mathbf{suf}\ \neg\phi \Rightarrow \cdot$$

By $(\mathbf{n}\,L)$ on $x \diamond Z \diamond y \diamond z' : \mathit{false}\,\mathbf{suf}\,\neg\phi$ on the left, we obtain $Z \diamond y \diamond z' : \neg\phi$. Then $Z \diamond y \diamond z' : \phi$ and $Z \diamond y \diamond z' : \neg\phi$ give a contradiction, closing the branch by $(\neg L)$ and (ax).

Case $\alpha = \chi?$ **(test).** Let $Y = Z \diamond y$. We need:

$$Z \diamond y : \langle \chi? \rangle \phi \Rightarrow x \diamond Z \diamond y : \langle \chi? \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi)$$

By $(\langle \phi? \rangle L)$ on the left, we get $y \diamond y : \chi$ and $Z \diamond y \diamond y : \phi$. By $(\langle \phi? \rangle R)$ on the right and then $(\neg R)$, we need:

$$Z \diamond y \diamond y : \phi, x \diamond Z \diamond y \diamond y : \mathit{false}\,\mathbf{suf}\,\neg\phi \Rightarrow \cdot.$$

By $(\mathbf{n}\,L)$, we obtain $Z \diamond y \diamond y : \neg\phi$ on the left side, which contradicts $Z \diamond y \diamond y : \phi$ and closes the branch.

Case $\alpha = \alpha_1 ; \alpha_2$ **(sequential composition).** We need to prove:

$$Y : \langle \alpha_1 ; \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 ; \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi).$$

By $(\langle ; \rangle)$:

$$Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi).$$

By (cut), we obtain:

(xiii.a.2.;.1) $Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\langle \alpha_2 \rangle \phi)$, which is closed by the induction hypothesis on α_1 (with $\langle \alpha_2 \rangle \phi$ in place of ϕ).

(xiii.a.2.;.2) $x \diamond Y : \langle \alpha_1 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\langle \alpha_2 \rangle \phi) \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi)$.

For (xiii.a.2.;.2), by Necessitation Lemma, it suffices to show:

$$(\mathit{xiii.a}.2.;.21)\quad x \diamond Y : \neg(\mathit{false}\,\mathbf{suf}\,\neg\langle \alpha_2 \rangle \phi) \Rightarrow x \diamond Y : \langle \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi).$$

By (cut) again, we need:

(xiii.a.2.;.211) $x \diamond Y : \neg(\mathit{false}\,\mathbf{suf}\,\neg\langle \alpha_2 \rangle \phi) \Rightarrow Y : \langle \alpha_2 \rangle \phi$. By $(\neg L)$ the left-side formula is moved to the right, and then $(\mathbf{n}\,R)$ replaces it by $Y : \neg\langle \alpha_2 \rangle \phi$. The right side then carries both $Y : \neg\langle \alpha_2 \rangle \phi$ and $Y : \langle \alpha_2 \rangle \phi$, so the branch is closed by $(\neg R)$ and (ax).

(xiii.a.2.;.212) $Y : \langle \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi)$. This is exactly provable by induction hypothesis on α_2.

Case $\alpha = \alpha_1 \cup \alpha_2$ **(choice).** We need

$$(\mathit{xiii.a}.2.\cup)\quad Y : \langle \alpha_1 \cup \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \cup \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi).$$

By $(\langle \cup \rangle L)$, we obtain

(xiii.a.2.∪.1) $Y : \langle \alpha_1 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \cup \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi)$.

(xiii.a.2.∪.2) $Y : \langle \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \cup \alpha_2 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi)$.

Take (xiii.a.∪.1) as an example. The other case is similar. By $(\langle \cup \rangle R)$ on the right we obtain

$$Y : \langle \alpha_1 \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi).$$

This is exactly the statement (xiii.a.2) with α_1 in place of α, hence it is closed by the induction hypothesis on α_1.

Case $\alpha = \alpha_1^*$ **(iteration).** We need to prove

$$(\mathit{xiii.a}.2.*)\quad Y : \langle \alpha_1^* \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1^* \rangle \neg(\mathit{false}\,\mathbf{suf}\,\neg\phi).$$

By $(\langle * \rangle L)$, we need

(xiii.a.2.*.1) $Y : \langle \mathit{true}? \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$.
(xiii.a.2.*.2) $Y : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$.

For (xiii.a.2.*.1), we obtain $Z \diamond y \diamond y : \phi$ by letting $Y = Z \diamond y$ and applying $(\langle \phi? \rangle L)$. By $(\langle * \rangle R)$ on the right, it suffices to show $x \diamond Y : \langle \mathit{true}? \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$, i.e., $x \diamond Z \diamond y \diamond y : \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$. By $(\neg R)$:

$$Z \diamond y \diamond y : \phi, x \diamond Z \diamond y \diamond y : \mathit{false}\, \mathbf{suf}\, \neg \phi \Rightarrow \cdot.$$

By $(\mathbf{n}\, L)$, we get $Z \diamond y \diamond y : \neg \phi$ on the left side. Contradiction with $Z \diamond y \diamond y : \phi$. Closed.

For (xiii.a.2.*.2): By $(\langle * \rangle R)$ on the right, it suffices to show

$$Y : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi).$$

By (cut), we need:

(xiii.a.2.*.21) $Y : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_1^* \rangle \phi)$, which is closed by the induction hypothesis on α_1.
(xiii.a.2.*.22) $x \diamond Y : \langle \alpha_1 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_1^* \rangle \phi) \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$.

From (xiii.a.2.*.22), by Necessitation Lemma, it suffices to show:

$$x \diamond Y : \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_1^* \rangle \phi) \Rightarrow x \diamond Y : \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi).$$

By $(\neg L)$:

$$\cdot \Rightarrow x \diamond Y : \mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_1^* \rangle \phi, x \diamond Y : \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi).$$

By $(\mathbf{n}\, R)$ on $x \diamond Y : \mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_1^* \rangle \phi$, this formula is replaced by $Y : \neg \langle \alpha_1^* \rangle \phi$ on the right side, i.e., by $(\neg R)$:

$$Y : \langle \alpha_1^* \rangle \phi \Rightarrow x \diamond Y : \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi),$$

which is exactly (xiii.a.2.*), forming a back-link. The derivation path is progressing since Necessitation Lemma is applied.

Case for proving $(\mathit{xiii.b})$: That is, $X : (\mathit{false}\, \mathbf{suf}\, \mathit{true}) \wedge \langle \alpha \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi) \Rightarrow X : \mathit{false}\, \mathbf{suf} \langle \alpha \rangle \phi$.

By $(\wedge L)$ on the left:

$$X : \mathit{false}\, \mathbf{suf}\, \mathit{true}, X : \langle \alpha \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi) \Rightarrow X : \mathit{false}\, \mathbf{suf} \langle \alpha \rangle \phi.$$

Without loss of generality, let $X = x \diamond Y$. By $(\mathbf{n}\, L)$ on $x \diamond Y : \mathit{false}\, \mathbf{suf}\, \mathit{true}$ on the left, this formula is replaced by $Y : \mathit{true}$, which is trivially satisfied and can be discarded by (wk). By $(\mathbf{n}\, R)$ on $x \diamond Y : \mathit{false}\, \mathbf{suf} \langle \alpha \rangle \phi$ on the right, we need $Y : \langle \alpha \rangle \phi$. So it suffices to show:

$$x \diamond Y : \langle \alpha \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi) \Rightarrow Y : \langle \alpha \rangle \phi$$

We prove this by structural induction on α.

Case $\alpha = a$ **(atomic action).** Let $Y = Z \diamond y$. We need:

$$x \diamond Z \diamond y : \langle a \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi) \Rightarrow Z \diamond y : \langle a \rangle \phi$$

By $(\langle a \rangle L)$ on the left, we get $y\, \mathrm{R}_a\, z'$ and $x \diamond Z \diamond y \diamond z' : \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$. By $(\langle a \rangle R)$ on the right (with the same z'), we need $Z \diamond y \diamond z' : \phi$. By $(\neg L)$ on $x \diamond Z \diamond y \diamond z' : \neg (\mathit{false}\, \mathbf{suf}\, \neg \phi)$, we need to show

$$\cdot \Rightarrow x \diamond Z \diamond y \diamond z' : \mathit{false}\, \mathbf{suf}\, \neg \phi, Z \diamond y \diamond z' : \phi.$$

By $(\mathbf{n}\, R)$ on $x \diamond Z \diamond y \diamond z' : \mathit{false}\, \mathbf{suf}\, \neg \phi$, this formula is replaced by $Z \diamond y \diamond z' : \neg \phi$. So we need $\Rightarrow Z \diamond y \diamond z' : \neg \phi, Z \diamond y \diamond z' : \phi$, which closes by $(\neg R)$ and (ax).

Case $\alpha = \chi?$ **(test).** Let $Y = Z \diamond y$. We need:

$$x \diamond Z \diamond y : \langle \chi? \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Z \diamond y : \langle \chi? \rangle \phi$$

By $(\langle \phi? \rangle L)$ on the left, we get $y \diamond y : \chi$ and $x \diamond Z \diamond y \diamond y : \neg(\mathit{false}\, \mathbf{suf}\, \neg\phi)$. By $(\langle \phi? \rangle R)$ on the right, we need:

$$x \diamond Z \diamond y \diamond y : \neg(\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Z \diamond y \diamond y : \phi.$$

By $(\neg L)$ on $x \diamond Z \diamond y \diamond y : \neg(\mathit{false}\, \mathbf{suf}\, \neg\phi)$, then $(\mathbf{n}\, R)$, this reduces to $\cdot \Rightarrow Z \diamond y \diamond y : \neg\phi, Z \diamond y \diamond y : \phi$, which closes by $(\neg R)$ and (ax).

Case $\alpha = \alpha_1 ; \alpha_2$ **(sequential composition).** We need to prove:

$$(xiii.b.;)\quad x \diamond Y : \langle \alpha_1 ; \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1 ; \alpha_2 \rangle \phi.$$

By $(\langle ; \rangle)$:

$$(xiii.b.;.1)\quad x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \phi.$$

By (cut), we obtain:

(xiii.b.;.11) $x \diamond Y : \langle \alpha_1 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_2 \rangle \phi) \Rightarrow Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \phi$. It is closed by induction hypothesis on α_1.

(xiii.b.;.12) $x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow x \diamond Y : \langle \alpha_1 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_2 \rangle \phi)$.

For (xiii.b.;.12), by the Necessitation lemma, we need

$$x \diamond Y : \langle \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow x \diamond Y : \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_2 \rangle \phi).$$

By (cut) again, this yields

(xiii.b.;.121) $x \diamond Y : \langle \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_2 \rangle \phi$, which is provable by induction hypothesis on α_2.

(xiii.b.;.122) $Y : \langle \alpha_2 \rangle \phi \Rightarrow x \diamond Y : \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_2 \rangle \phi)$, which can be proved by further extending $x \diamond Y : \neg (\mathit{false}\, \mathbf{suf}\, \neg \langle \alpha_2 \rangle \phi)$ using $(\mathbf{fus}\, R)$ and obtaining both $Y : \mathit{true}$ and $Y : \langle \alpha_2 \rangle \phi$ on the right.

Case $\alpha = \alpha_1 \cup \alpha_2$ **(choice).** We need to prove

$$(xiii.b.\cup)\quad x \diamond Y : \langle \alpha_1 \cup \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1 \cup \alpha_2 \rangle \phi.$$

By $(\langle \cup \rangle L)$, it gives

(xiii.b.∪.1) $x \diamond Y : \langle \alpha_1 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1 \cup \alpha_2 \rangle \phi$.

(xiii.b.∪.2) $x \diamond Y : \langle \alpha_2 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1 \cup \alpha_2 \rangle \phi$.

Take (xiii.b.∪.1) as an example, the other case is similar. By $(\langle \cup \rangle R)$ on the right, it suffices to show

$$x \diamond Y : \langle \alpha_1 \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1 \rangle \phi.$$

This follows directly from the induction hypothesis on α_1, closing the branch.

Case $\alpha = \alpha_1^*$ **(iteration).** We need to prove

$$(xiii.b.*)\quad x \diamond Y : \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1^* \rangle \phi.$$

By $(\langle * \rangle L)$, we need to prove:

(xiii.b.∗.1) $x \diamond Y : \langle \mathit{true}? \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1^* \rangle \phi$.

(xiii.b.∗.2) $x \diamond Y : \langle \alpha_1 \rangle \langle \alpha_1^* \rangle \neg (\mathit{false}\, \mathbf{suf}\, \neg\phi) \Rightarrow Y : \langle \alpha_1^* \rangle \phi$.

Subcase (xiii.b.$*$.1): Let $Y = Z \diamond y$. By $(\langle\phi?\rangle L)$, we can replace $x \diamond Y : \langle true?\rangle\neg(false\,\mathbf{suf}\,\neg\phi)$ with $y \diamond y : true$ and $x \diamond Z \diamond y \diamond y : \neg(false\,\mathbf{suf}\,\neg\phi)$. On the right side, by $(\langle*\rangle R)$, we obtain $Z \diamond y : \langle true?\rangle\phi$ and $Z \diamond y : \langle\alpha_1\rangle\langle\alpha_1^*\rangle\phi$. So it is sufficient to show

$$y \diamond y : true, x \diamond Z \diamond y \diamond y : \neg(false\,\mathbf{suf}\,\neg\phi) \Rightarrow Z \diamond y : \langle true?\rangle\phi.$$

By $(\langle\phi?\rangle R)$ we can replace $Z \diamond y : \langle true?\rangle\phi$ with $Z \diamond y \diamond y : \phi$. The branch is then closed by extending $x \diamond Z \diamond y \diamond y : \neg(false\,\mathbf{suf}\,\neg\phi)$ (using $(\neg L)$ and then $(\mathbf{n}\,R)$) and obtaining $Z \diamond y \diamond y : \neg\phi$ on the right.

Subcase (xiii.b.$*$.2): By $(\langle*\rangle R)$ on the right, it suffices to show

$$x \diamond Y : \langle\alpha_1\rangle\langle\alpha_1^*\rangle\neg(false\,\mathbf{suf}\,\neg\phi) \Rightarrow Y : \langle\alpha_1\rangle\langle\alpha_1^*\rangle\phi.$$

By (cut), this is equivalent to proving:

(xiii.b.$*$.21) $x \diamond Y : \langle\alpha_1\rangle\neg(false\,\mathbf{suf}\,\neg\langle\alpha_1^*\rangle\phi) \Rightarrow Y : \langle\alpha_1\rangle\langle\alpha_1^*\rangle\phi$. It is closed by the induction hypothesis for α_1.
(xiii.b.$*$.22) $x \diamond Y : \langle\alpha_1\rangle\langle\alpha_1^*\rangle\neg(false\,\mathbf{suf}\,\neg\phi) \Rightarrow x \diamond Y : \langle\alpha_1\rangle\neg(false\,\mathbf{suf}\,\neg\langle\alpha_1^*\rangle\phi)$.

From (xiii.b.$*$.22), by the Necessitation lemma, we need

$$x \diamond Y : \langle\alpha_1^*\rangle\neg(false\,\mathbf{suf}\,\neg\phi) \Rightarrow x \diamond Y : \neg(false\,\mathbf{suf}\,\neg\langle\alpha_1^*\rangle\phi)$$

By $(\neg R)$: $x \diamond Y : \langle\alpha_1^*\rangle\neg(false\,\mathbf{suf}\,\neg\phi), x \diamond Y : false\,\mathbf{suf}\,\neg\langle\alpha_1^*\rangle\phi \Rightarrow \cdot$. By $(\mathbf{n}\,L)$, we get $Y : \neg\langle\alpha_1^*\rangle\phi$ on the left side. By $(\neg L)$:

$$(xiii.b.*.221)\quad x \diamond Y : \langle\alpha_1^*\rangle\neg(false\,\mathbf{suf}\,\neg\phi) \Rightarrow Y : \langle\alpha_1^*\rangle\phi,$$

which is exactly (xiii.b.$*$), forming a back-link. The derivation path from (xiii.b.$*$) to (xiii.b.$*$.221) is progressing because the Necessitation lemma is applied.

□

Proposition A.14 (Rule (xiv)). *Rule (xiv):* $\langle\alpha\rangle true \rightarrow \mathbf{fin}$ *is derived in G3PPL. Recall that* $\mathbf{fin} =_{df} \bar{\mathbf{n}}\,false \vee (true\,\mathbf{suf}(true \wedge \bar{\mathbf{n}}\,false))$ *and* $\bar{\mathbf{n}}\,false = \neg\,\mathbf{n}\,\neg false = \neg(false\,\mathbf{suf}\,true)$.

Proof. Let $X \in TVar$. We prove:

$$(xiv.1)\quad X : \langle\alpha\rangle true \Rightarrow X : \bar{\mathbf{n}}\,false \vee (true\,\mathbf{suf}(true \wedge \bar{\mathbf{n}}\,false)).$$

Without loss of generality, let $X = x \diamond Y$.

If $Y = \varepsilon$, it suffices to prove

$$(xiv.11)\quad x : \langle\alpha\rangle true \Rightarrow x : \bar{\mathbf{n}}\,false$$

since $x : true\,\mathbf{suf}(true \wedge \bar{\mathbf{n}}\,false)$ is false and can be safely removed. By $(\neg R)$ and $(\mathbf{suf}\,x)$, (xiv.11) is closed.

If $Y \neq \varepsilon$, it suffices to prove

$$(xiv.12)\quad X : \langle\alpha\rangle true \Rightarrow X : (true\,\mathbf{suf}(true \wedge \bar{\mathbf{n}}\,false))$$

by weakening. By $(\mathbf{suf}\,R)$ on $x \diamond Y : true\,\mathbf{suf}(true \wedge \neg(false\,\mathbf{suf}\,true))$ on the right, we split into two sub-cases:

(xiv.121) $X : \langle\alpha\rangle true \Rightarrow Y : (true \wedge \neg(false\,\mathbf{suf}\,true)), Y : true$, which is closed since we have $Y : true$ on the right.
(xiv.122) $X : \langle\alpha\rangle true \Rightarrow Y : (true \wedge \neg(false\,\mathbf{suf}\,true)), Y : true\,\mathbf{suf}(true \wedge \neg(false\,\mathbf{suf}\,true))$.

For (xiv.122), by (wk) we obtain:

$$(xiv.1221)\quad X : \langle\alpha\rangle true \Rightarrow Y : true\,\mathbf{suf}(true \wedge \neg(false\,\mathbf{suf}\,true)).$$

Note that $X : \langle\alpha\rangle true \Rightarrow Y : \langle\alpha\rangle true$ is provable because, by the Necessitation lemma, this is equivalent to proving $X : true \Rightarrow Y : true$, which holds since $Y : true$ is on the right.

Based on $X : \langle\alpha\rangle true \Rightarrow Y : \langle\alpha\rangle true$, by (cut) on (xiv.1221), we only need to prove

$$(xiv.12111)\ \ Y : \langle\alpha\rangle true \Rightarrow Y : true\ \mathbf{suf}\ (true \wedge \neg(false\ \mathbf{suf}\ true)),$$

which is exactly (xiv.12) with Y in place of X, forming a back-link. The derivation path from (xiv.12) to (xiv.12111) is progressing, since the rule ($\mathbf{suf}\ R$) is applied from (xiv.12) to (xiv.122), where $(X : true\ \mathbf{suf}\ (true \wedge \bar{\mathbf{n}}\ false), Y : true\ \mathbf{suf}\ (true \wedge \bar{\mathbf{n}}\ false))$ is the progressing step.

□

Proposition A.15 (Rule (xv)). *Rule (xv):* $((\mathbf{n}\ \phi \rightarrow \phi)\ \mathbf{suf}\ \phi) \rightarrow \mathbf{n}\ \phi$ *is derived in G3PPL.*

Proof. Recall that $\mathbf{n}\ \phi =_{df} false\ \mathbf{suf}\ \phi$. So we need to prove:

$$(xv.1)\ \ X : ((false\ \mathbf{suf}\ \phi) \rightarrow \phi)\ \mathbf{suf}\ \phi \Rightarrow X : false\ \mathbf{suf}\ \phi.$$

By ($\neg L$) and ($\neg R$), we turn (xv.1) into

$$(xv.12)\ \ X : true\ \mathbf{fus}\ \neg\phi \Rightarrow X : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi.$$

Without loss of generality, let $X = x \diamond Y$. By applying ($\mathbf{fus}\ L$) and ($\mathbf{fus}\ R$) on both sides, we obtain the following 4 cases:

(xv.121) $Y : \neg\phi, Y : true \Rightarrow Y : \neg\phi$.

(xv.122) $Y : \neg\phi, Y : true\ \mathbf{fus}\ \neg\phi \Rightarrow Y : \neg\phi$.

(xv.123) $Y : \neg\phi, Y : true \Rightarrow Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi), Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi$.

(xv.124) $Y : \neg\phi, Y : true\ \mathbf{fus}\ \neg\phi \Rightarrow Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi), Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi$.

The cases (xv.121) and (xv.122) are closed directly by (ax), since they carry $Y : \neg\phi$ on both sides.

For the case (xv.123), by ($\wedge R$) on $Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)$ we split into:

(xv.1231) $Y : \neg\phi, Y : true \Rightarrow Y : false\ \mathbf{suf}\ \phi, Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi$.

(xv.1232) $Y : \neg\phi, Y : true \Rightarrow Y : \neg\phi, Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi$, which is closed by ($ax$).

In (xv.1231), $Y : false\ \mathbf{suf}\ \phi$ is in fact $Y : \neg(true\ \mathbf{fus}\ \neg\phi)$. So by ($\neg R$), we need

$$(xv.12311)\ Y : \neg\phi, Y : true, Y : true\ \mathbf{fus}\ \neg\phi \Rightarrow Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi.$$

From (xv.12311), by (wk), we need to prove:

$$(xv.123111)\ Y : true\ \mathbf{fus}\ \neg\phi \Rightarrow Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi,$$

which is exactly (xv.12), forming a back-link. The derivation path is progressing since the rule ($\mathbf{fus}\ R$) is applied from (xv.12) to (xv.123), with $(X : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi, Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi)$ the progressing step.

For the case (xv.124), by (wk) on both sides we obtain

$$(xv.1241)\ \ Y : true\ \mathbf{fus}\ \neg\phi \Rightarrow Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi,$$

which is exactly (xv.12) with Y in place of X, forming a back-link. The derivation path is progressing since the rule ($\mathbf{fus}\ R$) is applied from (xv.12) to (xv.124), with $(X : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi, Y : ((false\ \mathbf{suf}\ \phi) \wedge \neg\phi)\ \mathbf{fus}\ \neg\phi)$ the progressing step.

□

A.3 Completeness Proofs of G3FOPL

The proof of Lemma 7.2 requires two derived rules ($([*'])$ and $(\langle *' \rangle)$), whose unlabelled versions can be derived from the rules of FODL [17]. Below we show that they are derivable in G3FOPL.

Lemma A.1. *The following rules can be derived from G3FOPL:*

$$\frac{X : \Gamma \Rightarrow X : \varphi, X : \Delta \quad X : \varphi \Rightarrow X : [\alpha]\varphi \quad X : \varphi \Rightarrow X : \phi}{X : \Gamma \Rightarrow X : [\alpha^*]\phi, X : \Delta}\ ([*'])$$

$$\frac{X : \Gamma \Rightarrow X : \exists n \geq 0.\varphi(n), X : \Delta \quad X : \varphi(n+1) \Rightarrow X : \langle \alpha_1 \rangle \varphi(n) \quad X : \varphi(0) \Rightarrow X : \phi}{X : \Gamma \Rightarrow X : \langle \alpha^* \rangle \phi, X : \Delta}\ (\langle *' \rangle)$$

The proof of Lemma A.1 depends on Lemma A.2 below, which is itself established using Lemma 6.2 from Section 6. We therefore state and prove Lemma A.2 first.

Lemma A.2. *The following rules can be derived from G3FOPL:*

$$\frac{X : \phi \Rightarrow X : [\alpha]\phi}{X : \phi \Rightarrow X : [\alpha^*]\phi}\ (inv)$$

$$\frac{X : \phi(n+1) \Rightarrow X : \langle \alpha \rangle \phi(n)}{X : \phi(n) \Rightarrow X : \langle \alpha^* \rangle \phi(0)}\ (con)$$

In Lemma A.2, rule (inv) corresponds to the *loop invariance rule* of [17], while rule (con) corresponds to the *convergence rule* of [17].

Proof of Lemma A.2. The derivations for the rules (inv) and (con) are given below. Both rely on the rule $([*])$ to unfold the iteration and on the Necessitation lemma to handle modalities.

The derivation for rule (inv):

$$\dfrac{\dfrac{\text{Lemma 6.2}}{X : \phi \Rightarrow X : [true?]\phi} \quad \dfrac{X : \phi \Rightarrow X : [\alpha]\phi \quad \dfrac{X : \phi \Rightarrow X : [\alpha^*]\phi\ \text{(bud)}}{X : [\alpha]\phi \Rightarrow X : [\alpha][\alpha^*]\phi}\ nec}{X : \phi \Rightarrow X : [\alpha][\alpha^*]\phi}\ (cut)}{X : \phi \Rightarrow X : [\alpha^*]\phi}\ ([*]),$$

where from $X : \phi \Rightarrow X : [true?]\phi$ we apply Lemma 6.2.

The derivation for rule (con):

$$\dfrac{\dfrac{\dfrac{\dfrac{\text{Lemma 6.2}}{X : n = 0, X : \phi(0) \Rightarrow X : \langle true? \rangle \phi(0)}}{X : n = 0 \wedge \phi(0) \Rightarrow X : \langle true? \rangle \phi(0)}\ (\wedge L) \quad \dfrac{\text{Cont.}}{X : n \geq 1 \wedge \phi(n) \Rightarrow X : \langle \alpha \rangle \langle \alpha^* \rangle \phi(0)}\ (2)}{X : (n = 0 \wedge \phi(0)) \vee (n \geq 1 \wedge \phi(n)) \Rightarrow X : \langle \alpha \rangle \langle \alpha^* \rangle \phi(0), X : \langle true? \rangle \phi(0)}\ (\vee L)}{X : \phi(n) \Rightarrow X : \langle \alpha \rangle \langle \alpha^* \rangle \phi(0), X : \langle true? \rangle \phi(0)}\ (1)}{X : \phi(n) \Rightarrow X : \langle \alpha^* \rangle \phi(0)}\ (\langle * \rangle R),$$

$$\dfrac{X : \phi(n+1) \Rightarrow X : \langle \alpha \rangle \phi(n) \quad \dfrac{X : \phi(n) \Rightarrow X : \langle \alpha^* \rangle \phi(0)\ \text{(bud)}}{X : \langle \alpha \rangle \phi(n) \Rightarrow X : \langle \alpha \rangle \langle \alpha^* \rangle \phi(0)}\ nec}{\text{Cont.} : X : \phi(n+1) \Rightarrow X : \langle \alpha \rangle \langle \alpha^* \rangle \phi(0)}\ (cut),$$

where the intuition of derivation (1) is that $\phi(n)$ is logically equivalent to $(n = 0 \wedge \phi(0)) \vee (n \geq 1 \wedge \phi(n))$ by splitting n into two cases: $n = 0$ and $n \geq 1$. The intuition of derivation (2) is that $n \geq 1 \wedge \phi(n)$ is logically equivalent to $\phi(n+1)$ (where we assume $n \geq 0$). From $X : \phi(0) \Rightarrow X : \langle true?\rangle\phi(0)$, one can use Lemma 6.2 by first applying the rules $(\neg R)$ and $(\neg L)$ to transform the sequent into the required form.

The derivation (1) is obtained by (cut) and the fact that

$$(con.1)\ X : \phi(n) \Rightarrow X : (n = 0 \wedge \phi(0)) \vee (n \geq 1 \wedge \phi(n))$$

is provable. In fact, from $(con.1)$, by $(\vee R)$ and $(\wedge R)$, we need to prove:

(con.11) $X : \phi(n) \Rightarrow X : n = 0, X : n \geq 1$, which can be closed by (ter).
(con.12) $X : \phi(n) \Rightarrow X : n = 0, \phi(n)$, which is directly closed by (ax).
(con.13) $X : \phi(n) \Rightarrow X : \phi(0), X : n \geq 1$.
(con.14) $X : \phi(n) \Rightarrow X : \phi(0), X : \phi(n)$, which is directly proved by (ax).

For (con.13), consider two cases of n. If $n = 0$, then we need to prove

$$(con.131)\ X : \phi(0) \Rightarrow X : \phi(0), X : 0 \geq 1,$$

trivially closed by (ax). If $n \geq 1$, then by (wk), we need to prove

$$(con.132)\ \cdot \Rightarrow n \geq 1,$$

which is closed by (ter).

The details of derivation (2) are as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{X : n \geq 1, X : \phi(n) \Rightarrow X : \phi(n), X : \Theta}\,(ax)}{X : n \geq 1 \wedge \phi(n) \Rightarrow X : \phi(n), X : \Theta}\,(\wedge L) \quad \dfrac{}{X : n \geq 1 \wedge \phi(n), X : \Theta \Rightarrow X : \Theta}\,(ax)}{X : n \geq 1 \wedge \phi(n), X : (\phi(n-1+1) \to \Theta) \Rightarrow X : \Theta}\,(\to L)}{X : n \geq 1 \wedge \phi(n), X : \forall k.(\phi(k+1) \to \Theta) \Rightarrow X : \Theta}\,(\forall L) \quad \dfrac{\dfrac{X : \phi(n+1) \Rightarrow X : \Theta}{\cdot \Rightarrow X : \phi(n+1) \to \Theta}\,(\to R)}{\cdot \Rightarrow X : \forall k.(\phi(k+1) \to \Theta)}\,(\forall R)}{X : n \geq 1 \wedge \phi(n) \Rightarrow X : \langle\alpha\rangle\langle\alpha^*\rangle\phi(0)}\,(cut),$$

where let $\Theta =_{df} \langle\alpha\rangle\langle\alpha^*\rangle\phi(0)$.

□

We now give the derivations of the two rules stated in Lemma A.1.

Proof of Lemma A.1. **Derivation of rule $([*'])$ in Lemma A.1.** Starting from

$$(*'.a)\ X : \Gamma \Rightarrow X : [\alpha^*]\phi, X : \Delta,$$

by applying (cut), we need:

$(*'.a.1)$ $X : \Gamma \Rightarrow X : \varphi, X : [\alpha^*]\phi, X : \Delta$, from which by (wk) we obtain

$$(*'.a.11)\ X : \Gamma \Rightarrow X : \varphi, X : \Delta$$

$(*'.a.2)$ $X : \Gamma, X : \varphi \Rightarrow X : [\alpha^*]\phi, X : \Delta$.

From $(*'.a.2)$, by (cut) and (wk), we need:

$(*'.\text{a}.21)$ $X : \varphi \Rightarrow X : [\alpha^*]\varphi$, from which by rule (inv) in Lemma A.2, we have

$$(*'.a.211)\ \ X : \varphi \Rightarrow X : [\alpha]\varphi.$$

$(*'.\text{a}.22)$ $X : [\alpha^*]\varphi \Rightarrow X : [\alpha^*]\phi$, from which by Necessitation Lemma (Lemma 6.1), we have

$$(*'.a.221)\ \ X : \varphi \Rightarrow X : \phi$$

We have obtained the premises of the rule $([*'])$: $(*'.\text{a}.11)$, $(*'.\text{a}.211)$, and $(*'.\text{a}.221)$.

Derivation of rule $(\langle *' \rangle)$ in Lemma A.1. Starting from

$$(*'.b)\ \ X : \Gamma \Rightarrow X : \langle\alpha^*\rangle\phi, X : \Delta,$$

by applying (cut), we need:

$(*'.\text{b}.1)$ $X : \Gamma \Rightarrow X : \exists n.\varphi(n) \wedge n \geq 0, X : \langle\alpha^*\rangle\phi, X : \Delta$ (with n fresh), from which by (wk) we obtain

$$(*'.b.11)\ \ X : \Gamma \Rightarrow X : \exists n.\varphi(n) \wedge n \geq 0, X : \Delta$$

$(*'.\text{b}.2)$ $X : \Gamma, X : \exists n.\varphi(n) \wedge n \geq 0 \Rightarrow X : \langle\alpha^*\rangle\phi, X : \Delta$

From $(*'.\text{b}.2)$, by $(\exists L)$, $(\wedge L)$ and (wk), we have

$$(*'.b.21)\ \ X : \varphi(n) \Rightarrow X : \langle\alpha^*\rangle\phi.$$

By (cut), we need:

$(*'.\text{b}.211)$ $X : \varphi(n) \Rightarrow X : \langle\alpha^*\rangle\varphi(0)$, from which by rule (con) in Lemma A.2, we obtain

$$(*'.b.2111)\ \ X : \varphi(n+1) \Rightarrow X : \langle\alpha\rangle\varphi(n)$$

$(*'.\text{b}.212)$ $X : \langle\alpha^*\rangle\varphi(0) \Rightarrow X : \langle\alpha^*\rangle\phi$, from which, by Necessitation Lemma (Lemma 6.1), we need

$$(*'.b.2121)\ \ X : \varphi(0) \Rightarrow X : \phi$$

We have obtained the premises of the rule $(\langle *' \rangle)$: $(*'.\text{b}.11)$, $(*'.\text{b}.2111)$, and $(*'.\text{b}.2121)$. □

Proof of Lemma 7.2. We proceed by induction on the structure of the program α. We first consider the case in which op is $[\alpha]$; the case in which op is $\langle\alpha\rangle$ is similar except for star programs.

The base case is when α is an atomic program, namely a. Without loss of generality, let $X = Y \diamond x$. To prove $Y \diamond x : \phi^\flat \Rightarrow Y \diamond x : [a]\psi^\flat$, by the rule $([a]R)$, it is sufficient to prove

$$Y \diamond x : \phi^\flat, xR_a y \Rightarrow Y \diamond x \diamond y : \psi^\flat \tag{5}$$

By $\models (\phi^\flat \rightarrow [a]\psi^\flat)$, it is not hard to see that (5) is valid. So by (ter), we close the derivation.

Consider the case in which α is a sequential program, namely $\alpha_1 ; \alpha_2$. By the soundness of rule $([;])$, we obtain $\models X : \phi^\flat \Rightarrow X : [\alpha_1][\alpha_2]\psi^\flat$. By Lemma 7.1, there exists a pure arithmetical FOL formula $\varphi^\flat$ such that $\models [\alpha_2]\psi^\flat \leftrightarrow \varphi^\flat$. Hence $\models X : \varphi^\flat \Rightarrow X : [\alpha_2]\psi^\flat$ and $\models X : \phi^\flat \Rightarrow X : [\alpha_1]\varphi^\flat$. By induction hypothesis, we have $\vdash X : \varphi^\flat \Rightarrow X : [\alpha_2]\psi^\flat$ and $\vdash X : \phi^\flat \Rightarrow X : [\alpha_1]\varphi^\flat$. By Necessitation Lemma, since $\vdash X : \varphi^\flat \Rightarrow X : [\alpha_2]\psi^\flat$, $\vdash X : [\alpha_1]\varphi^\flat \Rightarrow X : [\alpha_1][\alpha_2]\psi^\flat$. Using this result together with $\vdash X : \phi^\flat \Rightarrow X : [\alpha_1]\varphi^\flat$ and the rule (cut), we obtain $\vdash X : \phi^\flat \Rightarrow X : [\alpha_1][\alpha_2]\psi^\flat$. The result follows directly from the rule $([;])$.

Consider the case in which α is a choice program, namely $\alpha_1 \cup \alpha_2$. By the soundness of rule ([∪]), we have both $\models X : \phi^\flat \Rightarrow X : [\alpha_1]\psi^\flat$ and $\models X : \phi^\flat \Rightarrow X : [\alpha_2]\psi^\flat$. By induction hypothesis, we have $\vdash X : \phi^\flat \Rightarrow X : [\alpha_1]\psi^\flat$ and $\vdash X : \phi^\flat \Rightarrow X : [\alpha_2]\psi^\flat$. Therefore $\vdash X : \phi^\flat \Rightarrow X : [\alpha_1 \cup \alpha_2]\psi^\flat$ by rule ([∪]).

The case when α is a star program, namely α_1^*. By Lemma 7.1, there exists an arithmetical FOL formula $\varphi^\flat$ such that $\models [\alpha_1^*]\psi^\flat \leftrightarrow \varphi^\flat$. Then from $\models X : \phi^\flat \Rightarrow X : [\alpha_1^*]\psi^\flat$, we also have $\models X : \phi^\flat \Rightarrow X : \varphi^\flat$. From $\models [\alpha_1^*]\psi^\flat \leftrightarrow \varphi^\flat$, by the soundness of the rules ([∗]), ([∪]), ([ϕ?]) and ([;]), it is not hard to see that $\models \varphi^\flat \leftrightarrow [\alpha_1^*]\psi^\flat \leftrightarrow \psi^\flat \wedge [\alpha_1][\alpha_1^*]\psi^\flat \leftrightarrow \psi^\flat \wedge [\alpha_1]\varphi^\flat$. From these logical equivalences we see that $\models \varphi^\flat \rightarrow [\alpha_1]\varphi^\flat$ and $\models \varphi^\flat \rightarrow \psi^\flat$. By induction hypothesis, from $\models X : \phi^\flat \Rightarrow X : \varphi^\flat$, $\models X : \varphi^\flat \Rightarrow X : [\alpha_1]\varphi^\flat$, and $\models X : \varphi^\flat \Rightarrow X : \psi^\flat$, we have $\vdash X : \phi^\flat \Rightarrow X : \varphi^\flat$, $\vdash X : \varphi^\flat \Rightarrow X : [\alpha_1]\varphi^\flat$ and $\vdash X : \varphi^\flat \Rightarrow X : \psi^\flat$. By rule ([∗′]) of Lemma A.1, we have $\vdash X : \phi^\flat \Rightarrow X : [\alpha_1^*]\psi^\flat$.

We now consider the case in which *op* is $\langle\alpha\rangle$. The cases in which α is an atomic program, a sequential composition, or a choice are similar to the corresponding cases in which *op* is $[\alpha]$, using the derived duals of the rules applied above. The only non-trivial case is when α is a star program α_1^*.

By Lemma 7.1 and the way of expressing regular programs in arithmetical FOL in [17], we know that for any $n \geq 0$, $\langle\alpha_1^n\rangle\psi^\flat$ can be expressed as a formula $\varphi^\flat(n)$, i.e., $\models \langle\alpha_1^n\rangle\psi^\flat \leftrightarrow \varphi^\flat(n)$. From the semantics of $\langle\alpha_1^*\rangle\psi^\flat$, it is easy to see that $\models \langle\alpha_1^*\rangle\psi^\flat \leftrightarrow \exists n \geq 0.\varphi^\flat(n)$. From $\models X : \phi^\flat \Rightarrow X : \langle\alpha_1^*\rangle\psi^\flat$, we have $\models X : \phi^\flat \Rightarrow X : \exists n \geq 0.\varphi^\flat(n)$. On the other hand, by the soundness of the rule ($\langle;\rangle$), we have $\models \varphi^\flat(n+1) \leftrightarrow \langle\alpha_1^{n+1}\rangle\psi^\flat \leftrightarrow \langle\alpha_1 ; \alpha_1^n\rangle\psi^\flat \leftrightarrow \langle\alpha_1\rangle\langle\alpha_1^n\rangle\psi^\flat \leftrightarrow \langle\alpha_1\rangle\varphi^\flat(n)$. From these logical equivalences we get that $\models \varphi^\flat(n+1) \rightarrow \langle\alpha_1\rangle\varphi^\flat(n)$. When $n = 0$, from $\models \langle\alpha_1^n\rangle\psi^\flat \leftrightarrow \varphi^\flat(n)$, we have $\models \langle\alpha_1^0\rangle\psi^\flat \leftrightarrow \langle \mathit{true}?\rangle\psi^\flat \leftrightarrow \psi^\flat \leftrightarrow \varphi^\flat(0)$. So $\models \varphi^\flat(0) \rightarrow \psi^\flat$. By induction hypothesis, from $\models X : \phi^\flat \Rightarrow X : \exists n \geq 0.\varphi^\flat(n)$, $\models X : \varphi^\flat(n+1) \Rightarrow X : \langle\alpha_1\rangle\varphi^\flat(n)$ and $\models X : \varphi^\flat(0) \Rightarrow X : \psi^\flat$, we have $\vdash X : \phi^\flat \Rightarrow X : \exists n \geq 0.\varphi^\flat(n)$, $\vdash X : \varphi^\flat(n+1) \Rightarrow X : \langle\alpha_1\rangle\varphi^\flat(n)$ and $\vdash X : \varphi^\flat(0) \Rightarrow X : \psi^\flat$. By rule $\langle*'\rangle$ of Lemma A.1, we obtain $\vdash X : \phi^\flat \Rightarrow X : \langle\alpha_1^*\rangle\psi^\flat$. □